MathAlgo

目录

8 数论 1

8.1 GCD、exGCD 与线性同余 1
8.2 素数、筛法、整数分解与约数 6
8.3 欧拉函数、莫比乌斯与积性函数 17
8.4 逆元、CRT 与模方程 50
8.5 离散对数、乘法阶与原根 58
8.6 floor_sum 与欧几里得类算法 86

9 组合数学与计数 110

9.1 排列组合与基础计数 110
9.2 经典数列与分拆 115
9.3 容斥与反演 122
9.4 置换与置换环 128

10 多项式、卷积与生成函数 133

10.1 卷积与变换基础 133
10.2 多项式基础 145
10.3 形式幂级数 149
10.4 多项式求值与插值 150
10.5 生成函数 153
10.6 线性递推 155
10.7 多项式高级技巧 158

11 线性代数、数值计算与数值类型 161

11.1 线性方程组 161
11.2 异或线性基 165
11.3 数值积分、方程与单纯形 171
11.4 Kirchhoff 矩阵树定理 176
11.5 矩阵求逆、行列式、最小二乘与最小范数 177

8. 数论

8.1 GCD、exGCD 与线性同余

GCD 与 LCM

- 求一组数全部相同模意义、平移后具有公共周期等问题时，经常需要对差分取 GCD：

```
i64 g{};
for (int i = 1; i < n; ++ i) g = gcd(g, a[i] - a[0]);
```

因为

$$x \mid (a_i - a_0)$$

等价于这些数模 x 同余

复杂度

一次

```
gcd(a, b)
```

时间复杂度为

$$O(\log \min(|a|, |b|))$$

额外空间为 $O(1)$
对 n 个绝对值不超过 V 的整数连续求 GCD，时间复杂度为

$$O(n \log V)$$

exGCD 与 Bézout 方程

返回

```
{g, x, y}
```

满足

$$ax + by = g$$

```
array<i64, 3> exgcd(i64 a, i64 b) {
    if (!b) return {a, 1, 0};
    auto [g, x, y] = exgcd(b, a % b);
    return {g, y, x - a / b * y};
}
```

若题目中 `a,b` 都保证非负，则返回的

$$g = \gcd(a, b) \geq 0$$

若允许任意符号并希望强制 `g > 0`，调用后统一处理即可：

```
auto [g, x, y] = exgcd(a, b);
if (g < 0) g = -g, x = -x, y = -y;
```

接口说明

- `exgcd(a,b)`：返回 `{g,x,y}`，满足

$$ax + by = g$$

- 对通常的非负输入，$g = \gcd(a, b)$
- 它不仅求 GCD，还给出 Bézout 系数 x, y
  Bézout 定理说明：对于 a, b 不全为 0，

$$ax + by = c$$

  存在整数解，当且仅当

$$\gcd(a, b) \mid c$$

  因此 exGCD 可以直接解决二元一次不定方程；这也是常见数论模板中 exGCD 的主要用途

注意事项

若

```
auto [g, x, y] = exgcd(a, b);
```

得到了

$$ax + by = g$$

那么方程

$$ax + by = c$$

在 $g \mid c$ 时的一组特解为

$$x_0 = x\frac{c}{g}, \qquad y_0 = y\frac{c}{g}.$$

全部整数解为

$$x = x_0 + \frac{b}{g}t$$

$$y = y_0 - \frac{a}{g}t$$

其中

$$t \in \mathbb{Z}$$

计算 `x * (c / g)` 等表达式时，若数值可能很大，应在乘法之前转成 `i128`：

```
i128 x0 = i128(x) * (c / g);
i128 y0 = i128(y) * (c / g);
```

如果后续还要求：

- $x > 0, y > 0$
- $L_x \le x \le R_x$
- 统计某个范围内解的数量
  只需要把上述通解代入区间，转化成对参数 t 的上下界。此时直接复用基础工具中的 `floor_div / ceil_div` 即可求正整数解

常见用法

求一组 Bézout 系数

```
auto [g, x, y] = exgcd(a, b);
// a * x + b * y = g
```

解 $ax + by = c$

```
auto [g, x, y] = exgcd(a, b);
if (c % g) {
```

```
    // 无整数解
} else {
    i128 k = c / g;
    i128 x0 = i128(x) * k;
    i128 y0 = i128(y) * k;
    // 全部解：
    // x = x0 + (b / g) * t
    // y = y0 - (a / g) * t
}
```

判断某个整数能否表示成 $ax + by$

只需要：

```
if (c % gcd(a, b) == 0) {
    // 可以
}
```

若只判断存在性，完全不需要调用 `exgcd()`

复杂度

`exgcd(a,b)` 与普通 Euclid 算法同阶：

$$O(\log \min(|a|, |b|))$$

时间
递归深度也是

$$O(\log \min(|a|, |b|))$$

线性同余

求解

$$ax \equiv b \pmod{m}$$

依赖 `exgcd()`
返回：

```
{x0, period}
```

表示全部整数解为

$$x \equiv x_0 \pmod{\text{period}}$$

若无解返回 `nullopt`

```
optional<pair<i64, i64>> linear_congruence(i64 a, i64 b, i64 m) {
    assert(m > 0);
    a %= m;
    b %= m;
    if (a < 0) a += m;
    if (b < 0) b += m;
    auto [g, x, y] = exgcd(a, m);
    if (b % g) return nullopt;
    i64 mod{m / g};
    i64 x0{i64(i128(x) * (b / g) % mod)};
    if (x0 < 0) x0 += mod;
    return pair{x0, mod};
}
```

把一个线性同余压缩成"最小非负解 + m/g 周期"的形式；相比直接枚举所有余数解，这更适合作为后续 CRT 等算法的底层接口

接口说明

```
auto sol = linear_congruence(a, b, m);
```

解决

$$ax \equiv b \pmod{m}, \quad m > 0$$

设

$$g = \gcd(a, m)$$

有解的充要条件是

$$g \mid b$$

若有解，返回

```
{x0, m / g}
```

其中

$$0 \le x_0 < \frac{m}{g}$$

全部整数解为

$$x = x_0 + k\frac{m}{g}, \quad k \in \mathbb{Z}$$

因此模原模数 m 看，恰好有 g 个不同解：

$$x_0, \quad x_0 + \frac{m}{g}, \quad \dots, \quad x_0 + (g-1)\frac{m}{g}$$

这里不直接枚举，是因为 `{x0, period}` 信息已经完整，而且更适合继续参与 CRT 合并

注意事项

`m` **必须为正**

接口要求：

```
m > 0
```

不要用负数模数

返回的第二项不是原模数

例如：

$$6x \equiv 8 \pmod{14}$$

有

$$g = \gcd(6, 14) = 2$$

所以结果的周期是

$$\frac{14}{2} = 7$$

也就是说：

$$x \equiv 6 \pmod{7}$$

模 14 下对应两个解：

$$x = 6,\ 13$$

`a=0` 也能处理

对于

$$0x \equiv b \pmod{m}$$

若

$$m \mid b$$

则任意整数都是解，函数返回：

```
{0, 1}
```

表示

$$x \equiv 0 \pmod{1}$$

即所有整数
否则返回 `nullopt`

乘法需要扩大类型

核心式子中：

```
x * (b / g)
```

即使最终已经对 `m / g` 取模，中间乘法仍可能溢出 `i64`，因此板子使用：

```
i128(x) * (b / g)
```

不要删掉这个扩大类型

常见用法

求一个线性同余

```
auto sol = linear_congruence(a, b, m);

if (!sol) {
    // 无解
} else {
    auto [x, mod] = *sol;
    // 全部解 x ≡ x0 (mod mod)
}
```

枚举 $[0, m)$ 中全部解

```
auto sol = linear_congruence(a, b, m);

if (sol) {
    auto [x, step] = *sol;
    for (i64 v = x; v < m; v += step) {
        // v 是一个解
    }
}
```

解的数量为

```
m / step
```

也就是

$$\gcd(a, m)$$

模逆元实际上是特殊情形

若

$$\gcd(a,m)=1$$

则

$$ax\equiv 1 \pmod m$$

唯一对应：

```
auto inv = linear_congruence(a, 1, m);
```

但正式的逆元接口、质数模快速幂求逆以及线性预处理逆元，仍统一放到后面的 8.4 逆元、CRT 与模方程，这里不重复

复杂度

主要耗时来自一次 `exgcd(a,m)`，因此时间复杂度为

$$O(\log m)$$

递归额外空间为

$$O(\log m)$$

若枚举模 m 下全部 $g=\gcd(a,m)$ 个不同余数解，则还需要

$$O(g)$$

时间

常用结论速查

这几条我建议直接放在文件最后，比单独做“数论公式”更容易赛场上找到：

$$ax+by=c \iff \gcd(a,b)\mid c$$

若 (x_0,y_0) 是一组解，则全部解：

$$x=x_0+\frac{b}{g}t,\quad y=y_0-\frac{a}{g}t,\quad t\in\mathbb{Z}$$

对于

$$ax\equiv b \pmod m$$

设

$$g=\gcd(a,m)$$

则：

- $g\nmid b$：无解；
- $g\mid b$：模 m 恰有 g 个不同解；
- 所有整数解可以压成一个同余类

  $$x\equiv x_0 \pmod{m/g}$$

  而

  $$a^{-1} \pmod m$$

  存在当且仅当

  $$\gcd(a,m)=1$$

8.2 素数、筛法、整数分解与约数

版本选择

场景	使用
少量整数，$\sqrt{x}$ 可接受	`is_prime_trial()` / `factor_trial()`
大量 $x \le N$ 的判素或分解	`LinearSieve`
大量 $x \le V$，但不能筛到 V	筛到 $\sqrt{V}$ 后使用 `factor_with_primes()`
筛很大的短区间 $[l, r)$	`segmented_primes()`
完整 `u64` 判素	`Rho::is_prime()`
完整 `u64` 分解	`Rho::factor()`
已有质因数分解，求约数	约数工具
在某个大数附近找少量质数	`PrimeSearch`，可选

8.2.1 试除判素与分解

适合只有少量整数、$\sqrt{n}$ 的枚举可以接受时使用

```
bool is_prime_trial(u64 n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    if (n % 3 == 0) return n == 3;
    for (u64 p = 5, d = 2; p <= n / p; p += d, d = 6 - d) {
        if (n % p == 0) return false;
    }
    return true;
}
vector<pair<u64, int>> factor_trial(u64 n) {
    vector<pair<u64, int>> fac;
    for (u64 p : {2ULL, 3ULL}) {
        if (n % p) continue;
        int e{};
        do n /= p, ++ e; while (n % p == 0);
        fac.push_back({p, e});
    }
    for (u64 p = 5, d = 2; p <= n / p; p += d, d = 6 - d) {
        if (n % p) continue;
        int e{};
        do n /= p, ++ e; while (n % p == 0);
        fac.push_back({p, e});
    }
    if (n > 1) fac.push_back({n, 1});
    return fac;
}
```

接口说明

- `is_prime_trial(n)`：试除判断 `n` 是否为质数
- `factor_trial(n)`：返回升序的 `{{p1,e1},{p2,e2},...}`，表示

$$n = \prod_i p_i^{e_i}$$

  `factor_trial()` 要求 $n \ge 1$；`n=1` 时返回空数组

注意事项

循环条件使用

```
p <= n / p
```

而不是 `p * p <= n`，避免无符号乘法溢出
这里只枚举 $2, 3$ 和之后的 $6k \pm 1$，比从 `2` 开始枚举所有整数常数明显更小
当查询较多时，不要重复调用 `factor_trial()`，应改用下一节的筛表

常见用法

```
u64 x;
cin >> x;
if (is_prime_trial(x)) {
    // x 是质数
}
auto fac = factor_trial(x);
for (auto [p, e] : fac) {
    cout << p << '^' << e << '\n';
}
```

复杂度

`is_prime_trial(n)` 和 `factor_trial(n)` 最坏均为

$$O(\sqrt{n})$$

时间，额外空间为 $O(\omega(n))$；其中 $\omega(n)$ 是不同质因子个数

8.2.2 线性筛、最小质因子与筛表分解

```
struct LinearSieve {
    int n;
    vector<int> minp, primes;
    explicit LinearSieve(int n_) : n(n_), minp(n + 1) {
        for (int i = 2; i <= n; ++ i) {
            if (!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
            }
            for (int p : primes) {
                if (1LL * i * p > n) break;
                minp[i * p] = p;
                if (p == minp[i]) break;
            }
        }
    }
    bool is_prime(int x) const { return x >= 2 && minp[x] == x; }
    // ===== 小范围分解 =====
    vector<pair<int, int>> factor(int x) const {
        vector<pair<int, int>> res;
        while (x > 1) {
            int p{minp[x]}, e{};
            do x /= p, ++ e; while (x % p == 0);
            res.push_back({p, e});
        }
        return res;
    }
    // ===== 只取不同质因子 =====
    vector<int> factor_distinct(int x) const {
        vector<int> res;
        while (x > 1) {
            int p{minp[x]};
            res.push_back(p);
            while (x % p == 0) x /= p;
        }
        return res;
    }
};
// 已筛出 sqrt(max_x) 内全部质数时，可分解远大于筛表范围的整数
template<class T>
vector<pair<T, int>> factor_with_primes(T n, const vector<int> &primes) {
    vector<pair<T, int>> res;
    for (int q : primes) {
        T p{T(q)};
        if (p > n / p) break;
        if (n % p) continue;
        int e{};
        do n /= p, ++ e; while (n % p == 0);
```

```
            res.push_back({p, e});
        }
        if (n > 1) res.push_back({n, 1});
        return res;
    }
```

接口说明

`LinearSieve(n)` 在线性时间内求出：

- `primes`：不超过 `n` 的全部质数，升序
- `minp[x]`：`x` 的最小质因子
- `is_prime(x)`：$O(1)$ 判素，要求 $0 \le x \le n$
- `factor(x)`：利用最小质因子分解，要求 $1 \le x \le n$
- `factor_distinct(x)`：只返回不同质因子
  `factor_with_primes(x, primes)` 用已经存在的质数表进行试除。它允许 `x` 大于筛表范围，但必须保证 `primes` 包含所有不超过原始 $\sqrt{x}$ 的质数

常见用法

大量 $x \le N$

```
LinearSieve sieve(N);
if (sieve.is_prime(x)) {}
auto fac = sieve.factor(x);
```

大量 $x \le 10^{14}$

只筛到 10^7：

```
LinearSieve sieve(10000000);
for (i64 x : queries) {
    auto fac = factor_with_primes(x, sieve.primes);
}
```

只要所有待分解整数都满足

$$x \le 10^{14}$$

质数表就已经覆盖到 $\sqrt{x}$

已经有最小质因子时只取不同质因子

```
auto ps = sieve.factor_distinct(x);
```

不要先 `factor()` 再手工去指数

复杂度

构造 `LinearSieve(n)`：

$$O(n)$$

时间，$O(n)$ 空间
`is_prime(x)`：

$$O(1)$$

`factor(x)` 与 `factor_distinct(x)`：

$$O(\Omega(x))$$

其中 $\Omega(x)$ 为计重数的质因子个数
`factor_with_primes(x, primes)` 最坏需要试除不超过 $\sqrt{x}$ 的全部质数，因此为

$$O(\pi(\sqrt{x}))$$

次取模

8.2.3 分段筛

依赖上一节提供的质数表

```
vector<i64> segmented_primes(i64 l, i64 r, const vector<int> &primes) {
    l = max<i64>(l, 2);
    if (l >= r) return {};
    vector<char> is_prime(r - l, true);
    for (i64 p : primes) {
        if (i128(p) * p >= r) break;
        i128 start = max<i128>(i128(p) * p, (i128(l) + p - 1) / p * p);
        for (i128 x = start; x < r; x += p) {
            is_prime[i64(x) - l] = false;
        }
    }
    vector<i64> res;
    for (i64 x = l; x < r; ++ x) {
        if (is_prime[x - l]) res.push_back(x);
    }
    return res;
}
```

接口说明

`segmented_primes(l,r,primes)` 返回半开区间

$$[l,r)$$

中的全部质数，结果递增
要求：

$$0 \le l \le r$$

并且传入的 `primes` 至少包含全部

$$p \le \sqrt{r-1}$$

的质数

注意事项

起点计算使用 `i128`：

```
(i128(l) + p - 1) / p * p
```

避免 `l + p - 1` 在大 `i64` 附近溢出

常见用法

```
i64 l, r;
cin >> l >> r;
int limit{int(sqrtl((long double)(r - 1))) + 1};
LinearSieve sieve(limit);
auto ps = segmented_primes(l, r, sieve.primes);
for (i64 p : ps) cout << p << '\n';
```

适合例如：

```
l ≈ 10^12
r - l ≈ 10^6
```

这种无法直接筛到 `r`、但区间本身不长的情况

复杂度

若上一节的质数表已经构造完成，本模块筛区间的时间约为

$$O((r-l)\log\log r + \pi(\sqrt{r}))$$

空间为

$$O(r-l)$$

若把预处理质数表也算入，则还需额外 $O(\sqrt{r})$ 时间与空间

8.2.4 Miller–Rabin 与 Pollard–Rho

依赖 8.1：

```
u64 mul_mod(u64 a, u64 b, u64 mod);
u64 qpow_mod(u64 a, u64 b, u64 mod);
```

以及：

```
using u64 = uint64_t;
using u128 = __uint128_t;
```

```
struct Rho {
    static u64 splitmix64(u64 &x) {
        u64 z{(x += 0x9e3779b97f4a7c15ULL)};
        z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL;
        z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL;
        return z ^ (z >> 31);
    }
    static u64 rng(u64 l, u64 r) {
        static u64 seed{u64(chrono::steady_clock::now().time_since_epoch().count())};
        return l + splitmix64(seed) % (r - l + 1);
    }
    // ===== Miller-Rabin: 若只需要判素，保留到这里即可 =====
    static bool is_prime(u64 n) {
        if (n < 2) return false;
        static constexpr u64 small[]{
            2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37
        };
        for (u64 p : small) {
            if (n % p == 0) return n == p;
        }
        int s{__builtin_ctzll(n - 1)};
        u64 d{(n - 1) >> s};
        auto witness = [&](u64 a) -> bool {
            u64 x{qpow_mod(a % n, d, n)};
            if (x == 1 || x == n - 1) return false;
            for (int i = 1; i < s; ++ i) {
                x = mul_mod(x, x, n);
                if (x == n - 1) return false;
            }
            return true;
        };
        static constexpr u64 bases[]{
            2, 325, 9375, 28178, 450775, 9780504, 1795265022
        };
        for (u64 a : bases) {
            if (a % n && witness(a)) return false;
        }
        return true;
    }
    // ===== Pollard-Rho: 若只需要判素，以下代码可直接删除 =====
    static u64 pollard(u64 n) {
        if (!(n & 1)) return 2;
        if (n % 3 == 0) return 3;
        while (true) {
            u64 c{rng(1, n - 1)}, y{rng(0, n - 1)};
            u64 g{1}, r{1}, x{}, ys{};
            auto f = [&](u64 v) -> u64 {
                return (u128(mul_mod(v, v, n)) + c) % n;
            };
            while (g == 1) {
                x = y;
```

```
                for (u64 i = 0; i < r; ++ i) y = f(y);
                u64 q{1};
                for (u64 k = 0; k < r && g == 1; k += 128) {
                    ys = y;
                    u64 len{min<u64>(128, r - k)};
                    for (u64 i = 0; i < len; ++ i) {
                        y = f(y);
                        u64 d{x > y ? x - y : y - x};
                        q = mul_mod(q, d, n);
                    }
                    g = gcd(q, n);
                }
                r <<= 1;
            }
            if (g == n) {
                do {
                    ys = f(ys);
                    u64 d{x > ys ? x - ys : ys - x};
                    g = gcd(d, n);
                } while (g == 1);
            }
            if (g != n) return g;
        }
    }
    static void dfs(u64 n, vector<u64> &fac) {
        if (n == 1) return;
        if (is_prime(n)) {
            fac.push_back(n);
            return;
        }
        u64 d{pollard(n)};
        dfs(d, fac);
        dfs(n / d, fac);
    }
    static vector<pair<u64, int>> factor(u64 n) {
        if (n <= 1) return {};
        vector<u64> a;
        dfs(n, a);
        sort(a.begin(), a.end());
        vector<pair<u64, int>> res;
        for (u64 p : a) {
            if (res.empty() || res.back().first != p) {
                res.push_back({p, 1});
            } else {
                ++ res.back().second;
            }
        }
        return res;
    }
    // ===== 只需要不同质因子时使用；否则可直接删除 =====
    static vector<u64> factor_distinct(u64 n) {
        if (n <= 1) return {};
        vector<u64> res;
        dfs(n, res);
        sort(res.begin(), res.end());
        res.erase(unique(res.begin(), res.end()), res.end());
        return res;
    }
};
```

接口说明

- `Rho::is_prime(n)`：判断完整 `u64` 范围内的整数是否为质数
- `Rho::factor(n)`：返回升序且合并指数的质因数分解
- `Rho::factor_distinct(n)`：只返回不同质因子
- `pollard()`、`dfs()`：内部接口，通常不直接调用

注意事项

七个 Miller–Rabin 底数：

```
2, 325, 9375, 28178, 450775, 9780504, 1795265022
```

与这里的完整 `u64` 范围、精确整数模乘配套使用
`Pollard-Rho` 是随机算法，因此运行时间有波动；公开的 `factor()` 已经排序，外部逻辑不要依赖递归搜索得到因子的先后顺序
`factor(1)` 返回空数组，表示 1 没有质因子
`0` 没有通常意义下的有限质因数分解，不应调用 `factor(0)`

常见用法

只判素

```
u64 x;
cin >> x;
if (Rho::is_prime(x)) {
    // x 是质数
}
```

只需要这个功能时，从 `pollard()` 开始的代码全部可以删除

分解完整 64 位整数

```
u64 x;
cin >> x;
auto fac = Rho::factor(x);
for (auto [p, e] : fac) {
    cout << p << '^' << e << '\n';
}
```

只需要不同质因子

```
auto ps = Rho::factor_distinct(x);
```

不要先调用 `factor()` 再自己去重

复杂度

Miller--Rabin 固定进行常数次模幂，每次模幂为

$$O(\log n)$$

次模乘，因此可视为

$$O(\log n)$$

的常数轮判素
Pollard--Rho 没有简单的严格竞赛复杂度保证。对找到最小质因子 p，常见期望复杂度约为

$$O(\sqrt{p})$$

次迭代
对最困难的“两个约为 $\sqrt{n}$ 的大质因子相乘”的情形，常用经验量级约为

$$O(n^{1/4}\operatorname{polylog} n)$$

递归额外空间为质因数分解深度

8.2.5 约数工具

所有函数接受已经合并指数的质因数分解：

```
{{p1,e1},{p2,e2},...}
```

```
template<class T>
vector<T> divisors_from_factor(const vector<pair<T, int>> &fac) {
    vector<T> divs{1};
```

```
    for (auto [p, e] : fac) {
        int old{int(divs.size())};
        T pw{1};
        for (int k = 1; k <= e; ++ k) {
            pw *= p;
            for (int i = 0; i < old; ++ i) {
                divs.push_back(divs[i] * pw);
            }
        }
    }
    return divs;
}
template<class T>
u64 divisor_count(const vector<pair<T, int>> &fac) {
    u64 res{1};
    for (auto [p, e] : fac) res *= e + 1;
    return res;
}
template<class T>
u128 divisor_sum(const vector<pair<T, int>> &fac) {
    u128 res{1};
    for (auto [p, e] : fac) {
        u128 pw{1}, sum{1};
        for (int i = 0; i < e; ++ i) {
            pw *= p;
            sum += pw;
        }
        res *= sum;
    }
    return res;
}
```

接口说明

若

$$n = \prod_i p_i^{e_i}$$

则：
`divisors_from_factor(fac)` 枚举全部正约数
`divisor_count(fac)` 计算

$$\tau(n) = \prod_i (e_i + 1)$$

`divisor_sum(fac)` 计算

$$\sigma(n) = \prod_i \left(1 + p_i + p_i^2 + \cdots + p_i^{e_i}\right)$$

空分解表示 $n = 1$，因此对应返回：

```
divisors = {1}
count    = 1
sum      = 1
```

注意事项

`divisors_from_factor()` **默认不排序**，这样枚举本身保持 $O(\tau(n))$
需要升序时：

```
auto ds = divisors_from_factor(fac);
sort(ds.begin(), ds.end());
```

`divisor_sum()` 使用 `u128` 减少中间溢出风险，但如果数学答案本身超过 `u128`，依然需要高精度

常见用法

从 Pollard-Rho 分解后枚举约数

```
auto fac = Rho::factor(x);
auto ds = divisors_from_factor(fac);
sort(ds.begin(), ds.end());
for (u64 d : ds) cout << d << ' ';
```

求约数个数和约数和

```
u64 cnt = divisor_count(fac);
u128 sum = divisor_sum(fac);
```

一次性求 $1\dots n$ 每个数的全部约数

这种场景甚至不用质因数分解：

```
vector<vector<int>> divisors(n + 1);
for (int d = 1; d ⩽ n; ++ d) {
    for (int x = d; x ⩽ n; x += d) {
        divisors[x].push_back(d);
    }
}
```

A^B 的约数

若

$$A = \prod_i p_i^{e_i}$$

则

$$A^B = \prod_i p_i^{e_i B}$$

因此只需把分解中的每个指数 `e` 替换为 `e * B`
例如：

$$\tau(A^B) = \prod_i (e_i B + 1)$$

求模意义下的约数和时，如果模数不保证与 $p_i - 1$ 互质，**不要直接使用**

$$\frac{p_i^{e_i B+1} - 1}{p_i - 1}$$

应改用不依赖逆元的等比数列快速求和

复杂度

若

$$\tau(n)$$

表示约数个数，则：
`divisors_from_factor()`：

$$O(\tau(n))$$

时间和空间
若额外排序：

$$O(\tau(n) \log \tau(n))$$

`divisor_count()`：

$$O(\omega(n))$$

`divisor_sum()`：

$$O(\Omega(n))$$

批量枚举 $1 \dots n$ 每个整数的约数：

$$\sum_{d=1}^{n} \left\lfloor \frac{n}{d} \right\rfloor = O(n \log n)$$

8.2.6 附近质数搜索（可选）

依赖 `Rho::is_prime()`

```
struct PrimeSearch {
    static optional<i64> next_prime(i64 x, i64 hi = numeric_limits<i64>::max()) {
        if (hi < 2 || x > hi) return nullopt;
        x = max<i64>(x, 2);
        if (x == 2) return 2;
        if (!(x & 1)) ++ x;
        while (x <= hi) {
            if (Rho::is_prime(u64(x))) return x;
            if (hi - x < 2) break;
            x += 2;
        }
        return nullopt;
    }
    static optional<i64> prev_prime(i64 x, i64 lo = 2) {
        lo = max<i64>(lo, 2);
        if (x < lo) return nullopt;
        if (x == 2) return 2;
        if (!(x & 1)) -- x;
        i64 low{max<i64>(lo, 3)};
        while (x >= low) {
            if (Rho::is_prime(u64(x))) return x;
            if (x - low < 2) break;
            x -= 2;
        }
        if (lo == 2) return 2;
        return nullopt;
    }
    static u64 distance(i64 a, i64 b) {
        i128 d{i128(a) - i128(b)};
        if (d < 0) d = -d;
        return u64(d);
    }
    static vector<i64> nearby_primes(
        i64 center, int need,
        i64 lo = 2, i64 hi = numeric_limits<i64>::max()
    ) {
        vector<i64> res;
        if (need <= 0 || lo > hi || hi < 2) return res;
        lo = max<i64>(lo, 2);
        optional<i64> left, right;
        if (center >= lo) left = prev_prime(min(center, hi), lo);
        if (center <= hi) right = next_prime(max(center, lo), hi);
        while (int(res.size()) < need && (left || right)) {
            if (left && right && *left == *right) {
                i64 p{*left};
                res.push_back(p);
                if (p > lo) left = prev_prime(p - 1, lo);
                else left.reset();
                if (p < hi) right = next_prime(p + 1, hi);
                else right.reset();
                continue;
            }
            bool take_left = left && (!right || distance(*left, center) <= distance(*right, center));
            if (take_left) {
                i64 p{*left};
                res.push_back(p);
```

```
                if (p > lo) left = prev_prime(p - 1, lo);
                else left.reset();
            } else {
                i64 p{*right};
                res.push_back(p);
                if (p < hi) right = next_prime(p + 1, hi);
                else right.reset();
            }
        }
        return res;
    }
};
```

接口说明

- `next_prime(x,hi)`：闭区间 $[x, hi]$ 中最小质数
- `prev_prime(x,lo)`：闭区间 $[lo, x]$ 中最大质数
- `nearby_primes(center,need,lo,hi)`：按与 `center` 的距离从小到大返回最多 `need` 个质数
- 距离相同时优先较小的质数
- 找不到时 `next_prime()` / `prev_prime()` 返回 `nullopt`

注意事项

它适合**找少量附近质数**
如果题目要求枚举一个长区间中的所有质数，应使用分段筛，而不是不断 Miller--Rabin
边界上的加减已经先检查剩余空间，因此允许一直搜索到 `LLONG_MAX` 附近

常见用法

```
i64 center{1'000'000'000'000'000'000LL};
auto p = PrimeSearch::next_prime(center);
if (p) cout << *p << '\n';
auto ps = PrimeSearch::nearby_primes(center, 5);
for (i64 x : ps) cout << x << '\n';
```

复杂度

设实际检查了 C 个奇数候选。
每个候选调用一次 Miller--Rabin，因此时间约为

$$O(C \log n)$$

除结果数组外，额外空间为

$$O(1)$$

8.3 欧拉函数、莫比乌斯与积性函数

解决问题： 计算欧拉函数 $\varphi(n)$、莫比乌斯函数 $\mu(n)$，批量筛出 φ/μ 及其前缀和；处理积性函数、狄利克雷卷积与整除关系上的 Zeta / Möbius 变换

单点欧拉函数与莫比乌斯函数

```
pair<i64, int> phi_mu(i64 n) {
    assert(n >= 1);
    i64 x{ n }, phi{ n };
    int mu{ 1 };
    for (i64 p = 2; p <= x / p; ++ p) {
        if (x % p) continue;
        phi = phi / p * (p - 1);
        x /= p;
        mu = -mu;
        if (x % p == 0) mu = 0;
        while (x % p == 0) x /= p;
    }
    if (x > 1) phi = phi / x * (x - 1), mu = -mu;
```

```
    return { phi, mu };
}
```

这里把 φ 和 μ 合成一次试除，避免需要二者时重复分解

接口说明

```
auto [phi, mu] = phi_mu(n);
```

要求

$$n \geq 1$$

返回：

$$\phi = \varphi(n), \qquad \mu = \mu(n)$$

欧拉函数定义为 $1 \sim n$ 中与 n 互质的整数个数：

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

若

$$n = \prod_{i=1}^{k} p_i^{e_i}$$

则

$$\varphi(n) = \prod_{i=1}^{k} p_i^{e_i - 1}(p_i - 1)$$

莫比乌斯函数定义为

$$\mu(1) = 1$$

若 n 含有平方质因子，则

$$\mu(n) = 0$$

否则若 n 含有 k 个不同质因子：

$$\mu(n) = (-1)^k$$

常用结论

对于质数 p：

$$\varphi(p) = p - 1$$

以及

$$\varphi(p^k) = p^{k-1}(p - 1)$$

对于 $n > 1$：

$$\sum_{d|n} \varphi(d) = n$$

更统一地：

$$\sum_{d|n} \varphi(d) = n$$

对所有 $n \geq 1$ 都成立
欧拉定理：

$$\gcd(a, m) = 1 \Longrightarrow a^{\varphi(m)} \equiv 1 \pmod{m}$$

因此在互质条件下：

$$a^b \equiv a^{b \bmod \varphi(m)} \pmod{m}$$

常见的扩展欧拉降幂形式为

$$a^b \bmod m = \begin{cases} a^{b \bmod \varphi(m)} \bmod m, & \gcd(a, m) = 1, \\ a^b \bmod m, & \gcd(a, m) \neq 1,\ b < \varphi(m), \\ a^{b \bmod \varphi(m) + \varphi(m)} \bmod m, & \gcd(a, m) \neq 1,\ b \geq \varphi(m) \end{cases}$$

莫比乌斯函数最重要的恒等式：

$$\sum_{d \mid n} \mu(d) = [n = 1]$$

其中 `[P]` 表示命题 P 成立时为 1，否则为 0

常见用法

只求欧拉函数：

```
auto [phi, mu] = phi_mu(n);
// 使用 phi
```

只求 Möbius：

```
auto [phi, mu] = phi_mu(n);
// 使用 mu
```

若整数已经在 8.2 中完成质因数分解，则不要再试除一次。直接由

```
vector<pair<u64, int>> fac;
```

计算：

```
u64 phi = n;
int mu = 1;
for (auto [p, e] : fac) {
    phi = phi / p * (p - 1);
    if (e > 1) mu = 0;
    else mu = -mu;
}
```

注意事项

`phi_mu(1)` 返回

```
{1, 1}
```

这是数学定义，不是特殊失败状态
若 n 已经大到试除不可接受，应先使用 8.2 的 Pollard--Rho 分解，而不是继续使用这个函数

复杂度

最坏时间复杂度：

$$O(\sqrt{n})$$

额外空间：

$$O(1)$$

线性筛 φ 与 μ

```
struct ArithmeticSieve {
    int n;
    vector<int> minp, primes, phi, mu;
    // ===== 前缀和：不需要莫反 / 杜教筛时可直接删除 =====
    vector<i64> sum_phi;
    vector<int> sum_mu;
    explicit ArithmeticSieve(int n_) : n(n_), minp(n + 1), phi(n + 1), mu(n + 1), sum_phi(n + 1), sum_mu(n + 1) {
        if (n >= 1) phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; ++ i) {
            if (!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : primes) {
                i64 v{ 1LL * i * p };
                if (v > n) break;
                minp[v] = p;
                if (p == minp[i]) {
                    phi[v] = phi[i] * p;
                    mu[v] = 0;
                    break;
                }
                phi[v] = phi[i] * (p - 1);
                mu[v] = -mu[i];
            }
        }
        // ===== 以下函数可直接删除：只在需要前缀和时保留 =====
        for (int i = 1; i <= n; ++ i) {
            sum_phi[i] = sum_phi[i - 1] + phi[i];
            sum_mu[i] = sum_mu[i - 1] + mu[i];
        }
    }
};
```

接口说明

构造：

```
ArithmeticSieve sieve(n);
```

成员变量：

成员	含义
`primes`	不超过 `n` 的全部质数
`minp[x]`	`x` 的最小质因子
`phi[x]`	$\varphi(x)$
`mu[x]`	$\mu(x)$
`sum_phi[x]`	$\sum_{i=1}^{x} \varphi(i)$
`sum_mu[x]`	$\sum_{i=1}^{x} \mu(i)$

合法查询范围：

$$1 \le x \le n$$

递推原理

若 $p \mid n$：

$$\varphi(np) = p\varphi(n)$$

且因为 np 出现平方质因子：

$$\mu(np) = 0$$

若

$$p \nmid n$$

则：

$$\varphi(np) = (p-1)\varphi(n)$$

以及

$$\mu(np) = -\mu(n)$$

因此在线性筛中，每个合数只由其最小质因子对应的状态转移一次

常见用法

查询：

```
ArithmeticSieve sieve(N);
cout << sieve.phi[x] << '\n';
cout << sieve.mu[x] << '\n';
```

区间 φ 和：

```
i64 sum = sieve.sum_phi[r] - sieve.sum_phi[l - 1];
```

区间 Möbius 和：

```
int sum = sieve.sum_mu[r] - sieve.sum_mu[l - 1];
```

只需要 `phi / mu`，不需要前缀和时，可以直接删除：

```
vector<i64> sum_phi;
vector<int> sum_mu;
```

以及构造函数最后的前缀循环

注意事项

这一份内部再次维护了 `minp / primes`，虽然与 8.2 的线性筛有少量重复，但这是**刻意保留的自包含依赖**：复制整个 8.3 线性筛后就能直接使用，不必再去拼 8.2
如果某道题已经使用了 8.2 的 `LinearSieve`，并且只需要少量额外信息，也可以直接在那份筛里加入 `phi / mu` 转移，不需要同时构造两个筛
`sum_phi[n]` 的数量级为 $O(n^2)$，因此必须使用 `i64`，不能跟 `phi` 一起使用 `int`

复杂度

构造：

$$O(n)$$

时间
空间：

$$O(n)$$

单点查询：

$$O(1)$$

前缀区间查询：

$$O(1)$$

通用积性函数线性筛

这一块属于**扩展功能，可整段删除**
若题目出现一个新的积性函数，而且已知其质数幂上的值

$$f(p^e),$$

则不必每次重新手写一套线性筛
`prime_power(p,e,pe)` 需要返回

$$f(p^e)$$

其中

$$pe = p^e$$

```
template<class T, class F>
vector<T> multiplicative_sieve(int n, F prime_power) {
    vector<int> minp(n + 1), primes, pp(n + 1), exp(n + 1);
    vector<T> f(n + 1);
    f[1] = T{1};
    for (int i = 2; i <= n; ++ i) {
        if (!minp[i]) {
            minp[i] = i;
            primes.push_back(i);
            pp[i] = i;
            exp[i] = 1;
            f[i] = prime_power(i, 1, i);
        }
        for (int p : primes) {
            i64 v{ 1LL * i * p };
            if (v > n) break;
            minp[v] = p;
            if (p == minp[i]) {
                pp[v] = pp[i] * p;
                exp[v] = exp[i] + 1;
                f[v] = f[i / pp[i]] * prime_power(p, exp[v], pp[v]);
                break;
            }
            pp[v] = p;
            exp[v] = 1;
            f[v] = f[i] * prime_power(p, 1, p);
        }
    }
    return f;
}
```

接口说明

要求 f 为积性函数，即：

$$\gcd(a,b)=1 \Longrightarrow f(ab)=f(a)f(b)$$

传入的回调：

```
prime_power(p, e, pe)
```

表示

$$pe = p^e$$

并返回

$$f(p^e)$$

函数返回：

```
f[x]
```

即所有

$$1 \le x \le n$$

的函数值

常见用法

欧拉函数

```
auto phi = multiplicative_sieve<i64>(n, [](i64 p, int e, i64 pe) {
    return pe / p * (p - 1);
});
```

莫比乌斯函数

```
auto mu = multiplicative_sieve<int>(n, [](int p, int e, int pe) {
    return e == 1 ? -1 : 0;
});
```

约数个数函数

若

$$\tau(p^e) = e + 1$$

则：

```
auto tau = multiplicative_sieve<i64>(n, [](i64 p, int e, i64 pe) {
    return i64(e + 1);
});
```

约数和函数

有

$$\sigma(p^e) = 1 + p + p^2 + \cdots + p^e$$

可以写：

```
auto sigma = multiplicative_sieve<i64>(n, [](i64 p, int e, i64 pe) {
    return (pe * p - 1) / (p - 1);
});
```

这里要求中间值 `pe * p` 不溢出
若有模数，则应自行写不依赖整数除法的质数幂公式

注意事项

这个通用版的价值是处理**临时出现的新积性函数**
对于 `phi / mu` 这种极常见函数，仍优先使用上一节专用 `ArithmeticSieve`：

- 更短
- 同时计算两个函数
- 能直接维护前缀和
- 常数也更小
  `prime_power()` 应为 $O(1)$。如果回调内部每次都重新快速幂，那么整个筛就不再是严格的 $O(n)$ 常数操作

复杂度

若 `prime_power()` 为 $O(1)$，则：

$$O(n)$$

时间

$$O(n)$$

空间

积性函数与狄利克雷卷积

基本定义

若算术函数 f 满足

$$f(1)=1$$

并且对于

$$\gcd(a,b)=1$$

有

$$f(ab)=f(a)f(b)$$

则称 f 为**积性函数**
若对任意 a,b 都有

$$f(ab)=f(a)f(b)$$

则称为**完全积性函数**
常见算术函数：

函数	定义
$\varepsilon(n)$	$[n=1]$
$\mathbf{1}(n)$	恒为 1
$\mathrm{id}(n)$	n
$\mathrm{id}_k(n)$	n^k
$\mu(n)$	Möbius 函数
$\varphi(n)$	欧拉函数
$\tau(n)$	正约数个数
$\sigma_k(n)$	$\sum_{d\mid n} d^k$

狄利克雷卷积定义为

$$(f*g)(n)=\sum_{d\mid n} f(d)g\left(\frac{n}{d}\right)$$

常用恒等式

单位元：

$$f*\varepsilon=f$$

Möbius 函数是常数函数 $\mathbf{1}$ 的 Dirichlet 逆：

$$\mu*\mathbf{1}=\varepsilon$$

欧拉函数：

$$\varphi=\mathrm{id}*\mu$$

因此：

$$\varphi(n) = \sum_{d|n} \mu(d) \frac{n}{d}$$

约数个数：

$$\tau = \mathbf{1} * \mathbf{1}$$

约数幂和：

$$\sigma_k = \mathrm{id}_k * \mathbf{1}$$

特别地：

$$\sigma_0 = \tau$$

两个积性函数的狄利克雷卷积仍然是积性函数

狄利克雷卷积

数组使用 `1..n`，下标 `0` 不参与计算

```
template<class T>
vector<T> dirichlet_conv(const vector<T> &a, const vector<T> &b) {
    int n{ int(min(a.size(), b.size())) - 1 };
    vector<T> c(n + 1);
    for (int i = 1; i ≤ n; ++ i) {
        for (int j = 1, ij = i; ij ≤ n; ++ j, ij += i) {
            c[ij] += a[i] * b[j];
        }
    }
    return c;
}
```

接口说明

返回

$$c = a * b$$

即：

$$c[n] = \sum_{d|n} a[d]\ b[n/d]$$

例如：

```
vector<i64> one(n + 1, 1);
one[0] = 0;
auto tau = dirichlet_convolution(one, one);
```

得到：

$$\tau = \mathbf{1} * \mathbf{1}$$

常见用法

验证：

```
for (int x = 1; x ≤ n; ++ x) {
    // c[x] = sum_{d | x} a[d] * b[x / d]
}
```

若运算在模意义下进行，可以直接令 `T` 为 `MInt`

注意事项

不要误认为这里是普通多项式卷积
普通卷积按照

$$i+j=n$$

组合下标
狄利克雷卷积按照

$$ij=n$$

组合下标
两者完全不同

复杂度

循环次数为

$$\sum_{i=1}^{n}\left\lfloor\frac{n}{i}\right\rfloor$$

因此时间复杂度为

$$O(n\log n)$$

空间:

$$O(n)$$

整除 Zeta 与 Möbius 变换

```cpp
// a[n] ← sum_{d | n} a[d]
template<class T>
void divisor_zeta(vector<T> &a) {
    int n{ int(a.size()) - 1 };
    for (int d = n; d >= 1; -- d) {
        for (int x = d + d; x <= n; x += d) a[x] += a[d];
    }
}
// divisor_zeta 的逆变换
template<class T>
void divisor_mobius(vector<T> &a) {
    int n{ int(a.size()) - 1 };
    for (int d = 1; d <= n; ++ d) {
        for (int x = d + d; x <= n; x += d) a[x] -= a[d];
    }
}
// a[d] ← sum_{d | n} a[n]
template<class T>
void multiple_zeta(vector<T> &a) {
    int n{ int(a.size()) - 1 };
    for (int d = 1; d <= n; ++ d) {
        for (int x = d + d; x <= n; x += d) a[d] += a[x];
    }
}
// multiple_zeta 的逆变换
template<class T>
void multiple_mobius(vector<T> &a) {
    int n{ int(a.size()) - 1 };
    for (int d = n; d >= 1; -- d) {
        for (int x = d + d; x <= n; x += d) a[d] -= a[x];
    }
}
```

接口说明

`divisor_zeta(a)`：

$$a'_n = \sum_{d|n} a_d$$

之后：

```
divisor_mobius(a);
```

可以恢复原数组
`multiple_zeta(a)`：

$$a'_d = \sum_{d|n} a_n$$

即统计所有 d 的倍数
`multiple_mobius()` 是它的逆操作

Möbius 反演

经典形式为：
若

$$F(n) = \sum_{d|n} f(d)$$

则

$$f(n) = \sum_{d|n} \mu(d) F\left(\frac{n}{d}\right)$$

本质上：

$$F = f * \mathbf{1}$$

两边卷积 μ：

$$F * \mu = f * \mathbf{1} * \mu = f$$

这就是 Möbius 反演

常见用法

求每个数的约数权值和

原数组：

```
vector<i64> f(n + 1);
```

执行：

```
divisor_zeta(f);
```

之后：

```
f[x]
```

就是

$$\sum_{d|x} f_{\text{old}}(d)$$

从约数和恢复原函数

```
divisor_mobius(f);
```

即可恢复

求所有倍数贡献

若需要：

$$g[d] = \sum_{d|x} f[x]$$

直接：

```
multiple_zeta(f);
```

互质指示函数

最常用的 Möbius 技巧：

$$[\gcd(a,b) = 1] = \sum_{d|\gcd(a,b)} \mu(d)$$

更一般地：

$$[\gcd(a,b) = k] = \sum_{d|\gcd(a/k,b/k)} \mu(d)$$

在 $k \mid a, b$ 时成立
很多“统计互质数对”的题，就是把 `gcd == 1` 换成上述约数求和，再交换求和顺序

注意事项

四个变换都是原地修改
例如：

```
divisor_zeta(a);
```

以后 `a` 已经不再保存原函数。如果后面仍需要原数组，应先复制
循环方向不能随便修改：

```
divisor_zeta       : d 从大到小
divisor_mobius     : d 从小到大
multiple_zeta      : d 从小到大
multiple_mobius    : d 从大到小
```

方向写反会让已经更新过的值再次参与贡献，产生重复计数
这里的 `divisor_mobius()` 是整除偏序上的 Möbius 逆变换；它与“直接把每个值乘上 `mu[i]`”不是一回事

复杂度

四个函数的循环次数都是调和级数：

$$\sum_{d=1}^{n} O\left(\frac{n}{d}\right) = O(n \log n)$$

因此时间复杂度：

$$O(n \log n)$$

除输入数组外额外空间：

$$O(1)$$

常用公式速查

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

$$\varphi(p^k) = p^{k-1}(p-1)$$

$$\sum_{d|n}\varphi(d)=n$$

$$\mu(n)=\begin{cases}1, & n=1,\\ 0, & \exists p^2\mid n,\\ (-1)^k, & n\text{ 含 }k\text{ 个不同质因子}\end{cases}$$

$$\sum_{d|n}\mu(d)=[n=1]$$

$$(f*g)(n)=\sum_{d|n}f(d)g(n/d)$$

$$\mu*\mathbf{1}=\varepsilon$$

$$\varphi=\mathrm{id}*\mu$$

$$\tau=\mathbf{1}*\mathbf{1}$$

$$\sigma_k=\mathrm{id}_k*\mathbf{1}$$

以及最常用的：

$$[\gcd(a,b)=1]=\sum_{d|\gcd(a,b)}\mu(d)$$

8.3.1 数论分块与莫比乌斯反演

解决问题： 将含有 $\lfloor n/i\rfloor$ 的求和压缩为 $O(\sqrt{n})$ 个整除块，并结合 Möbius 反演快速统计互质数对、$\gcd=k$ 数对以及各种 gcd 权值和。

数论分块

对于正整数 n，若当前左端点为 `l`：

```
for (i64 l = 1, r; l <= n; l = r + 1) {
    i64 q{ n / l };
    r = n / q;
    // i in [l, r] 时 n / i == q
}
```

若同时需要

$$\left\lfloor\frac{n}{i}\right\rfloor,\quad\left\lfloor\frac{m}{i}\right\rfloor$$

保持不变，则取两个右端点的最小值：

```
i64 lim{ min(n, m) };
for (i64 l = 1, r; l <= lim; l = r + 1) {
    i64 qn{ n / l }, qm{ m / l };
    r = min({ lim, n / qn, m / qm });
    // i in [l, r] 时 n / i、m / i 均不变
}
```

原理

固定

$$q=\left\lfloor\frac{n}{l}\right\rfloor$$

满足

$$\left\lfloor\frac{n}{i}\right\rfloor=q$$

的最大 i 为

$$r = \left\lfloor \frac{n}{q} \right\rfloor = \left\lfloor \frac{n}{\lfloor n/l \rfloor} \right\rfloor$$

因此可以将整个区间

$$[l, r]$$

一次处理
$\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 个不同取值：当 $i \le \sqrt{n}$ 时最多 $\sqrt{n}$ 个，而 $i > \sqrt{n}$ 时商小于 $\sqrt{n}$

常见用法

求

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor :$$

```
__int128_t sum_floor_div(i64 n) {
    __int128_t ans{};
    for (i64 l = 1, r; l ≤ n; l = r + 1) {
        i64 q{ n / l };
        r = n / q;
        ans += (__int128_t)(r - l + 1) * q;
    }
    return ans;
}
```

同时有恒等式：

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = \sum_{i=1}^{n} \tau(i)$$

因为左边统计的就是满足

$$ab \le n$$

的正整数对 (a, b) 数量

带权求和

若已经有前缀和

$$S(x) = \sum_{i=1}^{x} f(i)$$

要求

$$\sum_{i=1}^{n} f(i) G\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

则直接：

```
for (i64 l = 1, r; l ≤ n; l = r + 1) {
    i64 q{ n / l };
    r = n / q;
    ans += (pre[r] - pre[l - 1]) * calc(q);
}
```

这是后面 μ 前缀和、φ 前缀和参与数论分块时最常用的形式

注意事项

数论分块中的除法默认都是**正整数整除**。不要把同一写法直接用于负数，因为 C++ 的有符号整数除法和数学上的 floor division 不完全等价
单变量右端点：

```
r = n / (n / l);
```

多个整除项共同分块时，必须取各自右端点的最小值。比如：

```
r = min(n / (n / l), m / (m / l));
```

否则其中一个商可能已经改变
循环从 `l = 1` 开始；不要让 `n / l == 0` 后再计算：

```
n / (n / l)
```

否则会除以零
这里的“数论分块”与后面 8.6 的

$$\sum_i \left\lfloor \frac{ai+b}{m} \right\rfloor$$

型 `floor_sum` 不是同一个算法

复杂度

单个

$$\left\lfloor \frac{n}{i} \right\rfloor$$

的整除分块数量为

$$O(\sqrt{n})$$

因此每块 $O(1)$ 处理时，总时间为

$$O(\sqrt{n})$$

额外空间为

$$O(1)$$

两个整数 n, m 共同分块时，复杂度为块数，即至多

$$O(\sqrt{n}+\sqrt{m})$$

Möbius 反演与 gcd 条件

这一部分最值得记住的不是公式本身，而是一个固定套路
假设某个对象拥有参数 g，例如：

$$g = \gcd(x, y)$$

令：

$$F(d) = \#\{\text{对象} : d \mid g\}$$

以及：

$$f(k) = \#\{\text{对象} : g = k\}$$

显然：

$$F(d) = \sum_{d|k} f(k)$$

对“倍数关系”做 Möbius 反演：

$$f(d) = \sum_{t \geq 1} \mu(t) F(dt)$$

特别地：

$$f(1)=\sum_{t\geq 1}\mu(t)F(t)$$

所以当目标条件是

$$\gcd(x,y)=1$$

时，通常先把它改写成

$$[\gcd(x,y)=1]=\sum_{d\mid\gcd(x,y)}\mu(d)$$

这就是竞赛中 Möbius 反演处理 gcd 计数最常见的入口

互质数对与 $\gcd=k$

```
struct CoprimeCounter {
    int n;
    vector<int> sum_mu;
    explicit CoprimeCounter(int n_) : n(n_), sum_mu(n + 1) {
        vector<int> minp(n + 1), primes, mu(n + 1);
        if (n >= 1) mu[1] = 1;
        for (int i = 2; i <= n; ++ i) {
            if (!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
                mu[i] = -1;
            }
            for (int p : primes) {
                i64 v{ 1LL * i * p };
                if (v > n) break;
                minp[v] = p;
                if (p == minp[i]) {
                    mu[v] = 0;
                    break;
                }
                mu[v] = -mu[i];
            }
        }
        for (int i = 1; i <= n; ++ i) {
            sum_mu[i] = sum_mu[i - 1] + mu[i];
        }
    }
    // ===== 基础功能：统计 [1,a] × [1,b] 中的互质有序数对 =====
    i64 count(i64 a, i64 b) const {
        if (a <= 0 || b <= 0) return 0;
        i64 lim{ min(a, b) };
        assert(lim <= n);
        i64 ans{};
        for (i64 l = 1, r; l <= lim; l = r + 1) {
            i64 qa{ a / l }, qb{ b / l };
            r = min({ lim, a / qa, b / qb });
            ans += i64(sum_mu[r] - sum_mu[l - 1]) * qa * qb;
        }
        return ans;
    }
    // ===== 以下函数可直接删除：统计 gcd 恰好为 k =====
    i64 count_gcd(i64 a, i64 b, i64 k) const {
        assert(k > 0);
        return count(a / k, b / k);
    }
    // ===== 以下函数可直接删除：矩形内 gcd 恰好为 k =====
    i64 rect(i64 a, i64 b, i64 c, i64 d, i64 k) const {
        assert(1 <= a && a <= b && 1 <= c && c <= d && k > 0);
        return count_gcd(b, d, k) - count_gcd(a - 1, d, k)
             - count_gcd(b, c - 1, k) + count_gcd(a - 1, c - 1, k);
```

```
    }
};
```

原理

考虑

$$1 \le x \le n, \quad 1 \le y \le m$$

有：

$$[\gcd(x,y)=1] = \sum_{d \mid x,\, d \mid y} \mu(d)$$

交换求和：

$$\begin{aligned} C(n,m) &= \sum_{x=1}^{n}\sum_{y=1}^{m}[\gcd(x,y)=1] \\ &= \sum_{d=1}^{\min(n,m)} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor \end{aligned}$$

于是令

$$M(x) = \sum_{i=1}^{x} \mu(i)$$

对两个整除商共同分块：

$$C(n,m) = \sum_{[l,r]} \Big(M(r) - M(l-1) \Big) \left\lfloor \frac{n}{l} \right\rfloor \left\lfloor \frac{m}{l} \right\rfloor$$

这正是 `count()`

`count()` 接口

```
CoprimeCounter counter(N);
i64 ans = counter.count(n, m);
```

返回满足

$$1 \le x \le n, \quad 1 \le y \le m, \quad \gcd(x,y) = 1$$

的有序数对数量
要求：

$$\min(n,m) \le N$$

$\gcd = k$

若：

$$\gcd(x,y) = k$$

令：

$$x = kx', \qquad y = ky'$$

则：

$$\gcd(x',y') = 1$$

因此：

$$\#\{\gcd(x,y)=k\} = C\left(\left\lfloor \frac{n}{k} \right\rfloor, \left\lfloor \frac{m}{k} \right\rfloor \right)$$

对应：

```
i64 ans = counter.count_gcd(n, m, k);
```

矩形查询

求：

$$a \le x \le b, \quad c \le y \le d, \quad \gcd(x, y) = k$$

直接二维前缀容斥：

```
i64 ans = counter.rect(a, b, c, d, k);
```

等价于：

$$F(b, d) - F(a-1, d) - F(b, c-1) + F(a-1, c-1)$$

常见用法

$[1, n] \times [1, m]$ 互质数对

```
int N = min(n, m);
CoprimeCounter counter(N);
cout << counter.count(n, m) << '\n';
```

gcd 恰好为 k

```
cout << counter.count_gcd(n, m, k) << '\n';
```

多次矩形询问

若所有查询满足坐标除以 k 后不超过统一上界 `N`：

```
CoprimeCounter counter(N);
for (...) {
    cout << counter.rect(a, b, c, d, k) << '\n';
}
```

这是很经典的一类 Möbius + 数论分块查询

注意事项

`count()` 统计的是**有序数对**：

$$(x, y)$$

和

$$(y, x)$$

通常是两个不同对象
如果题目要求：

$$x < y, \quad x \le y$$

之类的限制，不能直接把答案除以 2，必须结合对角线是否允许单独处理
构造上界 `N` 必须覆盖所有查询真正需要的：

$$\min\left(\left\lfloor \frac{n}{k} \right\rfloor, \left\lfloor \frac{m}{k} \right\rfloor\right)$$

如果已经在 8.3 中构造了：

```
ArithmeticSieve sieve(N);
```

那么 `CoprimeCounter` 里那段 Möbius 线性筛可以删除，直接使用：

```
sieve.sum_mu
```

但单独复制 `CoprimeCounter` 时无需其他板子

复杂度

构造：

$$O(N)$$

时间和空间
一次 `count(n,m)`：

$$O(\sqrt{n}+\sqrt{m})$$

时间，额外空间 $O(1)$
若所有参数都不超过 N，通常直接写作：

$$O(\sqrt{N})$$

`count_gcd()` 同阶
`rect()` 调用四次，所以渐进复杂度仍为：

$$O(\sqrt{N})$$

q 次询问总复杂度：

$$O(N+q\sqrt{N})$$

gcd 权值和

Möbius反演不只是能判断 `gcd == 1`。更一般地，如果某个函数 F 可以写成：

$$F(n)=\sum_{d|n} f(d)$$

那么：

$$\sum_{i=1}^{n}\sum_{j=1}^{m} F(\gcd(i,j))$$

可以交换求和为：

$$\sum_{d=1}^{\min(n,m)} f(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$

于是只要有 f 的前缀和，又变成同一个数论分块模板
这个结论我建议哥哥一定保留，它能把很多看起来完全不同的 gcd 求和统一起来

gcd 总和

由经典恒等式：

$$\sum_{d|n}\varphi(d)=n$$

得到：

$$\gcd(i,j)=\sum_{d|\gcd(i,j)}\varphi(d)$$

因此：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\gcd(i,j)=\sum_{d=1}^{\min(n,m)}\varphi(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$

传入上一节已经预处理好的：

$$\Phi(x)=\sum_{i=1}^{x}\varphi(i)$$

```
__int128_t gcd_sum(i64 n, i64 m, const vector<i64> &sum_phi) {
    if (n ⩽ 0 || m ⩽ 0) return 0;
    i64 lim{ min(n, m) };
    assert(lim < i64(sum_phi.size()));
    __int128_t ans{};
    for (i64 l = 1, r; l ⩽ lim; l = r + 1) {
        i64 qn{ n / l }, qm{ m / l };
        r = min({ lim, n / qn, m / qm });
        ans += (__int128_t)(sum_phi[r] - sum_phi[l - 1]) * qn * qm;
    }
    return ans;
}
```

接口说明

```
auto ans = gcd_sum(n, m, sieve.sum_phi);
```

计算：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\gcd(i,j)$$

这里使用 `__int128_t`，因为即使 `n,m` 本身能放入 `i64`，GCD 总和也可能较早超过 `i64`

常见用法

```
ArithmeticSieve sieve(min(n, m));
__int128_t ans = gcd_sum(n, m, sieve.sum_phi);
```

如果题目要求取模，可以直接在每个分块贡献处取模，不必真的保存 `__int128_t` 总答案

复杂度

若 φ 前缀和已经预处理：

$$O(\sqrt{n}+\sqrt{m})$$

时间，

$$O(1)$$

额外空间
若连筛法一起计算，则还需：

$$O(\min(n,m))$$

预处理

其他 gcd 权值的统一写法

若：

$$F=f*\mathbf{1}$$

即：

$$F(n)=\sum_{d|n} f(d)$$

则：

$$\boxed{\sum_{i=1}^{n}\sum_{j=1}^{m} F(\gcd(i,j))=\sum_{d=1}^{\min(n,m)} f(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor}$$

几个非常常见的对应关系：

$F(n)$	$f(n)$	得到的统计
$[n=1]$	$\mu(n)$	互质数对
n	$\varphi(n)$	gcd 总和
$\tau(n)$	1	$\tau(\gcd)$ 总和
$\sigma_k(n)$	n^k	$\sigma_k(\gcd)$ 总和

例如：

$$\tau(n)=\sum_{d|n} 1$$

所以：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\tau(\gcd(i,j))=\sum_{d=1}^{\min(n,m)}\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$

此时分块内 $f(d)=1$，所以 `[l,r]` 的权值和直接是：

```
r - l + 1
```

Möbius 反演常见推导模板

遇到题目含有：

```
gcd(x, y) == 1
gcd(x, y) == k
gcd(x, y) | k
k | gcd(x, y)
F(gcd(x, y))
```

时，可以优先按下面这个思路检查：

1. 先把 gcd 条件变成约数条件
   例如：
   $$[\gcd(x,y)=1]=\sum_{d|x,\,d|y}\mu(d)$$
2. 交换枚举顺序
   原来枚举 (x,y)，改成先枚举公共约数 d
3. 计算 d 的倍数数量
   $[1,n]$ 中 d 的倍数有：
   $$\left\lfloor\frac{n}{d}\right\rfloor$$
4. 得到形如
   $$\sum_{d} f(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$
5. 对 f 建前缀和，再数论分块

注意事项

数论分块只有在“块内剩余部分可以 $O(1)$ 求出”时才真正有用。例如：

$$\sum_{i=l}^{r} \mu(i)$$

如果每个块里仍然逐项枚举 $\mu(i)$，复杂度又会退回 $O(n)$；因此要先有 Möbius 前缀和
同理：

$$\sum_{i=l}^{r} \varphi(i)$$

需要：

```
sum_phi[r] - sum_phi[l - 1]
```

而不是块内循环
另外，“莫比乌斯反演”本身通常不会自动降低复杂度。真正的加速往往来自：

```
莫比乌斯反演
→ 出现 floor(n / d)
→ 相同整除商分块
→ 用 μ / φ 等前缀和一次处理整块
```

这四步通常是一起出现的

复杂度速查

内容	时间复杂度	额外空间
单个 $\lfloor n/i \rfloor$ 数论分块	$O(\sqrt{n})$	$O(1)$
两个整除项共同分块	$O(\sqrt{n}+\sqrt{m})$	$O(1)$
Möbius 前缀预处理	$O(N)$	$O(N)$
`CoprimeCounter::count`	$O(\sqrt{N})$	$O(1)$
`count_gcd`	$O(\sqrt{N})$	$O(1)$
`rect`	$O(\sqrt{N})$	$O(1)$
已有 φ 前缀和的 `gcd_sum`	$O(\sqrt{N})$	$O(1)$
q 次互质 / gcd 查询	$O(N+q\sqrt{N})$	$O(N)$

8.3.2 杜教筛

解决问题： 已知积性函数 f，在 n 很大、无法线性筛到 n 时，利用狄利克雷卷积与数论分块求前缀和

$$S_f(n) = \sum_{i=1}^{n} f(i)$$

典型应用是

$$M(n) = \sum_{i=1}^{n} \mu(i), \quad \Phi(n) = \sum_{i=1}^{n} \varphi(i)$$

下面给出最常用的 μ/φ 自包含版本，内部直接带线性筛，复制这一块即可使用

```
struct Dujiao {
    int limit;
    vector<i64> pre_mu, pre_phi;
    unordered_map<i64, i64> memo_mu;
    unordered_map<i64, __int128_t> memo_phi;
    explicit Dujiao(int n) : limit(n) {
        assert(n >= 1);
        vector<int> minp(n + 1), primes;
        vector<i64> mu(n + 1), phi(n + 1);
        mu[1] = phi[1] = 1;
        for (int i = 2; i <= n; ++ i) {
            if (!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
                mu[i] = -1;
```

```
                phi[i] = i - 1;
            }
            for (int p : primes) {
                if (p > n / i) break;
                int v{ i * p };
                minp[v] = p;
                if (p == minp[i]) {
                    mu[v] = 0;
                    phi[v] = phi[i] * p;
                    break;
                }
                mu[v] = -mu[i];
                phi[v] = phi[i] * (p - 1);
            }
        }
        for (int i = 1; i <= n; ++ i) {
            mu[i] += mu[i - 1];
            phi[i] += phi[i - 1];
        }
        pre_mu = move(mu);
        pre_phi = move(phi);
    }
    // ===== Mertens 函数 =====
    i64 sum_mu(i64 n) {
        if (n <= limit) return pre_mu[n];
        if (auto it = memo_mu.find(n); it != memo_mu.end()) return it->second;
        __int128_t ans{ 1 };
        for (i64 l = 2, r; l <= n; l = r + 1) {
            i64 q{ n / l };
            r = n / q;
            ans -= (__int128_t)(r - l + 1) * sum_mu(q);
        }
        return memo_mu[n] = i64(ans);
    }

    // ===== phi 前缀和 =====
    __int128_t sum_phi(i64 n) {
        if (n <= limit) return pre_phi[n];
        if (auto it = memo_phi.find(n); it != memo_phi.end()) return it->second;
        __int128_t ans{ (__int128_t)n * (n + 1) / 2 };
        for (i64 l = 2, r; l <= n; l = r + 1) {
            i64 q{ n / l };
            r = n / q;
            ans -= (__int128_t)(r - l + 1) * sum_phi(q);
        }
        return memo_phi[n] = ans;
    }
};
```

采用 `sumMu(n)` 的经典递归形式，同时将 φ 前缀和补成对称接口，并避免把 `mu` 的点值和前缀值混在外部接口里

接口说明

- `Dujiao(limit)`：线性预处理 $1 \dots limit$ 的 μ, φ 前缀和
- `sum_mu(n)`：返回 Mertens 函数

$$M(n) = \sum_{i=1}^{n} \mu(i)$$

- `sum_phi(n)`：返回欧拉函数前缀和

$$\Phi(n) = \sum_{i=1}^{n} \varphi(i)$$

- `memo_mu / memo_phi`：缓存大于 `limit` 的递归状态，多次查询会自动复用
- 支持 `n = 0`，此时两个前缀和均为 0
  `sum_phi()` 返回 `__int128_t`，因为

$$\Phi(n) = \Theta(n^2)$$

即使 `n` 能放入 `i64`，答案也可能超过 `i64`

核心原理

设

$$h = f * g$$

并定义

$$S(n) = \sum_{i=1}^{n} f(i), \quad G(n) = \sum_{i=1}^{n} g(i), \quad H(n) = \sum_{i=1}^{n} h(i)$$

由狄利克雷卷积

$$h(n) = \sum_{d \mid n} f(d) g(n/d)$$

交换求和顺序可得

$$H(n) = \sum_{t=1}^{n} g(t) S\left(\left\lfloor \frac{n}{t} \right\rfloor\right)$$

于是

$$g(1) S(n) = H(n) - \sum_{t=2}^{n} g(t) S\left(\left\lfloor \frac{n}{t} \right\rfloor\right)$$

若 $g(1) = 1$，对 $\lfloor n/t \rfloor$ 数论分块，设当前块为 $[l, r]$

$$S(n) = H(n) - \sum_{[l,r]} \Big(G(r) - G(l-1)\Big) S\left(\left\lfloor \frac{n}{l} \right\rfloor\right)$$

因此杜教筛能使用的关键条件是：$f * g$ 的前缀和容易计算，g 的区间和容易计算，再用数论分块把递归压缩下来，这也是通用杜教筛的标准形式

Möbius 前缀和

取

$$f = \mu, \quad g = \mathbf{1}$$

由于

$$\mu * \mathbf{1} = \varepsilon$$

并且

$$\sum_{i=1}^{n} \varepsilon(i) = 1, \quad \sum_{i=1}^{n} \mathbf{1}(i) = n$$

得到

$$M(n) = 1 - \sum_{i=2}^{n} M\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

数论分块后

$$M(n) = 1 - \sum_{[l,r]} (r - l + 1) M\left(\left\lfloor \frac{n}{l} \right\rfloor\right)$$

对应代码核心就是

```
__int128_t ans{ 1 };
for (i64 l = 2, r; l <= n; l = r + 1) {
    i64 q{ n / l };
    r = n / q;
    ans -= (__int128_t)(r - l + 1) * sum_mu(q);
}
```

欧拉函数前缀和

取

$$f = \varphi, \quad g = \mathbf{1}$$

由于

$$\varphi * \mathbf{1} = \mathrm{id}$$

并且

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

得到

$$\Phi(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} \Phi\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

数论分块后

$$\Phi(n) = \frac{n(n+1)}{2} - \sum_{[l,r]} (r-l+1)\Phi\left(\left\lfloor \frac{n}{l} \right\rfloor\right)$$

因此 μ 与 φ 两个版本实际上只有初始项不同

```
// mu
__int128_t ans{ 1 };
// phi
__int128_t ans{ (__int128_t)n * (n + 1) / 2 };
```

常见用法

求

$$\sum_{i=1}^{n} \mu(i)$$

```
Dujiao dujiao(2'000'000);
i64 ans = dujiao.sum_mu(n);
```

求

$$\sum_{i=1}^{n} \varphi(i)$$

```
Dujiao dujiao(2'000'000);
__int128_t ans = dujiao.sum_phi(n);
```

同时需要二者时只构造一次

```
Dujiao dujiao(2'000'000);
i64 smu = dujiao.sum_mu(n);
__int128_t sphi = dujiao.sum_phi(n);
```

多次询问可以继续复用同一个对象

```
Dujiao dujiao(limit);
while (q --) {
    i64 n;
    cin >> n;
    auto ans = dujiao.sum_mu(n);
}
```

已经线性筛到 `limit` 的查询不会进入递归

```
dujiao.sum_mu(x);   // x ≤ limit 时直接 O(1)
dujiao.sum_phi(x);  // x ≤ limit 时直接 O(1)
```

预处理大小

设最大查询值为 N，经典复杂度分析中通常预处理约

$$B \approx N^{2/3}$$

个小值，此时杜教筛可做到单次约

$$O(N^{2/3})$$

实际比赛中不必机械计算 $N^{2/3}$，通常根据内存直接取一个合适的 `limit`，例如

```
Dujiao dujiao(1'000'000);
Dujiao dujiao(2'000'000);
Dujiao dujiao(5'000'000);
```

`limit` 越大，线性筛占用越多，但递归和哈希状态越少
ECNU 模板同样采用“小范围线性预处理 + 大范围递归缓存”的结构，并指出在经典 $N^{2/3}$ 预处理下可得到 $O(N^{2/3})$ 复杂度

为什么必须记忆化

递归中只会反复访问形如

$$\left\lfloor \frac{N}{i} \right\rfloor$$

的状态
若不缓存，同一个状态会被大量重复计算

```
if (auto it = memo_mu.find(n); it != memo_mu.end()) return it->second;
```

对于一次最大查询 N，大于 `limit` 的有效商值数量约为

$$O\left(\frac{N}{\text{limit}}\right)$$

所以缓存远小于直接保存 $1 \dots N$ 的所有值

通用杜教筛改法

遇到其他积性函数 f 时，不需要另写算法，按下面三步替换即可

1. 选择容易处理的 g，最好满足 $g(1) = 1$
2. 求出

$$H(n) = \sum_{i=1}^{n} (f * g)(i)$$

   和

$$G(n) = \sum_{i=1}^{n} g(i)$$

3. 将递归改为

```
T ans = H(n);
for (i64 l = 2, r; l ≤ n; l = r + 1) {
    i64 q{ n / l };
    r = n / q;
    ans -= (G(r) - G(l - 1)) * solve(q);
}
```

对应数学式

$$S_f(n) = H(n) - \sum_{[l,r]} \Big(G(r) - G(l-1)\Big) S_f\left(\left\lfloor \frac{n}{l} \right\rfloor\right)$$

若找不到一个使 $H(n)$、$G(n)$ 都容易求的 g，杜教筛通常就不是合适的方法

注意事项

`sum_mu()` 中虽然最终结果一定落在 `i64` 范围内，递推的中间乘积仍使用 `__int128_t`，不要提前改回 `i64`
`sum_phi()` 的初值必须先扩大类型

```
(__int128_t)n * (n + 1) / 2
```

不能写成

```
n * (n + 1) / 2
```

否则乘法会先在 `i64` 中溢出
若只需要 μ，可以删除 `pre_phi`、`memo_phi`、`sum_phi()` 以及线性筛中的 `phi` 转移
若只需要 φ，可以删除 `pre_mu`、`memo_mu`、`sum_mu()` 以及线性筛中的 `mu` 转移
若答案要求取模，可以把递推值改成模整数，但三角和最好先做整数除法

$$\frac{n(n+1)}{2} \bmod P$$

不需要假设 2 在模 P 下存在逆元
`unordered_map` 中保存的是函数前缀和而不是单点 $\mu(n), \varphi(n)$
杜教筛解决的是前缀和，若只要求单点函数值，应继续使用 8.2 的质因数分解或 8.3 的线性筛

复杂度

设最大查询为 N，预处理上界为 B
线性筛时间与空间

$$O(B)$$

经典取

$$B = \Theta(N^{2/3})$$

时，单次杜教筛时间

$$O(N^{2/3})$$

空间由线性筛占主导

$$O(N^{2/3})$$

8.3.3 Min_25 筛

解决问题： 对积性函数 f，在 n 很大时计算

$$S_f(n) = \sum_{i=1}^{n} f(i)$$

Min_25 的核心分两步：先求若干完全积性基函数在质数上的前缀和，再按最小质因子递归枚举质数幂
这里整理为可修改的通用基函数版，默认计算

$$\sum_{i=1}^{n} \varphi(i)$$

顶部四个函数是主要修改区，其余部分通常不用动

```
template<class T = __int128_t, int K = 2>
struct Min25 {
    using A = array<T, K>;
    i64 n, sq;
    vector<int> primes, id1, id2;
```

```
    vector<i64> w;
    vector<A> g, pre;
    // ===== 按题修改：f(p) 的基函数、基函数前缀和、f(p^e) =====
    // 默认 f = phi，f(p) = p - 1
    A basis(i64 p) const { return { T(1), T(p) }; }
    // 返回 sum_{i=2}^x basis(i)
    A basis_sum(i64 x) const {
        T s = x & 1 ? T(x) * T((x + 1) / 2) : T(x / 2) * T(x + 1);
        return { T(x - 1), s - T(1) };
    }
    // 输入各基函数在一段质数上的和，返回对应的 sum f(p)
    T prime_sum(const A &a) const { return a[1] - a[0]; }
    // f(p^e)，pe = p^e
    T prime_power(i64 p, int e, i64 pe) const { return T(pe - pe / p); }
    explicit Min25(i64 n_) : n(n_) {
        assert(n >= 1);
        sq = sqrtl((long double)n);
        while (sq + 1 <= n / (sq + 1)) ++ sq;
        while (sq > n / sq) -- sq;
        build_primes();
        build_prime_sum();
    }
    T solve() const { return T(1) + dfs(n, -1); }
private:
    int id(i64 x) const { return x <= sq ? id1[x] : id2[n / x]; }
    // 统计 <= x 且最小质因子 > primes[k] 的贡献
    T dfs(i64 x, int k) const {
        if (x <= 1 || (k >= 0 && primes[k] > x)) return T{};
        int t{ id(x) };
        A s{};
        for (int j = 0; j < K; ++ j) s[j] = g[t][j] - pre[k + 1][j];
        // ===== 先计入所有可选质数本身 =====
        T ans{ prime_sum(s) };
        // ===== 枚举最小质因子及其指数 =====
        for (int i = k + 1; i < int(primes.size()); ++ i) {
            i64 p{ primes[i] };
            if (p > x / p) break;
            // p 已在上面的质数贡献中出现，先删除再统一枚举 p^e
            ans -= prime_sum(basis(p));
            for (i64 pe = p, e = 1; pe <= x; ++ e) {
                ans += prime_power(p, int(e), pe) * (T(1) + dfs(x / pe, i));
                if (pe > x / p) break;
                pe *= p;
            }
        }
        return ans;
    }
    // ===== 筛 sqrt(n) 内质数，并维护各基函数的质数前缀和 =====
    void build_primes() {
        vector<int> minp(sq + 1);
        pre.push_back({});
        for (int i = 2; i <= sq; ++ i) {
            if (!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
                A s{ pre.back() }, b{ basis(i) };
                for (int j = 0; j < K; ++ j) s[j] += b[j];
                pre.push_back(s);
            }
            for (int p : primes) {
                if (p > sq / i) break;
                minp[i * p] = p;
                if (p == minp[i]) break;
            }
        }
    }
    // ===== Min_25 第一阶段：求各整除商位置的质数基函数前缀和 =====
    void build_prime_sum() {
        id1.assign(sq + 1, -1);
        id2.assign(sq + 1, -1);
        for (i64 l = 1, r; l <= n; l = r + 1) {
```

```
            i64 v{ n / l };
            r = n / v;
            int t{ int(w.size()) };
            w.push_back(v);
            g.push_back(basis_sum(v));
            if (v <= sq) id1[v] = t;
            else id2[n / v] = t;
        }
        for (int k = 0; k < int(primes.size()); ++ k) {
            i64 p{ primes[k] };
            A b{ basis(p) };
            for (int i = 0; i < int(w.size()) && p <= w[i] / p; ++ i) {
                int t{ id(w[i] / p) };
                for (int j = 0; j < K; ++ j) {
                    g[i][j] -= b[j] * (g[t][j] - pre[k][j]);
                }
            }
        }
    }
};
```

质数筛部分采用经典 Min_25 的整除商离散化：只保存 $\lfloor n/i \rfloor$ 的不同值，并用 `id1 / id2` 在 $O(1)$ 时间找到状态；质数更新也是标准的

$$G(v) \leftarrow G(v) - g(p)\left(G\left(\left\lfloor \frac{v}{p} \right\rfloor\right) - G(p^-)\right)$$

接口说明

构造

```
Min25 min25(n);
```

默认 `T = __int128_t`、`K = 2`，计算欧拉函数前缀和

```
auto ans = min25.solve();
```

返回

$$\sum_{i=1}^{n} \varphi(i)$$

成员含义：

- `sq`：$\lfloor \sqrt{n} \rfloor$
- `primes`：不超过 $\sqrt{n}$ 的质数
- `w`：所有不同的 $\lfloor n/i \rfloor$
- `id1 / id2`：将整除商映射到 `w` 下标
- `g[t][j]`：Min_25 第一阶段结束后，第 `j` 个基函数在 $p \le w[t]$ 的质数上的和
- `pre[k][j]`：前 `k` 个质数的第 `j` 个基函数之和
- `dfs(x,k)`：内部递归，统计不超过 x 且最小质因子大于 `primes[k]` 的数的贡献

使用条件

Min_25 处理的是积性函数

$$\gcd(a,b) = 1 \implies f(ab) = f(a)f(b)$$

还需要能 $O(1)$ 计算

$$f(p^e)$$

并且质数处的函数值能够写成少量完全积性基函数的线性组合

$$f(p) = \sum_{j=0}^{K-1} c_j g_j(p)$$

其中每个 g_j 满足

$$g_j(ab) = g_j(a)g_j(b)$$

并且

$$\sum_{i=2}^{x} g_j(i)$$

能够快速计算
常见基函数就是

$$1, \quad n, \quad n^2, \quad n^3$$

默认版本为什么是 φ

对于质数 p

$$\varphi(p) = p - 1$$

因此取两个基函数

$$g_0(p) = 1, \quad g_1(p) = p$$

对应

```
A basis(i64 p) const { return { T(1), T(p) }; }
```

它们从 2 到 x 的普通前缀和分别是

$$\sum_{i=2}^{x} 1 = x - 1$$

$$\sum_{i=2}^{x} i = \frac{x(x+1)}{2} - 1$$

所以

```
T prime_sum(const A &a) const { return a[1] - a[0]; }
```

而质数幂上

$$\varphi(p^e) = p^e - p^{e-1}$$

对应

```
T prime_power(i64 p, int e, i64 pe) const {
    return T(pe - pe / p);
}
```

第一阶段：筛质数前缀和

对每个基函数 g_j，初始化

$$G_j(v) = \sum_{i=2}^{v} g_j(i)$$

此时包含质数与合数
按质数 p_k 依次筛去最小质因子为 p_k 的合数

$$G_j(v) \mathrel{-}= g_j(p_k)\left(G_j\left(\left\lfloor \frac{v}{p_k} \right\rfloor\right) - \sum_{i<k} g_j(p_i)\right)$$

处理完成后

$$G_j(v) = \sum_{\substack{p \le v \\ p \text{ prime}}} g_j(p)$$

由于只关心

$$v = \left\lfloor \frac{n}{i} \right\rfloor$$

所有状态数量只有

$$O(\sqrt{n})$$

第二阶段：枚举最小质因子

设 `dfs(x,k)` 只允许使用

$$p > p_k$$

的质因子
先利用第一阶段直接得到所有允许质数的贡献

$$\sum_{\substack{p \le x \\ p > p_k}} f(p)$$

随后枚举下一个最小质因子 p_i 和指数 e

$$p_i^e$$

剩余部分必须只含大于 p_i 的质因子，因此递归到

$$\mathrm{dfs}\left(\left\lfloor \frac{x}{p_i^e} \right\rfloor, i\right)$$

每个整数按自己的最小质因子与其指数被唯一枚举一次，所以不会重复

修改为其他积性函数

通常只需要修改代码开头四个函数

约数个数 τ

有

$$\tau(p^e) = e + 1, \quad \tau(p) = 2$$

只需要常数基函数即可，可以仍保留 `K = 2`，让第二项恒为 0

```
A basis(i64 p) const { return { T(1), T(0) }; }
A basis_sum(i64 x) const { return { T(x - 1), T(0) }; }
T prime_sum(const A &a) const { return T(2) * a[0]; }
T prime_power(i64 p, int e, i64 pe) const { return T(e + 1); }
```

Jordan 函数 J_2

有

$$J_2(p) = p^2 - 1$$

$$J_2(p^e) = p^{2e} - p^{2e-2}$$

改成基函数

$$1, \quad p^2$$

其中平方和

$$\sum_{i=1}^{x} i^2 = \frac{x(x+1)(2x+1)}{6}$$

若在模意义下计算，建议先在整数中约掉 $2,3$ 再转为 `T`，避免要求模数下存在 6^{-1}

μ

有

$$\mu(p) = -1, \quad \mu(p^e) = 0 \quad (e \geq 2)$$

也可以套 Min_25，但这一类函数直接使用上一节杜教筛更短

基函数超过两个

若

$$f(p) = c_0 + c_1 p + c_2 p^2$$

将模板改成

```
Min25<T, 3>
```

并让

```
basis(p)
```

返回

```
{1, p, p * p}
```

`basis_sum(x)` 返回三个基函数从 2 到 x 的前缀和
`prime_sum(a)` 返回

```
c0 * a[0] + c1 * a[1] + c2 * a[2]
```

内部筛法和递归部分不需要修改
这里 `prime_sum()` 必须是各基函数和的线性组合，因为它需要满足

$$\text{prime_sum}\left(\sum_p \mathbf{g}(p)\right) = \sum_p f(p)$$

常见用法

默认求欧拉函数前缀和

```
i64 n;
cin >> n;
Min25 min25(n);
auto ans = min25.solve();
```

若答案保证在 `i64` 内，可以减少一半左右的大数组单元大小

```
Min25<i64> min25(n);
i64 ans = min25.solve();
```

若已经有模整数 `Z`

```
Min25<Z> min25(n);
Z ans = min25.solve();
```

此时 `basis_sum()` 等函数也应避免依赖不可逆整数的模逆元，优先先约分再乘

为什么默认用 `__int128_t`

Min_25 常用于 n 已经无法线性筛的情况，此时很多函数前缀和很容易超过 `i64`
例如

$$\sum_{i=1}^{n}\varphi(i)=\Theta(n^2)$$

所以默认结果类型使用 `__int128_t`
若题目要求取模，应将 `T` 换成模整数类型，这也能明显减少内存

注意事项

`Min25` 对象与构造时的 `n` 绑定，不能构造一次后直接调用另一个 `n`

```
Min25 a(n);
auto ans = a.solve();
```

若查询另一个 m，需要重新构造

```
Min25 b(m);
```

`basis()` 中的每一项必须对应一个完全积性函数，不能直接把

$$p^2+p+1$$

整个作为单个基函数
应该拆成

$$p^2,\quad p,\quad 1$$

`basis_sum(x)` 是对所有整数 $2\dots x$ 求和，不是对质数求和，质数限制会由 Min_25 第一阶段筛出来
`prime_power(p,e,pe)` 必须返回

$$f(p^e)$$

而不是仅返回 $f(p)$
`solve()` 已经补上

$$f(1)=1$$

因此 `dfs()` 本身不包含 1
循环中均使用

```
p > x / p
pe > x / p
```

而不是 `p * p > x`、`pe * p > x`，避免整数乘法溢出

与杜教筛的选择

场景	更适合
$\sum\mu(i)$、$\sum\varphi(i)$	杜教筛，代码更短
容易找到 $f*g$ 使卷积前缀和简单	杜教筛
$f(p)$ 是低次多项式或少量完全积性函数的线性组合	Min_25
$f(p^e)$ 容易计算，但整体没有好用的杜教卷积	Min_25
只求 $\pi(n)$ 或 $\sum_{p\le n}p$	Min_25 第一阶段即可

复杂度

设 `basis()`、`basis_sum()`、`prime_sum()`、`prime_power()` 均为 $O(1)$
不同整除商数量为

$$O(\sqrt{n})$$

空间复杂度为

$$O(K\sqrt{n})$$

固定 K 时即

$$O(\sqrt{n})$$

经典 Min_25 的时间复杂度为

$$O\left(\frac{n^{3/4}}{\log n}\right)$$

实际常数与基函数数量 K、`T` 的大小和递归中的质数幂数量有关

8.4 逆元、CRT 与模方程

解决问题： 求单个或批量模逆元，求解线性同余

$$ax \equiv b \pmod{m}$$

以及同余方程组

$$x \equiv r_i \pmod{m_i}$$

这里直接保留广义 CRT / exCRT，它本身就包含模数两两互质的普通 CRT，不再另存一份功能重复的代码

模逆元

整数 a 在模 m 下存在逆元，当且仅当

$$\gcd(a, m) = 1$$

此时存在唯一的 $a^{-1} \in [0, m)$ 满足

$$aa^{-1} \equiv 1 \pmod{m}$$

常见做法是质数模下使用费马小定理，一般模数下使用 exGCD；线性递推则可以 $O(n)$ 求质数模下 $1 \dots n$ 的全部逆元

```
i64 norm_mod(i64 x, i64 mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
i64 mul_mod(i64 a, i64 b, i64 mod) {
    return i64((__int128_t)norm_mod(a, mod) * norm_mod(b, mod) % mod);
}
i64 qpow_mod(i64 a, u64 b, i64 mod) {
    i64 res{ 1 % mod };
    a = norm_mod(a, mod);
    for (; b; b >>= 1, a = mul_mod(a, a, mod)) {
        if (b & 1) res = mul_mod(res, a, mod);
    }
    return res;
}
array<i64, 3> exgcd(i64 a, i64 b) {
    if (!b) return { a, 1, 0 };
    auto [g, x, y] = exgcd(b, a % b);
    return { g, y, x - a / b * y };
}
// 任意正模数，逆元不存在时返回 nullopt
optional<i64> inv_mod(i64 a, i64 mod) {
    if (mod <= 1) return nullopt;
    a = norm_mod(a, mod);
    auto [g, x, y] = exgcd(a, mod);
    if (g != 1) return nullopt;
    return norm_mod(x, mod);
}
```

```
// 质数模，要求 a % p != 0
i64 inv_prime(i64 a, i64 p) {
    return qpow_mod(a, p - 2, p);
}
// 质数模下预处理 1..n 的逆元，要求 1 ≤ n < p
vector<i64> inverses_prime(int n, i64 p) {
    vector<i64> inv(n + 1);
    inv[1] = 1;
    for (int i = 2; i ≤ n; ++ i) {
        inv[i] = (__int128_t)(p - p / i) * inv[p % i] % p;
    }
    return inv;
}
```

接口说明

`inv_mod(a, mod)`：一般模数逆元，要求 `mod > 1`

```
auto inv = inv_mod(a, mod);
if (!inv) {
    // gcd(a, mod) != 1
} else {
    i64 x = *inv;
}
```

`inv_prime(a, p)`：要求 p 为质数且 $p \nmid a$

```
i64 x = inv_prime(a, p);
```

`inverses_prime(n,p)`：要求 p 为质数且

$$1 \le n < p$$

返回 `inv[i] = i^{-1} mod p`

```
auto inv = inverses_prime(n, p);
```

原理

一般模数下由 Bézout

$$ax + my = 1$$

可知

$$ax \equiv 1 \pmod{m}$$

所以 exGCD 求出的 x 就是逆元
质数模下由费马小定理

$$a^{p-1} \equiv 1 \pmod{p}$$

得到

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

线性递推由

$$p = \left\lfloor \frac{p}{i} \right\rfloor i + (p \bmod i)$$

两边模 p:

$$(p \bmod i) \equiv -\left\lfloor \frac{p}{i} \right\rfloor i \pmod{p}$$

从而

$$i^{-1} \equiv -\left\lfloor \frac{p}{i} \right\rfloor (p \bmod i)^{-1} \pmod{p}$$

即代码中的

$$\mathrm{inv}[i] = \left(p - \left\lfloor \frac{p}{i} \right\rfloor\right) \mathrm{inv}[p \bmod i] \bmod p$$

常见用法

除法转乘逆元：

```
auto ib = inv_mod(b, mod);
if (ib) {
    i64 ans = mul_mod(a, *ib, mod);
}
```

质数模：

```
i64 ans = mul_mod(a, inv_prime(b, mod), mod);
```

一次预处理后大量查询：

```
auto inv = inverses_prime(n, mod);
i64 x = inv[k];
```

注意事项

模除法

$$\frac{a}{b} \bmod m$$

只有在

$$\gcd(b, m) = 1$$

时才能直接写成

$$ab^{-1} \bmod m$$

`inv_prime()` 不能因为代码更短就用于合数模
`inverses_prime()` 的递推版主要保留给质数模且 `n < p` 的场景
阶乘逆元属于组合数模块，不在这里重复保存

复杂度

`inv_mod()`：

$$O(\log m)$$

`inv_prime()`：

$$O(\log p)$$

`inverses_prime()`：

$$O(n)$$

任意数组批量逆元

若给定

$$a_1, a_2, \ldots, a_n$$

且每个数在模 m 下都有逆元，可以只调用一次逆元算法求出全部 a_i^{-1}

```
optional<vector<i64>> batch_inverse(const vector<i64> &a, i64 mod) {
    if (mod ⩽ 1) return nullopt;
    int n{ int(a.size()) };
    vector<i64> x(n), pre(n + 1, 1), inv(n);
    for (int i = 0; i < n; ++ i) {
        x[i] = norm_mod(a[i], mod);
        pre[i + 1] = mul_mod(pre[i], x[i], mod);
    }
    auto suf = inv_mod(pre[n], mod);
    if (!suf) return nullopt;
    i64 cur{ *suf };
    for (int i = n - 1; i ⩾ 0; -- i) {
        inv[i] = mul_mod(pre[i], cur, mod);
        cur = mul_mod(cur, x[i], mod);
    }
    return inv;
}
```

原理

记

$$P_i = \prod_{j=0}^{i-1} a_j$$

先求

$$P_n^{-1}$$

再从右向左维护后缀乘积的逆元
对于第 i 个数：

$$a_i^{-1} = P_i \left(\prod_{j=i}^{n} a_j \right)^{-1}$$

因此只需要一次 `inv_mod()`

常见用法

```
auto inv = batch_inverse(a, mod);
if (!inv) {
    // 至少一个 a[i] 与 mod 不互质
} else {
    // (*inv)[i] = a[i]^{-1} mod mod
}
```

注意事项

要求每个

$$\gcd(a_i, m) = 1$$

否则总乘积也没有逆元，函数返回 `nullopt`
空数组合法，返回空数组
这个版本适用于合数模，只要每个元素都与模数互质

复杂度

只有一次 exGCD，其余为线性扫描：

$$O(n + \log m)$$

空间：

$$O(n)$$

CRT 与 exCRT

普通 CRT 要求模数两两互质，exCRT 则逐个合并两个同余方程，因此不要求模数互质
以下模块完全自包含，同时提供单个线性同余、两个同余合并、CRT 与一般线性模方程组

```
struct Congruence {
    static i64 norm(i64 x, i64 mod) {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }
    static array<i64, 3> exgcd(i64 a, i64 b) {
        if (!b) return { a, 1, 0 };
        auto [g, x, y] = exgcd(b, a % b);
        return { g, y, x - a / b * y };
    }
    static i64 mul(i64 a, i64 b, i64 mod) {
        return i64((__int128_t)norm(a, mod) * norm(b, mod) % mod);
    }
    // ===== 单个线性同余 ax ≡ b (mod m) =====
    // 返回 {最小非负解, 解的周期}
    static optional<pair<i64, i64>> linear(i64 a, i64 b, i64 mod) {
        if (mod ⩽ 0) return nullopt;
        a = norm(a, mod), b = norm(b, mod);
        auto [g, x, y] = exgcd(a, mod);
        if (b % g) return nullopt;
        i64 m{ mod / g };
        i64 r{ m == 1 ? 0 : mul(x, b / g, m) };
        return pair{ r, m };
    }
    // ===== 合并 x ≡ r1 (mod m1), x ≡ r2 (mod m2) =====
    static optional<pair<i64, i64>> merge(i64 r1, i64 m1, i64 r2, i64 m2) {
        if (m1 ⩽ 0 || m2 ⩽ 0) return nullopt;
        r1 = norm(r1, m1), r2 = norm(r2, m2);
        auto [g, x, y] = exgcd(m1, m2);
        i64 d{ r2 - r1 };
        if (d % g) return nullopt;
        i64 m{ m2 / g };
        i64 t{ m == 1 ? 0 : mul(x, d / g, m) };
        __int128_t lcm{ (__int128_t)(m1 / g) * m2 };
        if (lcm > numeric_limits<i64>::max()) return nullopt;
        __int128_t ans{ (__int128_t)r1 + (__int128_t)m1 * t };
        i64 mod{ i64(lcm) };
        i64 r{ i64(ans % mod) };
        if (r < 0) r += mod;
        return pair{ r, mod };
    }
    // ===== CRT / exCRT: eq[i] = {ri, mi} =====
    static optional<pair<i64, i64>> crt(const vector<pair<i64, i64>> &eq) {
        pair<i64, i64> cur{ 0, 1 };
        for (auto [r, m] : eq) {
            auto nxt = merge(cur.first, cur.second, r, m);
            if (!nxt) return nullopt;
            cur = *nxt;
        }
        return cur;
    }
    // ===== 一般线性模方程组: eq[i] = {ai, bi, mi} =====
    // ai * x ≡ bi (mod mi)
    static optional<pair<i64, i64>> solve(const vector<array<i64, 3>> &eq) {
        pair<i64, i64> cur{ 0, 1 };
        for (auto [a, b, m] : eq) {
            auto one = linear(a, b, m);
            if (!one) return nullopt;
            auto nxt = merge(cur.first, cur.second, one->first, one->second);
            if (!nxt) return nullopt;
            cur = *nxt;
        }
        return cur;
```

```
    }
};
```

单个线性同余

`linear(a,b,m)` 求

$$ax \equiv b \pmod{m}$$

设

$$g = \gcd(a, m)$$

有解当且仅当

$$g \mid b$$

若有解，返回 `{r, mod}`，表示

$$x \equiv r \pmod{\frac{m}{g}}$$

其中

$$0 \le r < \frac{m}{g}$$

这与 8.1 中的线性同余接口一致，但这里重新带了一份短实现，使整个 CRT / 模方程模块可以独立复制

常见用法

```
auto sol = Congruence::linear(a, b, m);
if (sol) {
    auto [x, period] = *sol;
}
```

复杂度

$$O(\log m)$$

两个同余方程合并

求

$$x \equiv r_1 \pmod{m_1}$$

$$x \equiv r_2 \pmod{m_2}$$

令

$$x = r_1 + m_1 t$$

代入第二式：

$$m_1 t \equiv r_2 - r_1 \pmod{m_2}$$

因此有解当且仅当

$$\gcd(m_1, m_2) \mid (r_2 - r_1)$$

若有解，所有解构成一个新的同余类

$$x \equiv r \pmod{\operatorname{lcm}(m_1, m_2)}$$

常见用法

```
auto sol = Congruence::merge(r1, m1, r2, m2);
if (sol) {
    auto [r, mod] = *sol;
}
```

注意事项

要求

$$m_1 > 0 \quad , \quad m_2 > 0$$

板子内部使用 `__int128_t` 防止中间乘法溢出
最终周期仍保存为 `i64`，因此要求最终

$$\operatorname{lcm}(m_1, m_2) \le 2^{63} - 1$$

如果超过 `i64`，当前实现返回 `nullopt`

复杂度

一次 *ex*GCD：

$$O(\log \min(m_1, m_2))$$

CRT / exCRT 方程组

对于

$$x \equiv r_i \pmod{m_i}$$

输入：

```
vector<pair<i64, i64>> eq{
    {r1, m1},
    {r2, m2},
    {r3, m3}
};
auto sol = Congruence::crt(eq);
```

若有解：

```
auto [r, mod] = *sol;
```

表示全部整数解

$$x \equiv r \pmod{mod}$$

其中

$$0 \le r < mod$$

模数无需互质
若模数两两互质，则退化成普通 CRT，此时

$$mod = \prod_i m_i$$

普通 CRT 的闭式形式为：令

$$M=\prod_i m_i \quad , \quad M_i=\frac{M}{m_i}$$

再令

$$t_i \equiv M_i^{-1} \pmod{m_i}$$

则

$$x \equiv \sum_i r_i M_i t_i \pmod{M}$$

实际代码仍建议使用上面的逐个合并版，因为同一份实现可以同时覆盖普通 CRT 与 exCRT

空方程组

```
Congruence::crt({})
```

返回

```
{0, 1}
```

表示

$$x \equiv 0 \pmod{1}$$

即任意整数

复杂度

设有 k 个方程，当前合并模数为 M_i

$$O\left(\sum_{i=1}^{k} \log \min(M_i, m_i)\right)$$

通常简记为

$$O(k \log M)$$

一般线性模方程组

进一步求

$$a_i x \equiv b_i \pmod{m_i}$$

每个方程先化成

$$x \equiv r_i \pmod{m_i'}$$

再用 exCRT 合并
这个过程已经封装在 `solve()`

常见用法

```
vector<array<i64, 3>> eq{
    {a1, b1, m1},
    {a2, b2, m2},
    {a3, b3, m3}
};
auto sol = Congruence::solve(eq);
if (!sol) {
    // 无解
} else {
    auto [x, period] = *sol;
```

```
    // x ≡ x0 (mod period)
}
```

例如

$$2x \equiv 4 \pmod 6$$

先化为

$$x \equiv 2 \pmod 3$$

再与其他方程合并即可

注意事项

每个 `m_i` 必须为正
若某一个线性同余无解，整个方程组无解
若两个已经化简的同余类不兼容，整个方程组也无解
最终周期要求能放入 `i64`
指数方程

$$a^x \equiv b \pmod m$$

不属于这里，放到后面的 BSGS / exBSGS
平方同余

$$x^2 \equiv a \pmod p$$

也不属于这里，放到二次剩余与模平方根

CRT 与线性同余关系速查

问题	条件	返回形式
$a^{-1} \bmod m$	$\gcd(a,m)=1$	一个余数
$ax \equiv b \pmod m$	$\gcd(a,m) \mid b$	$x \equiv r \pmod{m/g}$
两个同余合并	$r_1 \equiv r_2 \pmod{\gcd(m_1,m_2)}$	$x \equiv r \pmod{\operatorname{lcm}(m_1,m_2)}$
多个同余方程	逐个合并均可行	`crt()`
多个线性模方程	每个可解且化简后可合并	`solve()`

8.5 离散对数、乘法阶与原根

解决问题： 求解

$$a^x \equiv b \pmod m$$

求元素 a 在模 m 乘法群中的阶，判断与寻找原根，并利用原根把乘法群问题转成指数上的线性同余
这一节建议保留两个主要代码块：

- `DiscreteLog`：BSGS / exBSGS，自包含，解决离散对数
- `ModGroup`：乘法阶、原根及中等范围试除辅助
  下面将接口统一为“返回最小非负解，无解返回 `-1`”，并使用 `__int128_t` 模乘覆盖 `i64` 模数

只要离散对数就整段复制 `DiscreteLog`；只要阶或原根就复制 `ModGroup`；模数很大时删掉里面的试除辅助，直接接 8.2 的 Pollard–Rho 分解结果

BSGS 与 exBSGS

```
struct DiscreteLog {
    struct Hash {
        static u64 splitmix64(u64 x) {
            x += 0x9e3779b97f4a7c15ULL;
```

```
            x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
            x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
            return x ^ (x >> 31);
        }
        size_t operator()(u64 x) const {
            static const u64 seed{ u64(chrono::steady_clock::now().time_since_epoch().count()) };
            return splitmix64(x + seed);
        }
    };
    static i64 norm(i64 x, i64 mod) {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }
    static i64 mul(i64 a, i64 b, i64 mod) {
        return i64((__int128_t)norm(a, mod) * norm(b, mod) % mod);
    }
private:
    // 解 k * a^x ≡ b (mod mod)，要求 gcd(a, mod) = 1
    static i64 bsgs_core(i64 a, i64 b, i64 mod, i64 k) {
        if (mod == 1) return 0;
        a = norm(a, mod), b = norm(b, mod), k = norm(k, mod);
        if (k == b) return 0;
        i64 m{ i64(sqrtl((long double)mod)) };
        while ((__int128_t)m * m < mod) ++ m;
        while (m > 0 && (__int128_t)(m - 1) * (m - 1) ≥ mod) -- m;
        unordered_map<u64, i64, Hash> mp;
        mp.reserve(size_t(m) * 2 + 3);
        mp.max_load_factor(0.7f);
        i64 cur{ b }, am{ 1 };
        for (i64 j = 0; j < m; ++ j) {
            mp[u64(cur)] = j;
            cur = mul(cur, a, mod);
            am = mul(am, a, mod);
        }
        i64 giant{ k };
        for (i64 i = 1; i ≤ m + 1; ++ i) {
            giant = mul(giant, am, mod);
            auto it = mp.find(u64(giant));
            if (it ≠ mp.end()) return i * m - it→second;
        }
        return -1;
    }
public:
    // gcd(a, mod) = 1，返回最小非负解，无解返回 -1
    static i64 bsgs(i64 a, i64 b, i64 mod) {
        if (mod ≤ 0 || gcd(a, mod) ≠ 1) return -1;
        return bsgs_core(a, b, mod, 1);
    }
    // 任意正模数，返回最小非负解，无解返回 -1
    static i64 exbsgs(i64 a, i64 b, i64 mod) {
        if (mod ≤ 0) return -1;
        if (mod == 1) return 0;
        a = norm(a, mod), b = norm(b, mod);
        if (b == 1) return 0;
        i64 k{ 1 }, add{};
        while (true) {
            i64 g{ gcd(a, mod) };
            if (g == 1) break;
            if (b % g) return -1;
            b /= g, mod /= g, ++ add;
            k = mod == 1 ? 0 : mul(k, a / g, mod);
            if (k == b) return add;
        }
        i64 x{ bsgs_core(a, b, mod, k) };
        return x < 0 ? -1 : x + add;
    }
};
```

接口说明

`bsgs(a,b,mod)` 求最小非负整数 x：

$$a^x \equiv b \pmod{mod}$$

要求

$$mod > 0, \quad \gcd(a, mod) = 1$$

无解返回 `-1`

```
i64 x = DiscreteLog::bsgs(a, b, mod);
if (x == -1) {
    // 无解
}
```

`exbsgs(a,b,mod)` 不要求 a 与 `mod` 互质：

```
i64 x = DiscreteLog::exbsgs(a, b, mod);
```

实际赛场上如果不确定是否互质，直接调用 `exbsgs()` 即可

BSGS 原理

设

$$M = \lceil \sqrt{m} \rceil$$

将指数写成

$$x = iM - j$$

则

$$a^{iM-j} \equiv b \pmod{m}$$

等价于

$$a^{iM} \equiv ba^j \pmod{m}$$

先预处理 baby step：

$$ba^j \quad 0 \le j < M$$

再依次枚举 giant step：

$$a^{iM} \quad 1 \le i \le M$$

发生碰撞时得到

$$x = iM - j$$

代码中同一 baby 值保留最大的 j：

```
mp[u64(cur)] = j;
```

并按 i 从小到大寻找，因此返回最小非负解

exBSGS 原理

当

$$g = \gcd(a, m) > 1$$

方程

$$a^x \equiv b \pmod{m}$$

若有正指数解，则必须满足

$$g \mid b$$

于是可以约去一个 g，不断得到更小的方程
维护已经约掉的次数 `add` 与左侧系数 `k`，最终化为

$$ka^y \equiv b \pmod{m'}$$

其中

$$\gcd(a, m') = 1$$

再交给 BSGS，最终

$$x = y + \text{add}$$

特殊情况

`exbsgs()` 将 $x = 0$ 时

$$a^0 = 1$$

因此：

```
DiscreteLog::exbsgs(a, 1, mod) == 0
```

对于

$$0^x \equiv 0 \pmod{m} \quad m > 1$$

最小非负解为

$$x = 1$$

模数为 1 时所有整数均同余，返回最小非负解 `0`

常见用法

普通离散对数：

```
i64 x = DiscreteLog::exbsgs(a, b, m);
```

例如

$$2^x \equiv 8 \pmod{13}$$

得到

```
x == 3
```

判断 b 是否属于 a 生成的子群：

```
bool in_group = DiscreteLog::bsgs(a, b, m) != -1;
```

其中要求

$$\gcd(a, m) = 1$$

注意事项

- BSGS / exBSGS 求的是**指数上的离散对数**，不要与实数上的 `log()` 混淆
- BSGS 的空间也是 $O(\sqrt{m})$，所以即使渐进复杂度看起来不错，`m` 接近完整 `i64` 范围时通常仍不可行
- 这里使用随机化整数哈希，避免普通 `unordered_map<i64,...>` 在恶意数据下出现严重冲突
- 如果题目模数只到 10^9 左右，也可以把哈希替换为 `map`，代码更简单但多一个对数
- 若已知指数只可能落在较短区间，优先利用该上界调整 BSGS，而不是机械按 $\sqrt{m}$ 建表

复杂度

设模数为 m
BSGS 期望时间：

$$O(\sqrt{m})$$

空间：

$$O(\sqrt{m})$$

exBSGS 前面的 gcd 消去只增加

$$O(\log m)$$

因此主复杂度仍为

$$O(\sqrt{m})$$

乘法阶与原根

```
struct ModGroup {
    static i64 norm(i64 x, i64 mod) {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }
    static i64 mul(i64 a, i64 b, i64 mod) {
        return i64((__int128_t)norm(a, mod) * norm(b, mod) % mod);
    }
    static i64 power(i64 a, i64 b, i64 mod) {
        i64 res{ 1 % mod };
        a = norm(a, mod);
        for (; b; b >>= 1, a = mul(a, a, mod)) {
            if (b & 1) res = mul(res, a, mod);
        }
        return res;
    }
    // ===== 试除辅助：已有 Rho 分解时可直接删除 =====
    static i64 phi_trial(i64 n) {
        i64 ans{ n };
        if (n % 2 == 0) {
            ans /= 2;
            while (n % 2 == 0) n /= 2;
        }
        for (i64 p = 3; p <= n / p; p += 2) if (n % p == 0) {
            ans = ans / p * (p - 1);
            while (n % p == 0) n /= p;
        }
        if (n > 1) ans = ans / n * (n - 1);
        return ans;
    }
    static vector<i64> factor_distinct_trial(i64 n) {
        vector<i64> fac;
        if (n % 2 == 0) {
            fac.push_back(2);
            while (n % 2 == 0) n /= 2;
        }
        for (i64 p = 3; p <= n / p; p += 2) if (n % p == 0) {
            fac.push_back(p);
```

```
            while (n % p == 0) n /= p;
        }
        if (n > 1) fac.push_back(n);
        return fac;
    }
    // ===== 乘法阶：phi = phi(mod)，fac_phi 为 phi 的不同质因子 =====
    static i64 order(i64 a, i64 mod, i64 phi, const vector<i64> &fac_phi) {
        if (mod <= 1 || gcd(a, mod) != 1) return -1;
        a = norm(a, mod);
        i64 ord{ phi };
        for (i64 p : fac_phi) {
            while (ord % p == 0 && power(a, ord / p, mod) == 1) ord /= p;
        }
        return ord;
    }
    // ===== 以下函数可直接删除：中等模数下自动试除 =====
    static i64 order_trial(i64 a, i64 mod) {
        if (mod <= 1 || gcd(a, mod) != 1) return -1;
        i64 phi{ phi_trial(mod) };
        return order(a, mod, phi, factor_distinct_trial(phi));
    }
    // 原根存在 iff mod = 2, 4, p^k, 2p^k, p 为奇质数
    static bool has_primitive_root_trial(i64 mod) {
        if (mod == 2 || mod == 4) return true;
        if (mod < 2) return false;
        if (mod % 2 == 0) mod /= 2;
        if (mod % 2 == 0) return false;
        i64 x{ mod }, p{ 3 };
        while (p <= x / p && x % p) p += 2;
        if (p > x / p) return true;
        while (x % p == 0) x /= p;
        return x == 1;
    }
    // ===== 原根：要求 mod 确实存在原根 =====
    static i64 primitive_root(i64 mod, i64 phi, const vector<i64> &fac_phi) {
        for (i64 g = 1; g < mod; ++ g) {
            if (gcd(g, mod) != 1) continue;
            bool ok{ true };
            for (i64 p : fac_phi) if (power(g, phi / p, mod) == 1) {
                ok = false;
                break;
            }
            if (ok) return g;
        }
        return -1;
    }
    // ===== 以下函数可直接删除：中等模数下自动判断并试除 =====
    static i64 primitive_root_trial(i64 mod) {
        if (!has_primitive_root_trial(mod)) return -1;
        i64 phi{ phi_trial(mod) };
        return primitive_root(mod, phi, factor_distinct_trial(phi));
    }
};
```

原根的判定基于检查 $\varphi(m)$ 的质因子：若对每个质因子 $q \mid \varphi(m)$ 都有 $g^{\varphi(m)/q} \not\equiv 1$，则 g 的阶达到 $\varphi(m)$

乘法阶

当

$$\gcd(a, m) = 1$$

定义 a 模 m 的乘法阶为最小正整数 k：

$$a^k \equiv 1 \pmod{m}$$

记作

$$\mathrm{ord}_m(a)$$

常用性质

由 Euler 定理：

$$\mathrm{ord}_m(a) \mid \varphi(m)$$

因此只要从 $\varphi(m)$ 开始不断尝试除掉质因子即可
若

$$d = \mathrm{ord}_m(a)$$

则

$$a^x \equiv 1 \pmod{m} \iff d \mid x$$

还有

$$\mathrm{ord}_m(a^k) = \frac{d}{\gcd(d,k)}$$

接口

中等范围直接：

```
i64 ord = ModGroup::order_trial(a, mod);
```

要求

$$mod > 1, \quad \gcd(a, mod) = 1$$

不满足时返回 `-1`
若已经知道

$$\varphi(mod)$$

及其不同质因子：

```
i64 ord = ModGroup::order(a, mod, phi, fac_phi);
```

其中 `fac_phi` 必须完整包含 $\varphi(mod)$ 的所有不同质因子

原理

初始令

$$d = \varphi(m)$$

对于每个质因子 $p \mid d$，只要

$$a^{d/p} \equiv 1 \pmod{m}$$

就说明真正的阶仍整除 d/p，因此：

```
while (ord % p == 0 && power(a, ord / p, mod) == 1) {
    ord /= p;
}
```

最终剩下的就是最小阶

大模数

`order_trial()` 内部使用 $O(\sqrt{m})$ 试除，因此主要面向中等范围
若 `mod` 很大，应复用 8.2 的 Pollard–Rho：

1. 分解 `mod` 得到 $\varphi(mod)$
2. 再分解 $\varphi(mod)$ 得到不同质因子

3. 调用 `order(a, mod, phi, fac_phi)`
   这样 `ModGroup` 不需要再复制整份 Pollard--Rho

复杂度

已知 $\varphi(m)$ 的不同质因子时，设

$$\varphi(m)=\prod p_i^{e_i}$$

需要快速幂次数至多

$$\sum_i e_i = O(\log m)$$

每次快速幂为 $O(\log m)$，粗略上界：

$$O(\log^2 m)$$

`order_trial()` 还需加上试除：

$$O(\sqrt{m})$$

原根

若

$$\mathrm{ord}_m(g)=\varphi(m)$$

则称 g 为模 m 的原根
这意味着

$$1, g, g^2, \dots, g^{\varphi(m)-1}$$

恰好遍历模 m 下所有与 m 互质的剩余类

原根存在条件

对于

$$m>1$$

原根存在当且仅当

$$m=2,\quad 4,\quad p^k,\quad 2p^k$$

其中 p 为奇质数，$k\ge 1$
因此并不是每个模数都有原根，例如：

$$8,\quad 12,\quad 15,\quad 16$$

都没有原根

判定原根

设

$$\varphi(m)=\prod q_i^{e_i}$$

在已知模 m 存在原根的前提下，g 为原根当且仅当

$$\gcd(g,m)=1$$

且对于所有不同质因子

$$q_i \mid \varphi(m)$$

都有

$$g^{\varphi(m)/q_i} \neq 1 \pmod{m}$$

因此无需真的调用一次完整的 `order()`

接口

中等范围直接：

```
i64 g = ModGroup::primitive_root_trial(mod);
```

若不存在原根返回 `-1`
若已知模数存在原根，并已经分解好 $\varphi(mod)$：

```
i64 g = ModGroup::primitive_root(mod, phi, fac_phi);
```

例如质数 p：

```
i64 phi = p - 1;
auto fac = ModGroup::factor_distinct_trial(phi);
i64 g = ModGroup::primitive_root(p, phi, fac);
```

由于质数模一定存在原根，不需要额外调用 `has_primitive_root_trial()`

全部原根

若 g 是模 m 的一个原根，则全部原根恰好为

$$g^k \pmod{m}$$

其中

$$1 \le k \le \varphi(m) \quad , \quad \gcd(k, \varphi(m)) = 1$$

所以原根数量为

$$\varphi(\varphi(m))$$

通常只需要一个原根，因此不建议默认枚举全部

大质数

若 p 很大但已知为质数，寻找原根只需要分解

$$p - 1$$

此时不要调用 `primitive_root_trial(p)`，因为它还会用试除法重新判断 p 的结构
更适合：

```
// fac 为通过 Pollard-Rho 得到的 p - 1 的不同质因子
i64 g = ModGroup::primitive_root(p, p - 1, fac);
```

复杂度

已知 $\varphi(m)$ 及其质因子后，每测试一个候选 g 需要

$$O(\omega(\varphi(m)) \log m)$$

其中 $\omega(n)$ 为不同质因子个数
设第一个原根为 g，则搜索总复杂度约为

$$O(g\,\omega(\varphi(m)) \log m)$$

`primitive_root_trial()` 还包含 $O(\sqrt{m})$ 的试除部分

同一小质数下大量离散对数

如果模数 p 较小、为质数，而且需要大量查询

$$g^x \equiv b \pmod{p}$$

则找到原根以后可以一次建立完整指数表，而不是每次跑 BSGS
这正是原根模板中 `pw[] / ind[]` 的用途

```
vector<int> primitive_log_table(int p, int g) {
    vector<int> ind(p, -1);
    i64 x{ 1 };
    for (int e = 0; e < p - 1; ++ e) {
        ind[x] = e;
        x = x * g % p;
    }
    return ind;
}
```

常见用法

```
int g = ModGroup::primitive_root_trial(p);
auto ind = primitive_log_table(p, g);
int x = ind[b];
```

对于

$$1 \le b < p$$

一定有

$$g^x \equiv b \pmod{p}$$

且

$$0 \le x < p - 1$$

复杂度

预处理：

$$O(p)$$

空间：

$$O(p)$$

之后单次离散对数查询：

$$O(1)$$

因此只适合“小模数 + 大量询问”

离散对数与乘法阶的关系

若

$$\gcd(a, m) = 1$$

并设

$$d = \mathrm{ord}_m(a)$$

则 a 生成的子群大小恰好是 d
若

$$a^{x_0} \equiv b \pmod{m}$$

有解，则所有指数解满足

$$x \equiv x_0 \pmod{d}$$

所以 BSGS 返回最小非负 x_0 后，可以用乘法阶描述全部解
特别地，如果 a 是原根：

$$d = \varphi(m)$$

用原根把乘法方程变成线性同余

这是原根最重要的实际用途之一
以质数模 p 为例，设 g 为原根，则任意非零数都可以唯一写成

$$a \equiv g^A \pmod{p}$$

考虑

$$x^k \equiv a \pmod{p}$$

令

$$x \equiv g^X \pmod{p}$$

则原方程变成

$$g^{kX} \equiv g^A \pmod{p}$$

因此只需要解决指数上的线性同余

$$kX \equiv A \pmod{p-1}$$

其中：

```
i64 g = ModGroup::primitive_root_trial(p);
i64 A = DiscreteLog::bsgs(g, a, p);
// 再交给 8.4 的 linear_congruence(k, A, p - 1)
```

设

$$d = \gcd(k, p-1)$$

若

$$d \nmid A$$

则无非零解
若可解，则模 p 下一共有 d 个不同非零解
下一节的二次剩余

$$x^2 \equiv a \pmod{p}$$

就是 $k = 2$ 的特殊情况，不过实际求平方根会使用更快、更专门的算法，不会真的跑一次原根 + BSGS

常用结论速查

$$\mathrm{ord}_m(a) = \min\{k > 0 : a^k \equiv 1 \pmod{m}\}$$

$$\mathrm{ord}_m(a) \mid \varphi(m)$$

$$a^x \equiv 1 \pmod{m} \iff \mathrm{ord}_m(a) \mid x$$

$$\mathrm{ord}_m(a^k) = \frac{\mathrm{ord}_m(a)}{\gcd(\mathrm{ord}_m(a), k)}$$

$$g \text{ 为原根} \iff \mathrm{ord}_m(g) = \varphi(m)$$

原根存在当且仅当

$$m = 2,\quad 4,\quad p^k,\quad 2p^k$$

若 g 为原根，则全部原根为

$$g^k \quad \gcd(k, \varphi(m)) = 1$$

若 $a^x \equiv b \pmod{m}$ 有解且 $\gcd(a, m) = 1$，则全部指数解为

$$x \equiv x_0 \pmod{\mathrm{ord}_m(a)}$$

选择方法

需求	方法
$a^x \equiv b \pmod{m}$ 且已知 $\gcd(a, m) = 1$	`bsgs()`
$a^x \equiv b \pmod{m}$，互质性未知	`exbsgs()`
求 a 的最小正周期	`order()` / `order_trial()`
求质数模原根	`primitive_root()`
一般模数判断并求原根	`primitive_root_trial()`
同一小质数下大量离散对数	原根 + 指数表
大模数求阶 / 原根	8.2 Pollard--Rho 分解后调用核心接口
$x^k \equiv a \pmod{p}$	原根 + 离散对数 + 8.4 线性同余
$x^2 \equiv a \pmod{p}$	下一节专门的模平方根算法

8.5.1 二次剩余与模平方根

解决问题： 判断

$$x^2 \equiv a \pmod{p}$$

是否有解，并在 p 为质数时求出模平方根；补充勒让德符号、Jacobi 符号以及合数模数下的处理框架
这里主板子用 Euler 判别 + Tonelli–Shanks，额外保留 Cipolla 作为可删除替代实现

勒让德符号

对于奇质数 p，勒让德符号定义为

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & p \mid a \\ 1 & a \not\equiv 0 \pmod{p} \text{ 且 } a \text{ 是二次剩余} \\ -1 & a \text{ 是二次非剩余} \end{cases}$$

Euler 判别：

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}$$

因此

$$a^{(p-1)/2} \equiv \begin{cases} 0 & p \mid a \\ 1 & a \text{ 是二次剩余} \\ -1 & a \text{ 是二次非剩余} \end{cases} \pmod{p}$$

常用结论：

$$\left(\frac{-1}{p}\right) = (-1)^{(p-1)/2} \quad , \quad \left(\frac{2}{p}\right) = (-1)^{(p^2-1)/8}$$

对不同奇质数 p, q，二次互反律：

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{(p-1)(q-1)/4}$$

质数模平方根

```
struct ModSqrt {
    static i64 norm(i64 x, i64 mod) {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }
    static i64 mul(i64 a, i64 b, i64 mod) {
        return i64((__int128_t)norm(a, mod) * norm(b, mod) % mod);
    }
    static i64 add(i64 a, i64 b, i64 mod) {
        return a >= mod - b ? a - (mod - b) : a + b;
    }
    static i64 power(i64 a, u64 b, i64 mod) {
        i64 res{ 1 % mod };
        a = norm(a, mod);
        for (; b; b >>= 1, a = mul(a, a, mod)) {
            if (b & 1) res = mul(res, a, mod);
        }
        return res;
    }
    // ===== 勒让德符号：要求 p 为奇质数 =====
    static int legendre(i64 a, i64 p) {
        assert(p > 2 && (p & 1));
        a = norm(a, p);
        if (!a) return 0;
        return power(a, (p - 1) / 2, p) == 1 ? 1 : -1;
    }
    // ===== Tonelli-Shanks：返回较小的平方根，无解返回 nullopt =====
    static optional<i64> sqrt_prime(i64 a, i64 p) {
        assert(p >= 2);
        a = norm(a, p);
        if (p == 2 || a == 0) return a;
        if (legendre(a, p) < 0) return nullopt;
        // p ≡ 3 (mod 4)
        if (p % 4 == 3) {
            i64 x{ power(a, p / 4 + 1, p) };
            return min(x, p - x);
        }
        // p ≡ 5 (mod 8)
        if (p % 8 == 5) {
            i64 x{ power(a, p / 8 + 1, p) };
            if (mul(x, x, p) != a) x = mul(x, power(2, (p - 1) / 4, p), p);
            return min(x, p - x);
        }
        // p - 1 = q * 2^s，q 为奇数
        i64 q{ p - 1 };
        int s{};
        while (!(q & 1)) q >>= 1, ++ s;
        // 找一个二次非剩余 z
        i64 z{ 2 };
```

```
        while (legendre(z, p) != -1) ++ z;
        i64 c{ power(z, q, p) };
        i64 x{ power(a, (q + 1) / 2, p) };
        i64 t{ power(a, q, p) };
        int m{ s };
        while (t != 1) {
            int i{ 1 };
            i64 tt{ mul(t, t, p) };
            while (tt != 1) tt = mul(tt, tt, p), ++ i;
            i64 b{ power(c, u64(1) << (m - i - 1), p) };
            x = mul(x, b, p);
            c = mul(b, b, p);
            t = mul(t, c, p);
            m = i;
        }
        return min(x, p - x);
    }
    // ===== 以下函数可直接删除：返回全部平方根 =====
    static vector<i64> roots_prime(i64 a, i64 p) {
        auto r = sqrt_prime(a, p);
        if (!r) return {};
        i64 x{ *r }, y{ norm(-x, p) };
        if (x == y) return { x };
        if (x > y) swap(x, y);
        return { x, y };
    }
    // ===== 以下函数可直接删除：Jacobi 符号，n 必须为正奇数 =====
    static int jacobi(i64 a, i64 n) {
        assert(n > 0 && (n & 1));
        a = norm(a, n);
        int res{ 1 };
        while (a) {
            while (!(a & 1)) {
                a >>= 1;
                i64 r{ n & 7 };
                if (r == 3 || r == 5) res = -res;
            }
            swap(a, n);
            if ((a & 3) == 3 && (n & 3) == 3) res = -res;
            a %= n;
        }
        return n == 1 ? res : 0;
    }
    // ===== 以下函数可直接删除：Cipolla 替代实现 =====
    static optional<i64> cipolla(i64 a, i64 p) {
        assert(p >= 2);
        a = norm(a, p);
        if (p == 2 || a == 0) return a;
        if (legendre(a, p) < 0) return nullopt;
        static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
        i64 w, v;
        do {
            w = i64(rng() % u64(p));
            v = norm(mul(w, w, p) - a, p);
        } while (legendre(v, p) != -1);
        struct E { i64 x, y; };
        auto prod = [&](E a, E b) {
            return E{
                add(mul(a.x, b.x, p), mul(mul(a.y, b.y, p), v, p), p),
                add(mul(a.x, b.y, p), mul(a.y, b.x, p), p)
            };
        };
        E res{ 1, 0 }, x{ w, 1 };
        u64 e{ u64(p / 2 + 1) };
        for (; e; e >>= 1, x = prod(x, x)) {
            if (e & 1) res = prod(res, x);
        }
        i64 r{ res.x };
        return min(r, p - r);
```

```
    }
};
```

接口说明

判断 a 是否为模奇质数 p 的非零二次剩余：

```
int s = ModSqrt::legendre(a, p);
if (s == 1) {
    // 非零二次剩余
} else if (s == 0) {
    // a ≡ 0 (mod p)
} else {
    // 二次非剩余
}
```

求一个平方根：

```
auto r = ModSqrt::sqrt_prime(a, p);
if (!r) {
    // 无解
} else {
    i64 x = *r;
}
```

返回的 `x` 满足

$$0 \le x \le \frac{p}{2} \quad , \quad x^2 \equiv a \pmod{p}$$

求全部平方根：

```
auto roots = ModSqrt::roots_prime(a, p);
```

对于奇质数 p：

- `a == 0` 时只有根 0
- 非零二次剩余恰有两个根 $x, p-x$
- 二次非剩余没有根

特殊模数的直接公式

若

$$p \equiv 3 \pmod{4}$$

并且 a 为二次剩余，则

$$x \equiv a^{(p+1)/4} \pmod{p}$$

所以这一情况只需要一次快速幂
若

$$p \equiv 5 \pmod{8}$$

先令

$$x = a^{(p+3)/8}$$

如果

$$x^2 \not\equiv a \pmod{p}$$

再令

$$x \leftarrow x \cdot 2^{(p-1)/4} \pmod{p}$$

真正需要完整 Tonelli–Shanks 的主要是

$$p \equiv 1 \pmod{8}$$

Tonelli–Shanks 原理

将

$$p - 1 = q2^s \quad , \quad q \text{ 为奇数}$$

选取一个二次非剩余 z，令

$$c = z^q, \quad x = a^{(q+1)/2}, \quad t = a^q$$

维护

$$x^2 \equiv at \pmod{p}, \quad t^{2^m} \equiv 1 \pmod{p}$$

若 $t \neq 1$，找最小的 i 使

$$t^{2^i} \equiv 1 \pmod{p}$$

再取

$$b = c^{2^{m-i-1}}$$

更新

$$x \leftarrow xb, \quad t \leftarrow tb^2, \quad c \leftarrow b^2, \quad m \leftarrow i$$

每轮都会严格缩小 m，最终得到

$$t = 1, \quad x^2 \equiv a \pmod{p}$$

Cipolla

来源中的实现采用 Cipolla：随机寻找 w 使

$$v = w^2 - a$$

为二次非剩余，然后在扩域

$$\mathbb{F}_p[\sqrt{v}]$$

中计算

$$(w + \sqrt{v})^{(p+1)/2}$$

其虚部最终为 0，实部就是一个模平方根
代码中的

```
cipolla(a, p)
```

与 `sqrt_prime()` 接口相同，同样返回较小根
实际板子中通常**二选一即可**

- 更习惯标准群论过程时保留 Tonelli–Shanks
- 想要实现结构固定、随机期望复杂度较好时可以保留 Cipolla

Jacobi 符号

Jacobi 符号将勒让德符号推广到任意正奇数 n
若

$$n = \prod_i p_i^{e_i}$$

则

$$\left(\frac{a}{n}\right) = \prod_i \left(\frac{a}{p_i}\right)^{e_i}$$

调用：

```
int s = ModSqrt::jacobi(a, n);
```

返回 `-1 / 0 / 1`
若 n 是奇质数，则 Jacobi 符号就是勒让德符号
但当 n 为合数时

$$\left(\frac{a}{n}\right) = 1$$

不能推出 a 是模 n 的二次剩余
例如

$$\left(\frac{2}{15}\right) = 1$$

但

$$x^2 \equiv 2 \pmod{15}$$

无解
因此一般合数模数不能只用 Jacobi 符号判断是否存在平方根

常见用法

判断是否有非零平方根

```
if (a % p && ModSqrt::legendre(a, p) == 1) {
    // 有两个不同根
}
```

求较小根

```
auto r = ModSqrt::sqrt_prime(a, p);
if (r) cout << *r << '\n';
```

输出全部根

```
auto roots = ModSqrt::roots_prime(a, p);
for (i64 x : roots) cout << x << ' ';
```

解二次方程

对于

$$Ax^2 + Bx + C \equiv 0 \pmod{p}$$

且

$$p \nmid 2A$$

判别式

$$\Delta = B^2 - 4AC \pmod{p}$$

先求

$$y^2 \equiv \Delta \pmod{p}$$

再由

$$x \equiv \frac{-B \pm y}{2A} \pmod{p}$$

得到两个候选根
代码中可直接结合上一节的逆元：

```
i64 delta = ModSqrt::norm(ModSqrt::mul(b, b, p) - ModSqrt::mul(ModSqrt::mul(4, a, p), c, p), p);
auto r = ModSqrt::sqrt_prime(delta, p);
if (r) {
    i64 inv = *inv_mod(2 * a % p, p);
    i64 x1 = ModSqrt::mul(ModSqrt::norm(-b + *r, p), inv, p);
    i64 x2 = ModSqrt::mul(ModSqrt::norm(-b - *r, p), inv, p);
}
```

奇素数幂

若模数不是质数，而是

$$p^e$$

其中 p 为奇质数，则先处理模 p 的平方根，再用 Hensel 提升
当

$$p \nmid a$$

时，模 p 的每个根都会**唯一**提升到模 p^e
已知

$$x^2 \equiv a \pmod{p^k}$$

令

$$x' = x + tp^k$$

需要满足

$$\frac{x^2 - a}{p^k} + 2xt \equiv 0 \pmod{p}$$

因此

$$t \equiv -\frac{x^2 - a}{p^k}(2x)^{-1} \pmod{p}$$

因为此时 $p \nmid x$，所以 $2x$ 一定存在模 p 逆元

$p \mid a$ 的情况

设模数为 p^e，p 为奇质数
若

$$a \equiv 0 \pmod{p^e}$$

则全部根满足

$$x \equiv tp^{\lceil e/2 \rceil} \pmod{p^e}$$

根的数量为

$$p^{\lfloor e/2 \rfloor}$$

否则令

$$v_p(a) = v < e$$

若 v 为奇数，则无解
若

$$v = 2s$$

令

$$a = p^{2s}u, \quad p \nmid u$$

则需要先解

$$y^2 \equiv u \pmod{p^{e-2s}}$$

若可解，则根的数量为

$$2p^s$$

因此 $p \mid a$ 时不能直接套“每个根唯一 Hensel 提升”的非奇异版本

一般合数模数

若

$$m = \prod_{i=1}^{k} p_i^{e_i}$$

求

$$x^2 \equiv a \pmod{m}$$

的一般流程：

1. 用 8.2 的整数分解得到全部 $p_i^{e_i}$
2. 分别求每个质数幂模数下的全部平方根
3. 对每种根的组合使用 8.4 的 CRT 合并
   例如如果两个互质模数分别有 r_1, r_2 个根，则合并后会有

   $$r_1 r_2$$

   个根
   所以一般合数模数的答案数量本身就可能很大，不适合把所有情况强行塞进一个短的 `sqrt_mod(m)` 接口

2^e 的补充结论

对奇数 a：

- 模 2 总有根
- 模 4 有根当且仅当

  $$a \equiv 1 \pmod{4}$$

- 当 $e \geq 3$ 时有根当且仅当

  $$a \equiv 1 \pmod{8}$$

且此时有 4 个不同平方根
因此一般合数模平方根中，2^e 需要单独处理，不能直接复用奇素数的 Hensel 公式

注意事项

`sqrt_prime(a,p)` 要求 `p` 为质数，函数本身不做 Miller--Rabin 判定
`legendre(a,p)` 要求 `p` 为奇质数
`jacobi(a,n)` 只要求 `n` 为正奇数，但 Jacobi 符号为 1 不代表合数模下存在平方根
对于奇质数 p 和非零二次剩余 a，两个平方根一定为

$$x \quad , \quad p-x$$

所以只要求任意一个根时无需继续搜索
板子使用 `__int128_t` 做模乘，可以安全处理 `i64` 范围内的正质数模数
当前 `sqrt_prime()` 返回两个根中较小的那个，因此结果具有稳定性

复杂度

`legendre()`：

$$O(\log p)$$

`jacobi()`：

$$O(\log n)$$

`p\equiv3\pmod4` 或 `p\equiv5\pmod8` 的快速求根：

$$O(\log p)$$

Tonelli--Shanks 在找到二次非剩余后：

$$O(\log^2 p)$$

额外空间：

$$O(1)$$

Cipolla 的随机选取二次非剩余期望尝试常数次，之后扩域快速幂：

$$O(\log p)$$

期望额外空间：

$$O(1)$$

8.5.2 Carmichael λ、欧拉降幂与幂塔取模

解决问题： 求乘法群的统一周期 Carmichael 函数 $\lambda(n)$，处理超大十进制指数，并计算右结合幂塔

$$a_0^{a_1^{a_2^{\cdot^{\cdot^{\cdot}}}}} \bmod m$$

已有单独的“欧拉降幂 / 扩展欧拉定理”内容，这里在其基础上补齐 Carmichael λ、非互质情况下的严格降幂和幂塔取模

Carmichael λ 函数

$\lambda(n)$ 定义为使所有与 n 互质的整数都满足

$$a^{\lambda(n)} \equiv 1 \pmod{n} \qquad \gcd(a,n)=1$$

的最小正整数
因此

$$\mathrm{ord}_n(a) \mid \lambda(n) \mid \varphi(n)$$

相比 Euler 定理，$\lambda(n)$ 给出的周期通常更短
输入 `fac = {{p1,e1},{p2,e2},...}` 为 n 的完整质因数分解

```
// 返回 {phi(n), lambda(n)}, fac = {{p, e}, ...}
pair<i64, i64> phi_lambda(const vector<pair<i64, int>> &fac) {
    __int128_t phi{ 1 };
    i64 lambda{ 1 };
    for (auto [p, e] : fac) {
        assert(p ⩾ 2 && e ⩾ 1);
        __int128_t pk{ 1 };
        for (int i = 0; i < e; ++ i) pk *= p;
        i64 ph{ i64(pk / p * (p - 1)) };
        i64 la{ p == 2 && e ⩾ 3 ? i64(pk / 4) : ph };
        phi *= ph;
        lambda = i64((__int128_t)(lambda / gcd(lambda, la)) * la);
    }
    return { i64(phi), lambda };
}
```

公式

对于奇质数 p:

$$\lambda(p^k) = \varphi(p^k) = p^{k-1}(p-1)$$

对于 2 的幂:

$$\lambda(2) = 1 \quad , \quad \lambda(4) = 2 \quad , \quad \lambda(2^k) = 2^{k-2} \ \ (k \geq 3)$$

不同质数幂之间取最小公倍数:

$$\lambda\left(\prod_i p_i^{e_i}\right) = \mathrm{lcm}_i\, \lambda(p_i^{e_i})$$

特别地:

$$\lambda(1) = 1$$

常见用法

若已经通过 8.2 Pollard--Rho 得到:

```
vector<pair<i64, int>> fac;
```

直接:

```
auto [phi, lambda] = phi_lambda(fac);
```

例如

$$n = 24 = 2^3 \cdot 3$$

有

$$\varphi(24) = 8, \quad \lambda(24) = 2$$

所以任意

$$\gcd(a, 24) = 1$$

都有

$$a^2 \equiv 1 \pmod{24}$$

与乘法阶、原根的关系

对任意

$$\gcd(a, m) = 1$$

都有

$$\mathrm{ord}_m(a) \mid \lambda(m)$$

因此上一节求乘法阶时，也可以从 $\lambda(m)$ 开始消质因子，而不是从 $\varphi(m)$ 开始
若模 m 存在原根，则单位群为循环群，因此

$$\lambda(m) = \varphi(m)$$

反过来，一般模数可能满足

$$\lambda(m) < \varphi(m)$$

例如

$$\lambda(8) = 2, \quad \varphi(8) = 4$$

复杂度

已知质因数分解后，设不同质因子数为 $\omega(n)$：

$$O(\omega(n) + \log n)$$

实际主要成本通常来自前面的整数分解

超大指数降幂

设指数 E 是一个无法放入整数类型的十进制字符串

互质时使用 Carmichael λ

若

$$\gcd(a, m) = 1$$

则

$$a^E \equiv a^{E \bmod \lambda(m)} \pmod{m}$$

这是处理互质底数时最短的周期

非互质时使用扩展 Euler 定理

任意 a 都可以使用：

$$a^E \equiv \begin{cases} a^E & E < \varphi(m) \\ a^{E \bmod \varphi(m) + \varphi(m)} & E \geq \varphi(m) \end{cases} \pmod{m}$$

关键不是只求

$$E \bmod \varphi(m)$$

还必须知道

$$E \geq \varphi(m)$$

是否成立

```
i64 norm_mod(i64 x, i64 mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
i64 mul_mod(i64 a, i64 b, i64 mod) {
    return i64((__int128_t)norm_mod(a, mod) * norm_mod(b, mod) % mod);
}
i64 qpow_mod(i64 a, u64 b, i64 mod) {
    i64 res{ 1 % mod };
    a = norm_mod(a, mod);
    for (; b; b >>= 1, a = mul_mod(a, a, mod)) {
        if (b & 1) res = mul_mod(res, a, mod);
    }
    return res;
}
// 返回 {E mod mod, E ≥ mod}
pair<i64, bool> decimal_mod_ge(string_view s, i64 mod) {
    assert(mod > 0 && !s.empty());
    i64 r{};
    __int128_t cap{};
    bool ge{};
    for (char c : s) {
        assert('0' ≤ c && c ≤ '9');
        int d{ c - '0' };
        r = i64(((__int128_t)r * 10 + d) % mod);
        if (!ge) {
            cap = cap * 10 + d;
            if (cap ≥ mod) ge = true;
        }
    }
    return { r, ge };
}
// 任意 a, phi = phi(mod)
i64 power_big(i64 a, string_view e, i64 mod, i64 phi) {
    assert(mod > 0 && phi > 0);
    if (mod == 1) return 0;
    auto [r, ge] = decimal_mod_ge(e, phi);
    u64 exp{ u64(r) + (ge ? u64(phi) : 0) };
    return qpow_mod(a, exp, mod);
}
// gcd(a, mod) = 1, lambda = lambda(mod)
i64 power_big_coprime(i64 a, string_view e, i64 mod, i64 lambda) {
    assert(mod > 0 && lambda > 0);
    assert(gcd(norm_mod(a, mod), mod) == 1);
    if (mod == 1) return 0;
    auto [r, ge] = decimal_mod_ge(e, lambda);
    return qpow_mod(a, u64(r), mod);
}
```

接口说明

任意底数:

```
i64 ans = power_big(a, exponent_string, mod, phi);
```

其中 `phi` 必须为

$$\varphi(mod)$$

若已知

$$\gcd(a, mod) = 1$$

优先:

```
i64 ans = power_big_coprime(a, exponent_string, mod, lambda);
```

其中

$$lambda = \lambda(mod)$$

为什么非互质时不能直接使用 λ

Carmichael 周期只保证可逆元素
例如

$$m = 8, \quad \lambda(8) = 2$$

取

$$a = 2, \quad E = 4$$

实际：

$$2^4 \equiv 0 \pmod 8$$

若错误地使用

$$E \bmod \lambda(8) + \lambda(8)$$

得到指数

$$4 \bmod 2 + 2 = 2$$

但

$$2^2 \equiv 4 \pmod 8$$

因此不互质时不能把扩展 Euler 定理中的 $\varphi(m)$ 直接替换成 $\lambda(m)$

> Tip
>
> 互质底数：优先 $\lambda(m)$
>
> 不保证互质：使用 $\varphi(m)$ 的扩展 Euler 降幂

幂塔取模

定义右结合幂塔

$$T_i = a_i^{T_{i+1}}$$

最后一层

$$T_{k-1} = a_{k-1}$$

例如

$$[2, 3, 2] \Longrightarrow 2^{3^2} = 512$$

下面约定

$$0^0 = 1$$

这样递归边界唯一，代码也与常见快速幂约定一致
内部使用 `phi_trial()` 计算每层模数的 Euler 函数，中等模数可以直接使用；完整 `i64` 大模数应将这里替换为 8.2 的 Pollard-Rho 分解

```
// ===== 以下函数可直接替换为 8.2 分解后的 phi =====
i64 phi_trial(i64 n) {
    assert(n ⩾ 1);
    i64 ans{ n };
    for (i64 p = 2; p ⩽ n / p; ++ p) if (n % p == 0) {
        ans = ans / p * (p - 1);
        while (n % p == 0) n /= p;
    }
    if (n > 1) ans = ans / n * (n - 1);
    return ans;
}
// min(a^e, cap)
u64 pow_cap(u64 a, u64 e, u64 cap) {
    if (!cap) return 0;
    __uint128_t res{ 1 }, x{ a };
    for (; e; e >>= 1) {
        if (e & 1) {
            res *= x;
            if (res ⩾ cap) return cap;
        }
        if (e >> 1) {
            x *= x;
            if (x ⩾ cap) x = cap;
        }
    }
    return u64(min<__uint128_t>(res, cap));
}
// min(T_l, cap)，用于判断真实指数是否 ⩾ cap
u64 tower_cap(const vector<u64> &a, int l, u64 cap) {
    if (!cap) return 0;
    u64 x{ a[l] };
    if (l + 1 == int(a.size())) return min(x, cap);
    if (x == 0) {
        u64 e{ tower_cap(a, l + 1, 1) };
        return min<u64>(e == 0, cap);
    }
    if (x == 1) return 1;
    if (cap == 1) return 1;
    // 最小 need，使 x^need ⩾ cap
    u64 need{}, v{ 1 };
    while (v < cap) {
        ++ need;
        if (x ⩾ cap || v > (cap - 1) / x) v = cap;
        else v *= x;
    }
    u64 e{ tower_cap(a, l + 1, need) };
    if (e ⩾ need) return cap;
    return pow_cap(x, e, cap);
}
i64 power_tower_mod_impl(const vector<u64> &a, int l, i64 mod) {
    if (mod == 1) return 0;
    if (l + 1 == int(a.size())) return i64(a[l] % u64(mod));
    i64 phi{ phi_trial(mod) };
    // 判断真实指数 T_{l+1} 是否已经达到 phi
    u64 small{ tower_cap(a, l + 1, u64(phi)) };
    u64 e;
    if (small < u64(phi)) {
        e = small; // 指数较小，必须使用真实值
    } else {
        e = u64(power_tower_mod_impl(a, l + 1, phi)) + u64(phi);
    }
    return qpow_mod(i64(a[l] % u64(mod)), e, mod);
}

i64 power_tower_mod(const vector<u64> &a, i64 mod) {
    assert(!a.empty() && mod > 0);
```

```
    return power_tower_mod_impl(a, 0, mod);
}
```

常见用法

```
vector<u64> a{ 2, 3, 2 };
i64 ans = power_tower_mod(a, mod);
```

计算

$$2^{3^2} \bmod mod$$

再例如：

```
vector<u64> a{ 3, 4, 5, 6 };
i64 ans = power_tower_mod(a, mod);
```

计算

$$3^{4^{5^6}} \bmod mod$$

为什么还需要 `tower_cap`

假设当前需要计算

$$a^E \bmod m$$

令

$$\phi = \varphi(m)$$

递归可以得到

$$E \bmod \phi$$

但仅有这个值无法判断应该使用

$$E \bmod \phi$$

还是

$$E \bmod \phi + \phi$$

因此额外计算

$$\min(E, \phi)$$

若结果小于 ϕ，说明指数还很小，可以直接使用真实指数
若结果等于 ϕ，说明

$$E \geq \phi$$

需要进入扩展 Euler 降幂

为什么 `tower_cap` 不会真的算出巨大幂

若需要判断

$$x^E \geq C$$

只需知道最小的

$$k$$

满足

$$x^k \geq C$$

对于

$$x \geq 2$$

有

$$k \leq \lceil \log_2 C \rceil$$

所以即使 E 是巨大的幂塔，也只需把下一层截断到一个很小的阈值

Euler φ 链

幂塔取模会不断递归：

$$m \to \varphi(m) \to \varphi(\varphi(m)) \to \cdots \to 1$$

这通常称为 Euler φ 链
链长很短，例如对 $m > 2$：

$$\varphi(m) < m$$

并且会很快下降到 1
因此幂塔即使高度很大，真正有意义的模递归层数也不会很深
若有大量相同模数的幂塔询问，可以预处理：

```
vector<i64> phi_chain(i64 m) {
    vector<i64> a{ m };
    while (m != 1) {
        m = phi_trial(m);
        a.push_back(m);
    }
    return a;
}
```

再复用整条链

λ 链与 φ 链

如果能保证每一层当前底数都与当前模数互质，理论上可以使用更短的

$$m \to \lambda(m) \to \lambda(\lambda(m)) \to \cdots$$

但一般幂塔不能保证这一点
所以通用幂塔模板使用 φ 链，而不是 λ 链

常见场景

巨大十进制指数

$$a^E \bmod m$$

其中 E 是长度 10^5 的字符串：

```
auto [phi, lambda] = phi_lambda(fac);

if (gcd(a, m) == 1) {
    ans = power_big_coprime(a, E, m, lambda);
} else {
```

```
    ans = power_big(a, E, m, phi);
}
```

指数本身又是幂

$$a^{b^c} \bmod m$$

直接：

```
vector<u64> tower{ u64(a), u64(b), u64(c) };
i64 ans = power_tower_mod(tower, m);
```

很高的固定底数幂塔

例如

$$2^{2^{2^{2^{\cdot^{\cdot^{\cdot}}}}}} \bmod m$$

只需要把足够多层的 `2` 放入数组
当高度超过 φ 链有效深度后，再增加高度往往不会增加递归深度

注意事项

`phi_lambda()` 要求传入 n 的完整质因数分解
`power_big_coprime()` 必须满足

$$\gcd(a, m) = 1$$

非互质时只能使用 `power_big()` 的扩展 Euler 规则
扩展 Euler 定理的关键判断是

$$E \geq \varphi(m)$$

不能只保存

$$E \bmod \varphi(m)$$

`power_tower_mod()` 当前只处理非负整数底数

```
vector<u64>
```

幂塔默认右结合，不能误写成

$$((a_0^{a_1})^{a_2})^{\cdots}$$

板子约定

$$0^0 = 1$$

若题目另有定义，需要单独修改
`phi_trial()` 是方便独立复制的短依赖，大模数应换成 8.2 的分解结果

复杂度

`phi_lambda()` 在已知分解后：

$$O(\log n)$$

`decimal_mod_ge()` 对长度为 L 的十进制指数：

$$O(L)$$

一次模快速幂：

$$O(\log E')$$

其中降幂后的指数

$$E' < 2\varphi(m)$$

幂塔本身的递归层数受 Euler φ 链限制
若各层 φ 已经给出或快速得到，模运算部分可以粗略看作

$$O(\log^2 m)$$

当前自包含版本使用 `phi_trial()`，实际复杂度主要取决于每层求 φ 的试除成本
完整 `i64` 范围下应接 8.2 Pollard--Rho，而不是依赖试除

8.6 floor_sum 与欧几里得类算法

解决问题： 快速计算线性取整和

$$\sum_{i=0}^{n-1} \left\lfloor \frac{ai+b}{m} \right\rfloor$$

以及进一步计算

$$\sum_{i=0}^{n-1} i \left\lfloor \frac{ai+b}{m} \right\rfloor, \quad \sum_{i=0}^{n-1} \left\lfloor \frac{ai+b}{m} \right\rfloor^2$$

核心过程与欧几里得算法相同：先提出整除部分，再交换分子斜率与分母继续递归，因此复杂度为对数级

floor_sum

下面统一使用数学意义的 `floor`，因此允许 `a,b` 为负数，并使用 `__int128_t` 保存中间结果和答案

```
__int128_t floor_sum(__int128_t n, __int128_t m, __int128_t a, __int128_t b) {
    assert(n ⩾ 0 && m > 0);
    // 先把 a、b 规范到 [0, m)，同时提出整数部分
    __int128_t qa{ a / m }, ra{ a % m };
    if (ra < 0) -- qa, ra += m;
    __int128_t qb{ b / m }, rb{ b % m };
    if (rb < 0) -- qb, rb += m;
    __int128_t ans{ n * (n - 1) / 2 * qa + n * qb };
    a = ra, b = rb;
    for (;;) {
        if (a ⩾ m) ans += n * (n - 1) / 2 * (a / m), a %= m;
        if (b ⩾ m) ans += n * (b / m), b %= m;
        __int128_t y{ a * n + b };
        if (y < m) break;
        n = y / m;
        b = y % m;
        swap(m, a);
    }
    return ans;
}
```

接口说明

```
auto ans = floor_sum(n, m, a, b);
```

返回

$$\sum_{i=0}^{n-1} \left\lfloor \frac{ai+b}{m} \right\rfloor$$

要求

$$n \geq 0, \quad m > 0$$

`a,b` 可以为任意有符号整数
原始常见模板通常要求

$$0 \leq a, b < m$$

或使用无符号数，这里额外做了负数规范化，避免 C++ 的整数除法向 0 截断与数学 `floor` 不一致

原理

首先将

$$a = q_a m + r_a, \quad 0 \leq r_a < m$$

$$b = q_b m + r_b, \quad 0 \leq r_b < m$$

则

$$\left\lfloor \frac{ai + b}{m} \right\rfloor = q_a i + q_b + \left\lfloor \frac{r_a i + r_b}{m} \right\rfloor$$

因此整数部分贡献为

$$q_a \frac{n(n-1)}{2} + q_b n$$

接下来只需考虑

$$0 \leq a, b < m$$

令

$$y = an + b$$

若

$$y < m$$

则所有项均为 0
否则通过格点区域转置，可将剩余问题转化为参数

$$n' = \left\lfloor \frac{an + b}{m} \right\rfloor$$

并交换

$$a \leftrightarrow m$$

继续处理
参数变化与欧几里得算法一致，因此只会进行 $O(\log m)$ 轮

常见用法

区间

$$L \leq i \leq R$$

上的取整和：

```
auto ans = floor_sum((__int128_t)R - L + 1, m, a, (__int128_t)a * L + b);
```

因为令

$$i = L + j$$

即可得到

$$\sum_{i=L}^{R} \left\lfloor \frac{ai+b}{m} \right\rfloor = \text{floor_sum}\,(R-L+1, m, a, aL+b)$$

若求

$$\sum_{i=0}^{n-1} \left\lceil \frac{ai+b}{m} \right\rceil$$

利用

$$\lceil x \rceil = -\lfloor -x \rfloor$$

直接：

```
auto ans = -floor_sum(n, m, -a, -b);
```

若题目使用下标 $0 \dots N$：

```
auto ans = floor_sum((__int128_t)N + 1, m, a, b);
```

注意事项

这里的 `n` 表示项数，下标范围固定为

$$0 \le i < n$$

不是 $0 \le i \le n$
答案可能远大于 `i64`，因此返回 `__int128_t`
如果题目保证结果在 `i64` 内，可以在调用后转换：

```
i64 ans = i64(floor_sum(n, m, a, b));
```

不要把普通 C++ 表达式

```
(a * i + b) / m
```

直接当作数学 floor 使用，若分子为负数，两者可能不同

复杂度

时间：

$$O(\log m + \log(|a|+1))$$

通常简记为

$$O(\log \max(|a|, m))$$

额外空间：

$$O(1)$$

类欧几里得三类和

有些题目不只需要普通 `floor_sum`，还会出现

$$f = \sum_{i=0}^{n-1} y_i, \quad g = \sum_{i=0}^{n-1} i y_i, \quad h = \sum_{i=0}^{n-1} y_i^2$$

其中

$$y_i = \left\lfloor \frac{ai+b}{m} \right\rfloor$$

传统类欧几里得正是同时维护这三个量
下面将常见的 $0 \dots N$ 递推统一改成与 `floor_sum` 一致的 $[0, n)$ 接口

```
struct FloorSums {
    __int128_t f{}, g{}, h{};
};
// 0 ≤ a, b < c, 计算 i = 0..n
FloorSums floor_sums_nonneg(__int128_t a, __int128_t b, __int128_t c, __int128_t n) {
    if (n < 0) return {};
    // ===== 提取整数部分 =====
    if (a ≥ c || b ≥ c) {
        __int128_t A{ a / c }, B{ b / c };
        auto t = floor_sums_nonneg(a % c, b % c, c, n);
        __int128_t s1{ n * (n + 1) / 2 }, s2{ n * (n + 1) * (2 * n + 1) / 6 }, cnt{ n + 1 };
        return { t.f + A * s1 + B * cnt, t.g + A * s2 + B * s1,
            t.h + A * A * s2 + B * B * cnt + 2 * A * B * s1 + 2 * A * t.g + 2 * B * t.f
        };
    }

    // ===== 交换坐标轴，进入下一层欧几里得递归 =====
    __int128_t k{ (a * n + b) / c };
    if (!k) return {};
    auto t = floor_sums_nonneg(c, c - b - 1, a, k - 1);
    __int128_t f{ n * k - t.f }, g{ (n * (n + 1) * k - t.f - t.h) / 2 };
    __int128_t h{ n * k * (k + 1) - 2 * t.g - 2 * t.f - f };
    return { f, g, h };
}
// ===== 对外接口：n 项，允许 a、b 为负 =====
FloorSums floor_sums(__int128_t n, __int128_t m, __int128_t a, __int128_t b) {
    assert(n ≥ 0 && m > 0);
    if (!n) return {};
    __int128_t A{ a / m }, aa{ a % m };
    if (aa < 0) -- A, aa += m;
    __int128_t B{ b / m }, bb{ b % m };
    if (bb < 0) -- B, bb += m;
    auto t = floor_sums_nonneg(aa, bb, m, n - 1);
    __int128_t s1{ n * (n - 1) / 2 }, s2{ n * (n - 1) * (2 * n - 1) / 6 };
    __int128_t f{ t.f }, g{ t.g };
    t.h += A * A * s2 + B * B * n + 2 * A * B * s1 + 2 * A * g + 2 * B * f;
    t.g += A * s2 + B * s1;
    t.f += A * s1 + B * n;
    return t;
}
```

接口说明

```
auto [f, g, h] = floor_sums(n, m, a, b);
```

令

$$y_i = \left\lfloor \frac{ai+b}{m} \right\rfloor, \quad 0 \le i < n$$

返回：

$$f = \sum_{i=0}^{n-1} y_i$$

$$g = \sum_{i=0}^{n-1} i y_i$$

$$h = \sum_{i=0}^{n-1} y_i^2$$

其中

$$n \geq 0, \quad m > 0$$

`a,b` 可以为负数

提取整数部分

写成

$$a = Am + a', \quad b = Bm + b'$$

则

$$y_i = Ai + B + z_i$$

其中

$$z_i = \left\lfloor \frac{a'i + b'}{m} \right\rfloor$$

于是

$$f = f' + A\sum i + B\sum 1$$

$$g = g' + A\sum i^2 + B\sum i$$

$$h = h' + A^2\sum i^2 + B^2\sum 1 + 2AB\sum i + 2Ag' + 2Bf'$$

需要的基础和为

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}, \quad \sum_{i=0}^{n-1} i^2 = \frac{n(n-1)(2n-1)}{6}$$

互换递推

规范化到

$$0 \leq a, b < c$$

后，定义

$$k = \left\lfloor \frac{an + b}{c} \right\rfloor$$

传统类欧几里得通过交换坐标轴，把问题转换为参数

$$(c, c - b - 1, a, k - 1)$$

常见用法

只需要

$$\sum \left\lfloor \frac{ai + b}{m} \right\rfloor$$

时优先使用前面的 `floor_sum()`，代码更短、常数更小
若同时需要：

```
auto s = floor_sums(n, m, a, b);
auto sum_floor = s.f;
auto sum_i_floor = s.g;
auto sum_floor_sq = s.h;
```

例如要求

$$\sum_{i=0}^{n-1}(ui+v)\left\lfloor\frac{ai+b}{m}\right\rfloor$$

直接利用

$$ug+vf$$

代码：

```
auto s = floor_sums(n, m, a, b);
__int128_t ans = (__int128_t)u * s.g + (__int128_t)v * s.f;
```

若要求

$$\sum\left(\left\lfloor\frac{ai+b}{m}\right\rfloor+c\right)^2$$

则：

```
auto s = floor_sums(n, m, a, b);
__int128_t ans = s.h + 2 * (__int128_t)c * s.f + (__int128_t)c * c * n;
```

注意事项

`floor_sums_nonneg()` 是内部递归，要求

$$c>0,\quad a,b\geq 0$$

正常使用只调用：

```
floor_sums(n, m, a, b)
```

递归中出现

```
floor_sums_nonneg(c, c - b - 1, a, k - 1)
```

时若 `k > 0`，必有 `a > 0`，因此不会出现除以 0
三类和都可能迅速超过 `i64`，统一使用 `__int128_t`
如果只需要 `f`，不要为了复用而调用 `floor_sums()`，直接使用 `floor_sum()` 更合适

复杂度

每次递归都会进行一次与欧几里得算法相同的参数交换和取模，因此递归层数为

$$O(\log m)$$

每层只进行常数次运算，所以：

$$O(\log m)$$

额外递归空间：

$$O(\log m)$$

与普通数论分块的区别

8.3.1 中的数论分块处理的是

$$\left\lfloor\frac{n}{i}\right\rfloor$$

利用商在连续区间内相同，将 $O(n)$ 枚举压到 $O(\sqrt{n})$
这里处理的是

$$\left\lfloor \frac{ai+b}{m} \right\rfloor$$

利用格点区域转置与欧几里得递归，将复杂度降到 $O(\log m)$
两者虽然都大量使用 floor，但优化思想不同：

类型	典型形式	核心思想	复杂度
数论分块	$\lfloor n/i \rfloor$	相同商分块	$O(\sqrt{n})$
`floor_sum`	$\lfloor (ai+b)/m \rfloor$	欧几里得递归	$O(\log m)$

8.6.1 万能欧几里得与线性模最值

解决问题： 将

$$y_i = \left\lfloor \frac{pi+r}{q} \right\rfloor$$

对应的格点路径抽象为一串 `U` / `R` 操作，在只要求操作满足结合律时仍能用欧几里得递归快速求整段操作的合成结果；同时求

$$\min_{0\le i<n} (ai+b) \bmod m \quad , \quad \max_{0\le i<n} (ai+b) \bmod m$$

万能欧几里得

考虑直线

$$y = \frac{px+r}{q}, \quad 0 < x \le n$$

生成一个操作序列：

- 穿过一条横线执行一次 `U`
- 穿过一条竖线执行一次 `R`
- 同时穿过整点时先 `U` 后 `R`
  因此第 i 个 `R` 前恰有

$$\left\lfloor \frac{pi+r}{q} \right\rfloor$$

个 `U`
`T{}` 为单位元，`operator+` 表示**按先左后右顺序合并两段操作**，只要求满足结合律，不要求交换律

```
template<class T>
T repeat(T a, i64 k) {
    T res{};
    for (; k; k >>= 1, a = a + a) {
        if (k & 1) res = res + a;
    }
    return res;
}
template<class T>
T euclid_word_impl(i64 p, i64 q, i64 r, i64 n, T U, T R) {
    if (!n) return T{};
    // ===== 提取斜率的整数部分 =====
    if (p >= q) {
        return euclid_word_impl(p % q, q, r, n, U, repeat(U, p / q) + R);
    }
    // ===== 此时 0 <= p < q，交换横纵坐标递归 =====
    i64 h{ i64((__int128_t(p) * n + r) / q) };
    if (!h) return repeat(R, n);
    i64 pre{ (q - r - 1) / p };
    i64 suf{ n - i64((__int128_t(q) * h - r - 1) / p) };
    return repeat(R, pre) + U
         + euclid_word_impl(q, p, (q - r - 1) % p, h - 1, R, U)
         + repeat(R, suf);
}
// p >= 0, q > 0, r >= 0, n >= 0
template<class T>
```

```
T euclid_word(i64 p, i64 q, i64 r, i64 n, T U, T R) {
    assert(p ≥ 0 && q > 0 && r ≥ 0 && n ≥ 0);
    if (!n) return T{};
    return repeat(U, r / q) + euclid_word_impl(p, q, r % q, n, U, R);
}
```

接口说明

```
T ans = euclid_word(p, q, r, n, U, R);
```

返回直线

$$y = \frac{px + r}{q}, \quad 0 < x \le n$$

生成的完整操作序列的合成结果
要求：

$$p \ge 0, \quad q > 0, \quad r \ge 0, \quad n \ge 0$$

并且类型 `T` 满足：

```
T{}        // 单位元
a + b      // 先执行 a，再执行 b
(a + b) + c == a + (b + c)
```

`operator+` **不需要满足交换律**，矩阵乘法、仿射变换、状态转移以及自定义区间摘要都可以使用

为什么能做任意结合操作

普通类欧几里得通常针对某个具体和式单独推：

$$\sum y_i, \quad \sum i y_i, \quad \sum y_i^2$$

万能欧几里得不再直接维护这些量，而是先求出 `U / R` 操作串的**压缩表示**
只要某个问题能设计出：

```
U：跨过一条横线后的状态变化
R：跨过一条竖线后的状态变化
```

并且相邻两段可以结合，就可以直接复用同一套递归

递推：斜率不小于 1

若

$$p = kq + p', \quad 0 \le p' < q$$

则每次 `R` 前都会额外多出 k 个 `U`
因此：

$$R \longrightarrow U^k R$$

对应代码：

```
euclid_word_impl(p % q, q, r, n, U, repeat(U, p / q) + R)
```

递推：斜率小于 1

设

$$0 < p < q, \quad h = \left\lfloor \frac{pn + r}{q} \right\rfloor$$

交换横纵轴后，`U / R` 的角色互换，新的参数为

$$(q, p, q - r - 1, h - 1)$$

两端还需要补上若干 `R`，于是得到：

```
repeat(R, pre) + U
+ euclid_word_impl(q, p, (q - r - 1) % p, h - 1, R, U)
+ repeat(R, suf)
```

这正是万能欧几里得的核心递归结构

万能欧几里得使用示例

例如重新计算

$$\sum_{i=1}^{n} \left\lfloor \frac{ai+b}{m} \right\rfloor$$

只需要令状态记录：

- `x`：出现多少个 `R`
- `y`：出现多少个 `U`
- `sum`：每次 `R` 时当前 `y` 的总和

```
struct FloorPath {
    i64 x{}, y{};
    __int128_t sum{};
    friend FloorPath operator+(const FloorPath &a, const FloorPath &b) {
        return { a.x + b.x, a.y + b.y, a.sum + b.sum + (__int128_t)a.y * b.x };
    }
};
```

调用：

```
FloorPath U{ 0, 1, 0 };
FloorPath R{ 1, 0, 0 };
auto s = euclid_word(a, m, b, n, U, R);
auto ans = s.sum;
```

得到

$$ans = \sum_{i=1}^{n} \left\lfloor \frac{ai+b}{m} \right\rfloor$$

因为把两段 `A + B` 拼起来时，右半段的每一个 `R` 都会额外看到左半段的 `a.y` 个 `U`，所以：

$$sum_{A+B} = sum_A + sum_B + y_A x_B$$

> Tip
>
> 如果只需要普通 `floor_sum` 或上一节的 f, g, h 三类和，直接使用上一节专门板子更短
> 万能欧几里得的价值主要在于**题目要求的状态变了，但操作仍能用结合律合并**

常见设计方式

`T` 常见可以设计成：

- 矩阵，`+` 表示矩阵乘法
- 仿射变换，`+` 表示函数复合
- 自定义统计量，合并时计算左右两段及跨段贡献
- 维护 x, y 的低次多项式和，例如 $\sum x^a y^b$
  因此万能欧几里得特别适合：

$$\sum P\left(i, \left\lfloor \frac{ai+b}{m} \right\rfloor\right)$$

  其中 P 为低次多项式，或对应操作能压缩进固定维状态

注意事项

`euclid_word()` 的下标约定是

$$1 \le i \le n$$

而上一节的 `floor_sum()` 使用

$$0 \le i < n$$

不要直接混用参数
模板要求 $p, r \ge 0$，负系数的普通取整和应优先使用上一节支持负数的 `floor_sum`
`T{}` 必须是真正的单位元
例如矩阵类型默认构造若是零矩阵，就不能直接使用，必须让默认构造表示单位矩阵，或者修改 `repeat()` 的单位元来源
组合顺序不能写反：

```
a + b
```

表示先执行 `a` 再执行 `b`
即使 `operator+` 不可交换也完全合法

复杂度

若一次 `T + T` 的复杂度为 C，欧几里得递归层数与快速重复总操作次数均为对数级：

$$O(C \log \max(p, q))$$

递归空间：

$$O(\log \max(p, q))$$

线性模最值

给定

$$r_i = (ai + b) \bmod m, \quad 0 \le i < n$$

求：

$$\min r_i, \quad \max r_i$$

```
i64 norm_mod(i64 x, i64 mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
// min_{0 ≤ i < n} (a * i + b) mod m
i64 min_mod_linear(i64 n, i64 m, i64 a, i64 b) {
    assert(n > 0 && m > 0);
    a = norm_mod(a, m);
    __int128_t bb{ norm_mod(b, m) };
    // ===== 完整覆盖一个周期时可以直接返回 =====
    i64 g{ gcd(a, m) };
    if (n ≥ m / g) return i64(bb % g);
    i64 ans{ m };
    for (; n; swap(a, m)) {
        a %= m;
        bb %= m;
        if (bb < 0) bb += m;
        ans = min(ans, i64(bb));
        __int128_t y{ __int128_t(n - 1) * a + bb };
        n = i64(y / m);
        bb -= __int128_t(m) * n;
    }
    return ans;
}
// 同时返回 {最小值, 最大值}
pair<i64, i64> mod_extrema_linear(i64 n, i64 m, i64 a, i64 b) {
    assert(n > 0 && m > 0);
    i64 aa{ norm_mod(a, m) }, bb{ norm_mod(b, m) };
```

```
    i64 g{ gcd(aa, m) };
    // ===== 已覆盖完整周期 =====
    if (n ≥ m / g) {
        i64 r{ bb % g };
        return { r, r + m - g };
    }
    i64 mn{ min_mod_linear(n, m, aa, bb) };
    // m - 1 - r_i = (-a * i - b - 1) mod m
    i64 na{ aa ? m - aa : 0 };
    i64 nb{ m - 1 - bb };
    i64 mx{ m - 1 - min_mod_linear(n, m, na, nb) };
    return { mn, mx };
}
```

接口说明

最小值:

```
i64 mn = min_mod_linear(n, m, a, b);
```

返回:

$$\min_{0\le i<n}(ai+b) \bmod m$$

同时求最小值与最大值:

```
auto [mn, mx] = mod_extrema_linear(n, m, a, b);
```

要求:

$$n>0,\quad m>0$$

`a,b` 可以为负数

周期

令

$$g=\gcd(a,m)$$

则:

$$(ai+b) \bmod m$$

的周期长度为

$$\frac{m}{g}$$

一个完整周期中恰好出现所有满足

$$x\equiv b \pmod{g}$$

的模 m 剩余类
因此若

$$n\ge\frac{m}{g}$$

令

$$r=b \bmod g$$

则可以直接得到:

$$\min=r,\quad \max=r+m-g$$

这也是板子中的 $O(1)$ 快速出口

最小值递推原理

先规范化：

$$0 \le a, b < m$$

当前一定要考虑候选值 b，随后计算：

$$n' = \left\lfloor \frac{(n-1)a + b}{m} \right\rfloor$$

并将截距变成：

$$b' = b - mn'$$

随后交换：

$$a \leftrightarrow m$$

新的问题规模与欧几里得算法一致
代码核心因此只有：

```
ans = min(ans, i64(bb));

__int128_t y{ __int128_t(n - 1) * a + bb };
n = i64(y / m);
bb -= __int128_t(m) * n;
swap(a, m);
```

不断交换直到 `n == 0`

最大值为什么可以转成最小值

对于任意：

$$0 \le r < m$$

有：

$$m - 1 - r = (-r - 1) \bmod m$$

于是：

$$\max r_i = m - 1 - \min\Big((-ai - b - 1) \bmod m\Big)$$

所以最大值不需要再写一套欧几里得递推

常见用法

普通最小模值：

```
i64 ans = min_mod_linear(n, m, a, b);
```

同时求最值：

```
auto [mn, mx] = mod_extrema_linear(n, m, a, b);
```

求区间

$$L \le i \le R$$

上的最值，只需要平移下标
令

$$i = L + j$$

则：

$$ai + b = aj + (aL + b)$$

代码：

```
i64 len = R - L + 1;
__int128_t shift = (__int128_t)a * L + b;
i64 bb = i64(shift % m);
if (bb < 0) bb += m;

auto [mn, mx] = mod_extrema_linear(len, m, a, bb);
```

求最值所在位置

若已经得到最小值 `mn`，要求某个达到最小值的下标，只需解：

$$ai + b \equiv mn \pmod{m}$$

即：

$$ai \equiv mn - b \pmod{m}$$

可以直接交给 8.4 的线性同余：

```
i64 aa = norm_mod(a, m);
i64 bb = norm_mod(b, m);
i64 rhs = norm_mod(mn - bb, m);

auto sol = Congruence::linear(aa, rhs, m);
auto [i, period] = *sol;
```

因为 `mn` 确实来自区间 $0 \le i < n$，所以返回的最小非负解必满足：

$$i < n$$

最大值位置同理，将 `mn` 换成 `mx`

与 floor_sum 的关系

两者处理的都是线性函数经过整除或取模后的序列，但目标不同：

问题	形式	算法
取整和	$\sum \lfloor (ai+b)/m \rfloor$	`floor_sum`
复杂取整路径统计	自定义 U/R 状态	万能欧几里得
模最小值	$\min(ai+b) \bmod m$	`min_mod_linear`
模最大值	$\max(ai+b) \bmod m$	最小值取补

注意事项

`min_mod_linear()` 中的 `%` 每轮都会重新规范化 `bb`，这是因为递归过程中 `bb` 可以变成负数中间量：

$$(n-1)a + b$$

和

$$mn$$

的计算使用 `__int128_t`，避免 `i64` 乘法溢出
`min_mod_linear()` 要求 `n > 0`，空区间没有最小值
当

$$a \equiv 0 \pmod{m}$$

整个序列为常数：

$$b \bmod m$$

模板中的完整周期出口会直接处理
如果只需要判断序列中是否出现 0，也可以直接解：

$$ai \equiv -b \pmod{m}$$

并检查最小非负解是否小于 n，这比先求最小值语义更直接

复杂度

万能欧几里得：

$$O(\log \max(p, q))$$

次状态合并
线性模最小值：

$$O(\log m)$$

线性模最大值：

$$O(\log m)$$

同时求最小值与最大值：

$$O(\log m)$$

全部额外空间均为 $O(1)$，万能欧几里得递归版额外递归栈为

$$O(\log \max(p, q))$$

8.6.2 Stern-Brocot、连分数与有理逼近

解决问题： 在只使用整数运算的前提下处理有理数搜索、最佳逼近、Stern-Brocot 路径、连分数展开与收敛分数，以及模意义下的小分数还原
“取模还原分数”和带分子、分母上界的 Stern-Brocot 二分，这里将它们与连分数统一整理，并将 Stern-Brocot 搜索改成显式有界跳跃，避免逐层走树

分数基础

后面的所有接口统一使用约分后的

$$\frac{p}{q}, \quad q > 0$$

```
struct Frac {
    i64 p{}, q{ 1 };
};
Frac make_frac(i64 p, i64 q) {
    assert(q != 0);
    if (q < 0) p = -p, q = -q;
    i64 g{ gcd(p < 0 ? -p : p, q) };
    return { p / g, q / g };
}
int frac_cmp(const Frac &a, const Frac &b) {
    __int128_t x{ (__int128_t)a.p * b.q };
    __int128_t y{ (__int128_t)b.p * a.q };
```

```
    return x < y ? -1 : x > y ? 1 : 0;
}
```

`frac_cmp(a,b)` 返回：

- `-1`：$a < b$
- `0`：$a = b$
- `1`：$a > b$

连分数

任意有理数均可写成有限简单连分数

$$\frac{p}{q} = [a_0; a_1, a_2, \ldots, a_k] = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cdots}}$$

其中对标准表示有

$$a_i > 0 \quad (i \geq 1)$$

连分数展开本质上就是欧几里得算法的商序列

```
pair<i64, i64> floor_divmod(i64 a, i64 b) {
    assert(b > 0);
    i64 q{ a / b }, r{ a % b };
    if (r < 0) -- q, r += b;
    return { q, r };
}
// p / q 的简单连分数，允许 p < 0，要求 q > 0
vector<i64> continued_fraction(i64 p, i64 q) {
    assert(q > 0);
    vector<i64> a;
    while (q) {
        auto [k, r] = floor_divmod(p, q);
        a.push_back(k);
        p = q, q = r;
    }
    return a;
}
// 根据连分数求所有收敛分数
vector<Frac> convergents(const vector<i64> &a) {
    __int128_t p0{ 0 }, p1{ 1 }, q0{ 1 }, q1{ 0 };
    vector<Frac> res;
    for (i64 x : a) {
        __int128_t p2{ (__int128_t)x * p1 + p0 };
        __int128_t q2{ (__int128_t)x * q1 + q0 };
        res.push_back({ i64(p2), i64(q2) });
        p0 = p1, p1 = p2;
        q0 = q1, q1 = q2;
    }
    return res;
}
```

接口说明

```
auto cf = continued_fraction(p, q);
auto conv = convergents(cf);
```

例如

$$\frac{355}{113} = [3; 7, 16]$$

其收敛分数为

$$3, \quad \frac{22}{7}, \quad \frac{355}{113}$$

递推

设第 k 个收敛分数为

$$\frac{P_k}{Q_k}$$

初始化

$$P_{-2}=0,\quad P_{-1}=1$$

$$Q_{-2}=1,\quad Q_{-1}=0$$

递推

$$P_k=a_kP_{k-1}+P_{k-2}$$

$$Q_k=a_kQ_{k-1}+Q_{k-2}$$

相邻收敛分数满足

$$P_kQ_{k-1}-P_{k-1}Q_k=\pm1$$

因此天然互质

复杂度

设

$$g=\gcd(|p|,q)$$

连分数展开与收敛分数数量均为

$$O\left(\log\frac{q}{g}\right)$$

分母受限的最佳有理逼近

给定精确有理数

$$x=\frac{p}{q}$$

以及

$$1\le d\le D$$

求使

$$\left|x-\frac{n}{d}\right|$$

最小的分数
这不能只枚举普通收敛分数，边界处还可能需要半收敛分数

下面的误差比较不用浮点数

```
__uint128_t abs128(__int128_t x) {
    return x < 0 ? __uint128_t(-x) : __uint128_t(x);
}
// 比较 a / b 与 c / d，不做交叉乘法，避免 u128 再次溢出
int ratio_cmp(__uint128_t a, __uint128_t b, __uint128_t c, __uint128_t d) {
    bool rev{};
    for (;;) {
        __uint128_t x{ a / b }, y{ c / d };
        if (x != y) {
```

```
            int res{ x < y ? -1 : 1 };
            return rev ? -res : res;
        }
        a %= b, c %= d;
        if (!a || !c) {
            int res{ !a && !c ? 0 : !a ? -1 : 1 };
            return rev ? -res : res;
        }
        swap(a, b), swap(c, d);
        rev = !rev;
    }
}
// 比较 a、b 到 p / q 的绝对误差
int error_cmp(const Frac &a, const Frac &b, i64 p, i64 q) {
    assert(a.q > 0 && b.q > 0 && q > 0);
    auto x = abs128((__int128_t)p * a.q - (__int128_t)a.p * q);
    auto y = abs128((__int128_t)p * b.q - (__int128_t)b.p * q);
    return ratio_cmp(x, a.q, y, b.q);
}
// 分母 ≤ limit 的最近分数，误差相同时优先收敛分数
Frac best_approx_den(i64 p, i64 q, i64 limit) {
    assert(q > 0 && limit > 0);
    i64 g{ gcd(p < 0 ? -p : p, q) };
    p /= g, q /= g;
    if (q ≤ limit) return { p, q };
    i64 n{ p }, d{ q };
    __int128_t p0{ 0 }, p1{ 1 }, q0{ 1 }, q1{ 0 };
    for (;;) {
        auto [a, r] = floor_divmod(n, d);
        __int128_t q2{ q0 + (__int128_t)a * q1 };
        if (q2 > limit) break;
        __int128_t p2{ p0 + (__int128_t)a * p1 };
        p0 = p1, p1 = p2;
        q0 = q1, q1 = q2;
        n = d, d = r;
    }
    __int128_t k{ (limit - q0) / q1 };
    Frac a{ i64(p0 + k * p1), i64(q0 + k * q1) };
    Frac b{ i64(p1), i64(q1) };
    return error_cmp(b, a, p, q) ≤ 0 ? b : a;
}
```

接口说明

```
Frac ans = best_approx_den(p, q, D);
```

返回约分分数

$$\frac{ans.p}{ans.q}$$

满足

$$1 \le ans.q \le D$$

并使

$$\left|\frac{p}{q} - \frac{ans.p}{ans.q}\right|$$

最小

常见用法

逼近 $\pi \approx 355/113$，限制分母不超过 10：

```
auto x = best_approx_den(355, 113, 10);
// 22 / 7
```

限制分母不超过 100：

```
auto x = best_approx_den(355, 113, 100);
// 311 / 99
```

接口支持负数：

```
auto x = best_approx_den(-3, 7, 5);
// -2 / 5
```

为什么要半收敛分数

若当前两个连续收敛分数为

$$\frac{P_{k-2}}{Q_{k-2}},\quad \frac{P_{k-1}}{Q_{k-1}}$$

但下一个完整收敛分数的分母已经超过 D，则还能沿这一段走

$$\frac{P_{k-2}+tP_{k-1}}{Q_{k-2}+tQ_{k-1}}$$

其中最大的合法

$$t=\left\lfloor \frac{D-Q_{k-2}}{Q_{k-1}} \right\rfloor$$

最佳答案只需要比较这个半收敛分数与上一个完整收敛分数

复杂度

$$O(\log q)$$

时间

$$O(1)$$

额外空间

Stern-Brocot 树

Stern-Brocot 树恰好包含所有正既约分数且每个只出现一次
从边界

$$\frac{0}{1},\quad \frac{1}{0}$$

开始，两个相邻分数

$$\frac{a}{b}<\frac{c}{d}$$

的中介数为

$$\frac{a+c}{b+d}$$

若它们满足

$$bc-ad=1$$

则任意严格位于二者之间的既约分数都可以写成

$$\frac{xa+yc}{xb+yd},\quad x,y>0$$

因此中介数的分子、分母分别都是区间中可能出现的最小值

Stern-Brocot 路径

从根

$$\frac{1}{1}$$

向左记 `L`，向右记 `R`
逐步相减会退化为减法版欧几里得算法，因此同方向操作可以一次压缩成一个 run

```
vector<pair<char, i64>> stern_path(i64 p, i64 q) {
    assert(p > 0 && q > 0 && gcd(p, q) == 1);
    vector<pair<char, i64>> path;
    while (p != q) {
        if (p < q) {
            i64 k{ (q - 1) / p };
            path.push_back({ 'L', k });
            q -= k * p;
        } else {
            i64 k{ (p - 1) / q };
            path.push_back({ 'R', k });
            p -= k * q;
        }
    }
    return path;
}
Frac stern_from_path(const vector<pair<char, i64>> &path) {
    __int128_t lp{ 0 }, lq{ 1 }, rp{ 1 }, rq{ 0 };
    for (auto [c, k] : path) {
        assert(k > 0);
        if (c == 'L') rp += (__int128_t)k * lp, rq += (__int128_t)k * lq;
        else lp += (__int128_t)k * rp, lq += (__int128_t)k * rq;
    }
    return { i64(lp + rp), i64(lq + rq) };
}
```

常见用法

```
auto path = stern_path(2, 5);
// L^2 R

auto x = stern_from_path(path);
// 2 / 5
```

例如

$$\frac{2}{5}$$

的路径为

$$L^2R$$

而

$$\frac{5}{2}$$

的路径为

$$R^2L$$

与连分数的关系

Stern-Brocot 的连续 `L / R` 长度，与连分数的商本质上是同一组欧几里得信息
连分数记录

$$a_0, a_1, \ldots, a_k$$

Stern-Brocot 则将这些商解释为交替的左右移动次数
因此：

- 连分数适合直接做逼近和收敛分数
- Stern-Brocot 适合在有理数集合上做单调搜索和边界约束

复杂度

压缩路径的 run 数：

$$O(\log \max(p, q))$$

若真的将 `L^k / R^k` 展开成单字符路径，长度可能达到

$$O(p + q)$$

带分子、分母上界的 Stern-Brocot 二分

给定

$$0 \le p \le P, \quad 1 \le q \le Q$$

以及关于分数值 p/q 的单调谓词 `f`
要求谓词满足从小到大形如：

```
false false false true true true
```

可以同时求：

- 最大的合法 `false` 分数
- 最小的合法 `true` 分数

```
struct SternBounds {
    optional<Frac> lo; // 最大 false
    optional<Frac> hi; // 最小 true
};
template<class F>
SternBounds stern_brocot_bounds(i64 max_p, i64 max_q, F f) {
    assert(max_p >= 0 && max_q >= 1);
    Frac lo{ 0, 1 }, hi{ 1, 0 };
    if (f(0, 1)) return { nullopt, lo };
    auto add = [&](Frac a, Frac b, i64 k) {
        return Frac{
            i64((__int128_t)a.p + (__int128_t)b.p * k),
            i64((__int128_t)a.q + (__int128_t)b.q * k)
        };
    };
    auto max_step = [&](Frac a, Frac b) {
        i64 k{ numeric_limits<i64>::max() };
        if (b.p) k = min(k, (max_p - a.p) / b.p);
        if (b.q) k = min(k, (max_q - a.q) / b.q);
        return k;
    };
    for (;;) {
        if (max_step(lo, hi) < 1) break;
        Frac mid{ lo.p + hi.p, lo.q + hi.q };
        if (f(mid.p, mid.q)) {
            i64 l{ 1 }, r{ max_step(hi, lo) };
            while (l < r) {
                i64 k{ l + (r - l + 1) / 2 };
                Frac x{ add(hi, lo, k) };
                if (f(x.p, x.q)) l = k;
```

```
                else r = k - 1;
            }
            hi = add(hi, lo, l);
        } else {
            i64 l{ 1 }, r{ max_step(lo, hi) };
            while (l < r) {
                i64 k{ l + (r - l + 1) / 2 };
                Frac x{ add(lo, hi, k) };
                if (!f(x.p, x.q)) l = k;
                else r = k - 1;
            }
            lo = add(lo, hi, l);
        }
    }
    return { lo, hi.q ? optional<Frac>{ hi } : nullopt };
}
```

`fracBS` 从 $0/1$ 与 $1/0$ 出发，并一次跳过多个相同方向的 Stern-Brocot 步骤；这里将跳跃长度为受分子、分母上界约束的显式二分，更方便检查边界

接口说明

例如在

$$p \le P, \quad q \le Q$$

中寻找最靠近阈值

$$\frac{a}{b}$$

两侧的分数：

```
auto res = stern_brocot_bounds(P, Q, [&](i64 p, i64 q) {
    return (__int128_t)p * b >= (__int128_t)a * q;
});
```

此时：

```
res.lo // < a / b 的最大合法分数
res.hi // ≥ a / b 的最小合法分数
```

若没有有限的 `true` 分数：

```
!res.hi
```

若连 $0/1$ 都满足谓词：

```
!res.lo
res.hi == 0 / 1
```

为什么终止条件正确

若相邻 Stern-Brocot 边界为

$$\frac{a}{b} < \frac{c}{d}$$

且

$$bc - ad = 1$$

区间中的任意分数都至少需要分子、分母达到中介数

$$\frac{a+c}{b+d}$$

因此若

$$a+c>P$$

或

$$b+d>Q$$

就已经不存在满足两个上界且位于二者之间的分数

复杂度

Stern-Brocot 的方向切换次数为欧几里得级别

$$O(\log(P+Q))$$

每一段对跳跃长度二分，因此谓词调用次数为

$$O(\log^2(P+Q))$$

空间为

$$O(1)$$

同时限制分子与分母的最近逼近

若目标为非负有理数

$$x=\frac{a}{b}$$

同时要求

$$0\le p\le P,\quad 1\le q\le Q$$

先通过 Stern-Brocot 求目标左右紧邻分数，再比较两边误差即可

```
optional<Frac> best_approx_box(i64 a, i64 b, i64 max_p, i64 max_q) {
    assert(b > 0 && max_p >= 0 && max_q >= 1);
    bool neg{ a < 0 };
    if (neg) a = -a;
    auto s = stern_brocot_bounds(max_p, max_q, [&](i64 p, i64 q) {
        return (__int128_t)p * b >= (__int128_t)a * q;
    });
    optional<Frac> ans;
    if (s.lo) ans = s.lo;
    if (s.hi && (!ans || error_cmp(*s.hi, *ans, a, b) < 0)) ans = s.hi;
    if (ans && neg) ans->p = -ans->p;
    return ans;
}
```

常见用法

```
auto x = best_approx_box(a, b, max_num, max_den);
```

适合题目同时限制：

$$|p|\le P$$

以及

$$q\le Q$$

只限制分母时，前面的 `best_approx_den()` 更短且复杂度更优

模意义下的有理数还原

有时已知

$$r \equiv \frac{a}{b} \pmod{m}$$

希望从模值 `r` 恢复一个“小分数”
等价于寻找

$$a \equiv rb \pmod{m}$$

并要求

$$|a| \le A, \quad 0 < b \le B$$

这是扩展欧几里得 / 连分数的另一个直接应用

```
optional<Frac> rational_reconstruction(i64 r, i64 mod, i64 A, i64 B) {
    assert(mod > 0 && A >= 0 && B > 0);
    r %= mod;
    if (r < 0) r += mod;
    __int128_t r0{ mod }, r1{ r };
    __int128_t t0{}, t1{ 1 };
    auto abs_i128 = [](__int128_t x) {
        return x < 0 ? -x : x;
    };
    while (abs_i128(r1) > A) {
        if (!r1) return nullopt;
        __int128_t q{ r0 / r1 };
        __int128_t nr{ r0 - q * r1 };
        __int128_t nt{ t0 - q * t1 };
        r0 = r1, r1 = nr;
        t0 = t1, t1 = nt;
    }
    if (t1 < 0) r1 = -r1, t1 = -t1;
    if (t1 <= 0 || t1 > B || abs_i128(r1) > A) return nullopt;
    i64 p{ i64(r1) }, q{ i64(t1) };
    if (gcd(p < 0 ? -p : p, q) != 1 || gcd(q, mod) != 1) return nullopt;
    if (((__int128_t)r * q - p) % mod) return nullopt;
    return Frac{ p, q };
}
```

接口说明

```
auto x = rational_reconstruction(r, mod, A, B);
```

若成功返回

$$\frac{p}{q}$$

满足

$$|p| \le A, \quad 1 \le q \le B$$

$$\gcd(p, q) = 1, \quad \gcd(q, m) = 1$$

以及

$$p \equiv rq \pmod{m}$$

于是

$$r \equiv pq^{-1} \pmod{m}$$

唯一性

若满足经典唯一性条件

$$2AB < m$$

则这样的既约分数至多只有一个
因此这一条件下函数成功返回时，可以将结果视为唯一重构

常见用法

若

$$r \equiv \frac{3}{7} \pmod{101}$$

因为

$$7^{-1} \equiv 29 \pmod{101}$$

所以

$$r \equiv 87 \pmod{101}$$

调用：

```
auto x = rational_reconstruction(87, 101, 10, 10);
// 3 / 7
```

复杂度

本质为一次欧几里得算法

$$O(\log m)$$

时间

$$O(1)$$

空间

三者之间的关系

这三个工具本质上都来自同一套 Euclidean 结构

工具	保存的信息	更适合解决
连分数	欧几里得商序列	收敛分数、最佳逼近
Stern-Brocot	左右路径 / 邻接分数	有理数单调搜索、双上界
有理重构	欧几里得余数与 Bézout 系数	从模值恢复小分数

连分数中的一个大商，对应 Stern-Brocot 中连续走很多次同一方向
半收敛分数，对应 Stern-Brocot 一条边上只走一部分
模分数还原，则是在欧几里得过程中寻找第一个足够小的余数及其系数

注意事项

`continued_fraction()`、`best_approx_den()` 要求

$$q > 0$$

`stern_path()` 要求输入已经约分且

$$p > 0, \quad q > 0$$

Stern-Brocot 搜索本身只覆盖非负有理数，负数问题通常先取相反数再恢复符号
`stern_brocot_bounds()` 的谓词必须关于分数值单调，不能依赖分子、分母的其他非单调性质
涉及分数比较时不要转成 `double`

```
double(p) / q
```

应优先使用交叉乘法或这里的精确误差比较
`best_approx_den()` 返回的是**绝对误差最小**的分数，并非单纯“最后一个收敛分数”
`rational_reconstruction()` 若不满足

$$2AB < m$$

仍可能返回一个符合约束的候选，但唯一性不再由该条件保证

复杂度速查

接口	时间复杂度	空间
`continued_fraction`	$O(\log q)$	$O(\log q)$
`convergents`	$O(k)$	$O(k)$
`best_approx_den`	$O(\log q)$	$O(1)$
`stern_path`	$O(\log \max(p,q))$ 个压缩段	$O(\log \max(p,q))$
`stern_from_path`	$O(k)$	$O(1)$
`stern_brocot_bounds`	$O(\log^2(P+Q))$ 次谓词调用	$O(1)$
`best_approx_box`	$O(\log^2(P+Q))$	$O(1)$
`rational_reconstruction`	$O(\log m)$	$O(1)$

9. 组合数学与计数

9.1 排列组合与基础计数

杨辉三角（Pascal's Triangle）

```
vector<vector<i64>> pascal(int n, i64 mod) {
    vector c(n + 1, vector<i64>(n + 1));
    c[0][0] = 1 % mod;
    for (int i = 1; i <= n; ++ i) {
        c[i][0] = c[i][i] = 1 % mod;
        for (int j = 1; j < i; ++ j) {
            c[i][j] = (i128(c[i - 1][j - 1]) + c[i - 1][j]) % mod;
        }
    }
    return c;
}
```

- `pascal(n, mod)`：返回所有 $0 \le k \le i \le n$ 的 $\binom{i}{k} \bmod mod$；要求 `n >= 0`、`mod > 0`，$O(n^2)$

精确组合数

```
i128 comb_i128(i64 n, i64 k) {
    if (n < 0 || k < 0 || k > n) return 0;
    k = min(k, n - k);
    i128 ans{ 1 };
    for (i64 i = 1; i <= k; ++ i) {
        i64 a{ n - k + i }, b{ i };
        i64 g{ gcd(a, b) };
        a /= g;
        b /= g;
        g = gcd(i64(ans % b), b);
        ans /= g;
        b /= g;
        ans *= a;
    }
```

```
    return ans;
}
template<class Big>
Big comb_big(i64 n, i64 k) {
    if (n < 0 || k < 0 || k > n) return Big(0);
    k = min(k, n - k);
    Big ans{ 1 };
    for (i64 i = 1; i <= k; ++ i) {
        ans *= n - k + i;
        ans /= i;
    }
    return ans;
}
```

9.1.1 Lucas 与任意模组合数

固定质数模直接使用已有 `Comb::Lucas()`。本节只补充运行时质数 Lucas与任意合数模组合数

```
// ===== 运行时质数 Lucas: p 很大但每个数位较小的一侧不大时使用 =====
i64 comb_p(i64 n, i64 k, i64 p) {
    if (n < 0 || k < 0 || k > n || n >= p) return 0;
    k = min(k, n - k);
    i64 a{ 1 }, b{ 1 };
    for (i64 i = 1; i <= k; ++ i) {
        a = i64(i128(a) * (n - k + i) % p);
        b = i64(i128(b) * i % p);
    }
    return i64(i128(a) * qpow_mod(b, p - 2, p) % p);
}
i64 lucas(i64 n, i64 k, i64 p) {
    if (n < 0 || k < 0 || k > n) return 0;
    i64 ans{ 1 };
    while (k) {
        i64 a{ n % p }, b{ k % p };
        if (b > a) return 0;
        ans = i64(i128(ans) * comb_p(a, b, p) % p);
        n /= p;
        k /= p;
    }
    return ans;
}
// ===== 任意合数模: 素数幂组合数 + CRT =====
struct CombMod {
    struct PP {
        int p, e, m;
        vector<i64> pre;
        PP(int p_, int e_, int m_) : p(p_), e(e_), m(m_), pre(m + 1, 1) {
            for (int i = 1; i <= m; ++ i) {
                pre[i] = pre[i - 1];
                if (i % p) pre[i] = i64(i128(pre[i]) * i % m);
            }
        }
        i64 vp(i64 n) const {
            i64 res{};
            while (n) n /= p, res += n;
            return res;
        }
        i64 fac0(i64 n) const {
            i64 res{ 1 };
            while (n) {
                if ((n / m) & 1) res = i64(i128(res) * pre[m] % m);
                res = i64(i128(res) * pre[n % m] % m);
                n /= p;
            }
            return res;
        }
        i64 C(i64 n, i64 k) const {
            if (n < 0 || k < 0 || k > n) return 0;
            i64 c{ vp(n) - vp(k) - vp(n - k) };
```

```
            if (c >= e) return 0;
            i64 a{ fac0(n) }, b{ fac0(k) }, d{ fac0(n - k) };
            i64 iv{ *inverse_mod(i64(i128(b) * d % m), m) };
            return i64(i128(a) * iv % m * qpow_mod(p, c, m) % m);
        }
    };
    int m;
    vector<PP> a;
    explicit CombMod(int m_) : m(m_) {
        int x{ m };
        for (int p = 2; i64(p) * p <= x; ++ p) if (x % p == 0) {
            int e{}, q{ 1 };
            while (x % p == 0) {
                x /= p;
                ++ e;
                q *= p;
            }
            a.emplace_back(p, e, q);
        }
        if (x > 1) a.emplace_back(x, 1, x);
    }
    i64 C(i64 n, i64 k) const {
        if (n < 0 || k < 0 || k > n || m == 1) return 0;
        i64 ans{}, mod{ 1 };
        for (const auto &f : a) {
            i64 r{ f.C(n, k) }, d{ (r - ans) % f.m };
            if (d < 0) d += f.m;
            i64 t{ i64(i128(d) * *inverse_mod(mod % f.m, f.m) % f.m) };
            ans += mod * t;
            mod *= f.m;
            ans %= mod;
        }
        return ans;
    }
};
```

选择

条件	使用
固定质数模，$n < mod$	`comb.C(n,k)`
固定质数模，$n \geq mod$	`comb.Lucas(n,k)`
运行时质数 `p`，且 Lucas 每位较小的一侧不大	`lucas(n,k,p)`
任意合数模，各素数幂因子可以预处理	`CombMod(mod).C(n,k)`

接口

- `comb_p(n,k,p)`：计算 $0 \leq n < p$ 时的 $\binom{n}{k} \bmod p$；要求 `p` 为质数
- `lucas(n,k,p)`：运行时质数 Lucas，不做 $O(p)$ 预处理
- `CombMod(m)`：分解合数模并预处理各素数幂；要求 `m > 0`
- `CombMod::C(n,k)`：计算 $\binom{n}{k} \bmod m$

  `CombMod` 内部的 `PP` 仅用于模 p^e 计算，通常不需要单独调用

依赖已有：

```
qpow_mod(a, b, mod);
inverse_mod(a, mod);
```

以及 `i128`

注意事项

- `comb.Lucas()` 与 `lucas()` 都要求模数为质数
- `comb.Lucas()` 内部调用 `Comb::C()`；若模数很大且 Lucas 数位很大，自动扩容可能不可接受，此时使用无预处理的 `lucas()`
- `lucas()` 每个数位需要 $O(\min(k_i, n_i - k_i))$ 次乘法和一次快速幂，只适合较小的一侧不大
- `CombMod` 不能直接对 `n! / k!(n-k)!` 求逆；需要先拆出各素因子的指数，再对剩余部分求逆
- `CombMod` 需要为每个素数幂 p^e 开 $O(p^e)$ 空间，因此模数含很大的素数幂时不适用
- `CombMod` 使用试除分解 `m`；这里 `m` 为 `int` 时已经足够，没必要为此依赖 Pollard--Rho

常见用法

固定质数模

```
Comb<> comb;
Z ans = comb.Lucas(n, k);
```

运行时质数

```
i64 ans = lucas(n, k, p);
```

任意合数模

```
CombMod comb(mod);
i64 ans = comb.C(n, k);
```

复杂度

操作	时间复杂度	空间复杂度
`lucas(n,k,p)`	$O\left(\sum_i \min(k_i, n_i - k_i) + \log_p n \log p\right)$	$O(1)$
`CombMod` 构造	$O(\sqrt{m} + \sum p_i^{e_i})$	$O(\sum p_i^{e_i})$
`CombMod::C`	$O\left(\sum_i \log_{p_i} n + \log m\right)$	$O(1)$ 额外空间

9.1.2 排列与组合排名

排列使用康托展开；组合排名为较低频扩展，可整段删除。依赖已有 `BIT<int>`，组合部分额外依赖 `comb_i128()`

```
struct Cantor {
    static vector<i128> fac(int n) {
        vector<i128> f(n + 1, 1);
        for (int i = 1; i <= n; ++ i) f[i] = f[i - 1] * i;
        return f;
    }
    static i128 rank(const vector<int> &p) {
        int n{ int(p.size()) };
        auto f{ fac(n) };
        BIT<int> fw(vector<int>(n, 1));
        i128 ans{};
        for (int i = 0; i < n; ++ i) {
            ans += i128(fw.sum(p[i])) * f[n - i - 1];
            fw.add(p[i], -1);
        }
        return ans;
    }
    // ===== 以下函数可直接删除：不需要逆排名时 =====
    static vector<int> unrank(int n, i128 r) {
        auto f{ fac(n) };
        BIT<int> fw(vector<int>(n, 1));
        vector<int> p(n);
        for (int i = 0; i < n; ++ i) {
            i128 d{ f[n - i - 1] };
            int k{ int(r / d) };
            r %= d;
            p[i] = fw.kth(k + 1);
            fw.add(p[i], -1);
        }
        return p;
    }
};
// ===== 以下代码可直接删除：组合字典序排名 / 逆排名 =====
i128 comb_rank(int n, const vector<int> &a) {
    int k{ int(a.size()) }, pre{ -1 };
    i128 ans{};
    for (int i = 0; i < k; ++ i) {
        ans += comb_i128(n - pre - 1, k - i) - comb_i128(n - a[i], k - i);
```

```
        pre = a[i];
    }
    return ans;
}
vector<int> comb_unrank(int n, int k, i128 r) {
    vector<int> a(k);
    int pre{ -1 };
    for (int i = 0; i < k; ++ i) {
        int rem{ k - i };
        i128 tot{ comb_i128(n - pre - 1, rem) };
        int l{ pre + 1 }, rr{ n - rem };
        while (l < rr) {
            int mid{ (l + rr) / 2 };
            i128 cnt{ tot - comb_i128(n - mid - 1, rem) };
            if (cnt > r) rr = mid;
            else l = mid + 1;
        }
        r -= tot - comb_i128(n - l, rem);
        a[i] = pre = l;
    }
    return a;
}
```

接口说明

康托展开

- `Cantor::rank(p)`：返回排列 `p` 的 `0base` 字典序排名，$O(n \log n)$
- `Cantor::unrank(n,r)`：恢复 `0base` 排名 `r` 对应的排列，$O(n \log n)$
- `p` 必须恰为 `0..n-1` 的排列
- 使用 `i128` 保存阶乘和排名，因此要求 $n! \le 2^{127}-1$，即 `n ≤ 33`
- `unrank()` 要求 $0 \le r < n!$

组合排名

- `comb_rank(n,a)`：严格递增组合 `a` 转为 `0base` 字典序排名
- `comb_unrank(n,k,r)`：恢复排名 `r` 对应的严格递增 `k` 元组合
- `a` 要求满足 $0 \le a_0 < a_1 < \cdots < a_{k-1} < n$
- `comb_unrank()` 要求 $0 \le r < \binom{n}{k}$
- 所有涉及的精确组合数都必须能放入 `i128`

注意事项

- `BIT::sum(r)` 统计 `[0,r)`，因此 `fw.sum(p[i])` 正好表示剩余元素中小于 `p[i]` 的数量
- `BIT::kth(k)` 的 `k` 从 `1` 开始，所以逆康托使用 `fw.kth(k + 1)`
- 这里直接利用 `BIT(vector<T>)` 的 $O(n)$ 建树，不需要逐个 `add(i,1)`
- 康托代码默认输入合法，不额外检查重复元素和越界
- 组合排名是字典序排名，与排列的康托展开不是同一种编码
- 组合排名必须使用精确的 `comb_i128()`，不能使用取模后的 `comb.C()`
- `comb_unrank()` 二分确定每一位，适合 n 较大而 k 较小的情况

常见用法

排列排名 / 逆排名

```
i128 r = Cantor::rank(p);
vector<int> q = Cantor::unrank(p.size(), r);
```

组合排名 / 逆排名

```
i128 r = comb_rank(n, a);
vector<int> b = comb_unrank(n, a.size(), r);
```

例如 `n=5,k=3` 时：

```
0 1 2
0 1 3
0 1 4
0 2 3
0 2 4
```

```
0 3 4
1 2 3
...
```

复杂度

操作	时间复杂度	空间复杂度
`Cantor::rank`	$O(n \log n)$	$O(n)$
`Cantor::unrank`	$O(n \log n)$	$O(n)$
`comb_rank`	$O(k)$ 次 `comb_i128()`	$O(1)$
`comb_unrank`	$O(k \log n)$ 次 `comb_i128()`	$O(k)$

9.2 经典数列与分拆

```cpp
// ===== 第一类 Stirling: n 元排列恰有 k 个置换环 =====
template<class T = Z>
vector<vector<T>> stir1(int n) {
    vector s(n + 1, vector<T>(n + 1));
    s[0][0] = 1;
    for (int i = 1; i <= n; ++ i) {
        for (int k = 1; k <= i; ++ k) {
            s[i][k] = s[i - 1][k - 1] + T(i - 1) * s[i - 1][k];
        }
    }
    return s;
}
// ===== 第二类 Stirling: n 个不同元素分成 k 个非空无标号集合 =====
template<class T = Z>
vector<vector<T>> stir2(int n) {
    vector s(n + 1, vector<T>(n + 1));
    s[0][0] = 1;
    for (int i = 1; i <= n; ++ i) {
        for (int k = 1; k <= i; ++ k) {
            s[i][k] = s[i - 1][k - 1] + T(k) * s[i - 1][k];
        }
    }
    return s;
}
// ===== Eulerian: n 元排列恰有 k 个下降位置 =====
template<class T = Z>
vector<vector<T>> euler(int n) {
    vector a(n + 1, vector<T>(n + 1));
    a[0][0] = 1;
    for (int i = 1; i <= n; ++ i) {
        for (int k = 0; k < i; ++ k) {
            a[i][k] = T(k + 1) * a[i - 1][k];
            if (k) a[i][k] += T(i - k) * a[i - 1][k - 1];
        }
    }
    return a;
}
// ===== Bell: n 个不同元素划分成任意个非空无标号集合 =====
// Bell 三角，O(n) 空间；只需要 Bell 时比先求 stir2 更省空间。
template<class T = Z>
vector<T> bell(int n) {
    vector<T> b(n + 1), a(n + 1);
    b[0] = a[0] = 1;
    for (int i = 1; i <= n; ++ i) {
        T pre{ a[0] };
        a[0] = a[i - 1];
        for (int k = 1; k <= i; ++ k) {
            T cur{ a[k] };
            a[k] = a[k - 1] + pre;
            pre = cur;
        }
        b[i] = a[0];
```

```
    }
    return b;
}
// ===== p(n,k): n 无序分拆成恰好 k 个正整数 =====
template<class T = Z>
vector<vector<T>> part_k(int n) {
    vector p(n + 1, vector<T>(n + 1));
    p[0][0] = 1;
    for (int s = 1; s <= n; ++ s) {
        for (int k = 1; k <= s; ++ k) {
            p[s][k] = p[s - 1][k - 1] + p[s - k][k];
        }
    }
    return p;
}
// ===== p(n): Euler 五边形数递推 =====
template<class T = Z>
vector<T> part(int n) {
    vector<T> p(n + 1);
    p[0] = 1;
    for (int i = 1; i <= n; ++ i) {
        for (i64 k = 1; ; ++ k) {
            i64 a{ k * (3 * k - 1) / 2 };
            if (a > i) break;
            if (k & 1) p[i] += p[i - a];
            else p[i] -= p[i - a];
            i64 b{ k * (3 * k + 1) / 2 };
            if (b <= i) {
                if (k & 1) p[i] += p[i - b];
                else p[i] -= p[i - b];
            }
        }
    }
    return p;
}
```

接口说明

- `stir1<T>(n)`：返回无符号第一类 Stirling 数 $[i\ k]$，$0 \le i,k \le n$
- `stir2<T>(n)`：返回第二类 Stirling 数 $\{i\ k\}$
- `euler<T>(n)`：返回 Eulerian 数，`a[i][k]` 表示 i 元排列恰有 k 个下降位置
- `bell<T>(n)`：返回 $B_0,\dots,B_n$
- `part_k<T>(n)`：返回 $p(i,k)$，即 i 无序分拆成恰好 k 个正整数的方案数
- `part<T>(n)`：返回整数分拆数 $p(0),\dots,p(n)$

注意事项

- `stir1()` 返回的是**无符号**第一类 Stirling 数；有符号版本为 $(-1)^{n-k}[n\ k]$
- `stir2[n][k]\cdot k!` 是从 n 元集合到 k 元集合的满射数
- `euler[n][k]` 中的 `k` 表示下降位置数量，范围为 $0 \le k < n$
- `part_k()` 的分拆不计顺序；正整数的**有序拆分**是 composition，不是整数分拆
- `part()` 中五边形数的符号按 `+,+,-,-,+,+,...` 循环
- 显式公式出现除法时要确认分母可逆；上述递推代码只有加减乘，没有这一限制
- 若只求很大的某一个下标，应使用对应的快速算法或生成函数方法，不应机械预处理到该下标

常见用法

Catalan 数

Catalan 数常见于：

- n 对合法括号
- Dyck 路径
- n 个结点的有根有序二叉树
- 凸 $(n+2)$ 边形三角剖分
- 不越过对角线的 (n,n) 单调路径

$$\mathrm{Cat}_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

已有 `Comb` 时无需单独板子：

```
Z cat = comb.C(2 * n, n) - comb.C(2 * n, n + 1);
```

后一个形式不需要显式除以 $n+1$
若普通 `Comb::C()` 的范围不满足要求，换用 Lucas 或任意模组合数接口
递推为

$$\mathrm{Cat}_0 = 1, \qquad \mathrm{Cat}_n = \sum_{i=0}^{n-1} \mathrm{Cat}_i \, \mathrm{Cat}_{n-1-i}$$

相邻项满足

$$\mathrm{Cat}_n = \mathrm{Cat}_{n-1} \frac{4n-2}{n+1}$$

Motzkin 数

Motzkin 路径每步为 $(1,1)$、$(1,0)$ 或 $(1,-1)$，从 $(0,0)$ 到 $(n,0)$ 且不低于横轴

$$M_0 = 1, \qquad M_n = M_{n-1} + \sum_{i=0}^{n-2} M_i M_{n-2-i}$$

也有

$$M_n = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{2k} \mathrm{Cat}_k$$

递推很短且使用频率较低，不单独保留代码

Stirling、Bell 与 Eulerian

数列	计数对象	递推
第一类 Stirling $[n\ k]$	n 元排列恰有 k 个置换环	$[n\ k] = [n-1\ k-1] + (n-1)[n-1\ k]$
第二类 Stirling $\{n\ k\}$	n 个不同元素划分成 k 个非空无标号集合	$\{n\ k\} = \{n-1\ k-1\} + k\{n-1\ k\}$
Bell B_n	划分成任意数量的非空无标号集合	$B_n = \sum_{k=0}^{n} \{n\ k\}$
Eulerian $\langle n\ k \rangle$	n 元排列恰有 k 个下降位置	$\langle n\ k \rangle = (k+1)\langle n-1\ k \rangle + (n-k)\langle n-1\ k-1 \rangle$

第二类 Stirling 的显式式：

$$\left\{ {n \atop k} \right\} = \frac{1}{k!} \sum_{i=0}^{k} (-1)^i \binom{k}{i} (k-i)^n$$

Eulerian 数显式式：

$$\left\langle {n \atop k} \right\rangle = \sum_{j=0}^{k+1} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

$$\left\langle {n \atop k} \right\rangle = \left\langle {n \atop n-1-k} \right\rangle$$

$$\sum_{k=0}^{n-1} \left\langle {n \atop k} \right\rangle = n!$$

Worpitzky 恒等式：

$$x^n = \sum_{k=0}^{n-1} \left\langle {n \atop k} \right\rangle \binom{x+k}{n}$$

它可以理解成另一种“普通幂 → 组合数基”的换基关系

如果已经计算 `stir2`，Bell 数可直接求：

```
auto s = stir2(n);
Z B{};
for (int k = 0; k ≤ n; ++ k) B += s[n][k];
```

只需要所有 Bell 数时使用 `bell(n)`，可把空间降到 $O(n)$

错排

错排数 D_n：

$$D_0 = 1, \quad D_1 = 0$$

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

亦有

$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$$

递推只有一行，不单独保留代码

对合排列

所有置换环长度只能为 1 或 2：

$$I_0 = I_1 = 1$$

$$I_n = I_{n-1} + (n-1) I_{n-2}$$

也可写为

$$I_n = \sum_{k=0}^{\lfloor n/2 \rfloor} \frac{n!}{2^k k! (n-2k)!}$$

同样不单独维护代码

指定置换环结构

若长度为 i 的置换环恰有 m_i 个，且

$$\sum_{i \geq 1} i m_i = n$$

则排列数为

$$\frac{n!}{\prod_{i \geq 1} i^{m_i} m_i!}$$

整数分拆

$p(n,k)$ 表示把 n 无序分拆成恰好 k 个正整数：

$$p(n,k) = p(n-1,k-1) + p(n-k,k)$$

且

$$p(n) = \sum_{k=0}^{n} p(n,k)$$

只需要 $p(n)$ 时，五边形数递推更快。令

$$g_k^- = \frac{k(3k-1)}{2}, \qquad g_k^+ = \frac{k(3k+1)}{2}$$

则

$$p(0) = 1$$

$$p(n) = \sum_{k \geq 1} (-1)^{k-1} \left(p(n - g_k^-) + p(n - g_k^+) \right)$$

负下标项视为 0
整数分拆生成函数：

$$\sum_{n\geq 0} p(n)x^n = \prod_{i\geq 1} \frac{1}{1-x^i}$$

分拆成互不相同的正整数，与分拆成奇数的方案数相同：

$$\prod_{i\geq 1} (1+x^i) = \prod_{i\geq 1} \frac{1}{1-x^{2i-1}}$$

复杂度

接口	时间复杂度	空间复杂度
stir1(n)	$O(n^2)$	$O(n^2)$
stir2(n)	$O(n^2)$	$O(n^2)$
euler(n)	$O(n^2)$	$O(n^2)$
bell(n)	$O(n^2)$	$O(n)$
part_k(n)	$O(n^2)$	$O(n^2)$
part(n)	$O(n\sqrt{n})$	$O(n)$

9.2.1 Stirling 变换与快速算法

解决问题： 在普通幂、下降幂、上升幂之间换基，并在 n 较大时快速求固定一行 Stirling 数。

第二类 Stirling 单行

固定 n，一次求

$$\left\{ {n \atop 0} \right\}, \left\{ {n \atop 1} \right\}, \ldots, \left\{ {n \atop n} \right\}$$

可转为一次卷积

```
vector<Z> stir2_row(int n) {
    Comb<> comb(n);
    vector<Z> a(n + 1), b(n + 1);
    for (int i = 0; i <= n; ++ i) {
        a[i] = power(Z(i), n) * comb.IFact(i);
        b[i] = (i & 1 ? -Z(1) : Z(1)) * comb.IFact(i);
    }
    auto c = convolution(a, b);
    c.resize(n + 1);
    return c;
}
```

依赖已有：

```
Comb<>
power()
convolution()
```

第一类 Stirling 单行无需另外维护代码：

$$\prod_{i=0}^{n-1} (x+i) = \sum_{k=0}^{n} \left[{n \atop k} \right] x^k$$

因此直接使用多项式分治乘积树计算

```
x · (x+1) · ... · (x+n-1)
```

即可得到整行无符号第一类 Stirling 数

Stirling 变换

定义下降幂与上升幂：

$$x^{\underline{n}} = x(x-1)\cdots(x-n+1)$$

$$x^{\overline{n}} = x(x+1)\cdots(x+n-1)$$

令有符号第一类 Stirling 数

$$s(n,k) = (-1)^{n-k}\begin{bmatrix} n \\ k \end{bmatrix}$$

则

$$x^{\underline{n}} = \sum_{k=0}^{n} s(n,k)x^k$$

$$x^n = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} x^{\underline{k}}$$

因此两类 Stirling 数给出的变换互逆：

$$f_n = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} g_k \iff g_n = \sum_{k=0}^{n} s(n,k) f_k$$

无符号第一类 Stirling 数同时满足

$$x^{\overline{n}} = \sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

常用特值

$$\begin{Bmatrix} n \\ 2 \end{Bmatrix} = 2^{n-1} - 1$$

$$\begin{bmatrix} n \\ 1 \end{bmatrix} = (n-1)!$$

$$\sum_{k} \begin{bmatrix} n \\ k \end{bmatrix} = n!$$

Lah 数

Lah 数连接上升幂与下降幂

抽象模型

$L(n,k)$ 表示：

> 将 n 个不同元素划分成 k 个非空线性序列；每个序列内部有序，但 k 个序列之间无序

例如与第二类 Stirling 的区别是：

```
Stirling II: 每一组内部无序
Lah:         每一组内部有序
```

公式

$$L(n,k) = \binom{n-1}{k-1}\frac{n!}{k!}, \qquad 1 \le k \le n$$

边界：

$$L(0,0) = 1$$

递推：

$$L(n,k) = L(n-1,k-1) + (n+k-1)L(n-1,k)$$

由于已有闭式，通常不需要维护 $O(n^2)$ 的 Lah DP

换基

$$x^{\overline{n}} = \sum_{k=0}^{n} L(n,k) x^{\underline{k}}$$

$$x^{\underline{n}} = \sum_{k=0}^{n} (-1)^{n-k} L(n,k) x^{\overline{k}}$$

同一行 Stirling 的快速计算

第二类

由显式式

$$\left\{ {n \atop k} \right\} = \frac{1}{k!} \sum_{i=0}^{k} (-1)^{k-i} \binom{k}{i} i^n$$

得到

$$\left\{ {n \atop k} \right\} = \sum_{i=0}^{k} \frac{i^n}{i!} \frac{(-1)^{k-i}}{(k-i)!}$$

令

$$a_i = \frac{i^n}{i!}, \quad b_i = \frac{(-1)^i}{i!}$$

则

$$\left\{ {n \atop k} \right\} = (a * b)_k$$

所以一整行可通过一次卷积求出

第一类

因为

$$\prod_{i=0}^{n-1} (x+i) = \sum_{k=0}^{n} \left[{n \atop k} \right] x^k$$

用分治计算所有一次多项式

$$x,\ x+1, \ldots, x+n-1$$

的乘积即可

接口说明

- `stir2_row(n)`：返回第二类 Stirling 第 `n` 行，`res[k] = {n \brace k}`
- 普通 $O(n^2)$ Stirling 表仍使用 经典数列与分拆 中的 `stir1(n)` / `stir2(n)`
- 第一类整行快速算法直接复用已有多项式分治乘积与卷积，不维护重复实现

注意事项

- `stir1()` 通常保存无符号第一类 Stirling 数；换基时使用的是有符号版本

$$s(n,k) = (-1)^{n-k} [n\ k]$$

- 第二类 Stirling 单行卷积要求阶乘逆元存在，因此通常要求 `n < mod`
- 只需要较小的二维表时，$O(n^2)$ DP 比 NTT 更简单，常数也更小
- Lah 数不是通常意义下必须掌握的“第三类 Stirling 数”；建议直接使用名称 Lah 数

复杂度

方法	时间复杂度	空间复杂度
普通 `stir1/stir2` 表	$O(n^2)$	$O(n^2)$

方法	时间复杂度	空间复杂度
`stir2_row(n)`	$O(M(n))$	$O(n)$
第一类 Stirling 单行乘积树	$O(M(n)\log n)$	$O(n\log n)$ 或依实现

9.3 容斥与反演

解决问题： 已知若干条件同时成立时的计数，反推出“不满足任何条件”“恰满足若干条件”等答案；或在集合包含关系上的前缀和与原函数之间相互转换

子集容斥

设 `f(s)` 表示编号集合 `s` 中的所有条件**同时成立**时的方案数，则没有任何坏条件成立的方案数为

```
template<class T, class F>
T ie(int n, F f) {
    T ans{};
    for (int s = 0; s < (1 << n); ++ s) {
        if (__builtin_parity((unsigned)s)) ans -= f(s);
        else ans += f(s);
    }
    return ans;
}
```

子集 Zeta / Möbius

```
// f[S] ← sum_{T subseteq S} f[T]
template<class T>
void zeta_sub(vector<T> &f) {
    int n{ __lg(f.size()) };
    for (int i = 0; i < n; ++ i)
        for (int s = 0; s < (1 << n); ++ s)
            if (s >> i & 1) f[s] += f[s ^ (1 << i)];
}
// zeta_sub 的逆变换
template<class T>
void mobius_sub(vector<T> &f) {
    int n{ __lg(f.size()) };
    for (int i = 0; i < n; ++ i)
        for (int s = 0; s < (1 << n); ++ s)
            if (s >> i & 1) f[s] -= f[s ^ (1 << i)];
}
// f[S] ← sum_{T supseteq S} f[T]
template<class T>
void zeta_sup(vector<T> &f) {
    int n{ __lg(f.size()) };
    for (int i = 0; i < n; ++ i)
        for (int s = 0; s < (1 << n); ++ s)
            if (!(s >> i & 1)) f[s] += f[s | (1 << i)];
}
// zeta_sup 的逆变换
template<class T>
void mobius_sup(vector<T> &f) {
    int n{ __lg(f.size()) };
    for (int i = 0; i < n; ++ i)
        for (int s = 0; s < (1 << n); ++ s)
            if (!(s >> i & 1)) f[s] -= f[s | (1 << i)];
}
```

容斥原理

设全集为 U，坏事件为 $A_1,\dots,A_n$

至少一个条件成立

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\varnothing\neq S\subseteq[n]} (-1)^{|S|+1} \left|\bigcap_{i\in S} A_i\right|$$

没有任何坏条件成立

$$\left|U \setminus \bigcup_{i=1}^{n} A_i\right| = \sum_{S \subseteq [n]} (-1)^{|S|} \left|\bigcap_{i \in S} A_i\right|$$

这正是 `ie(n,f)` 的含义，其中

```
f(S) = S 中所有坏条件同时成立的方案数
```

因此容斥的核心不是背符号，而是：

> **先强制若干条件同时成立，使计数变简单；再用正负号消除重复计数**

如何识别

典型信号：

- 所有条件都满足很难算，但指定一批条件违反后容易算
- 每个对象不能具有某些性质
- 禁止位置、禁止颜色、禁止质因子
- 至少 / 至多 / 恰好满足若干性质
- 答案中出现交集计数，而各交集结构相似

按命中个数容斥

这是普通容斥非常重要的推广
设一个方案可能满足 n 个性质中的若干个
定义

$$S_j = \sum_{\substack{T \subseteq [n] \\ |T| = j}} \#\{\text{同时满足 } T \text{ 中全部性质的方案}\}$$

注意：一个恰好满足 r 个性质的方案，会在 S_j 中被计算

$$\binom{r}{j}$$

次

恰好满足 k 个性质

记答案为 E_k，则

$$\boxed{E_k = \sum_{j=k}^{n} (-1)^{j-k} \binom{j}{k} S_j}$$

这可以理解为一次二项式反演
特别地，

$$E_0 = \sum_{j=0}^{n} (-1)^j S_j$$

就是普通容斥

至少满足 k 个性质

记答案为 A_k：

$$\boxed{A_k = \sum_{j=k}^{n} (-1)^{j-k} \binom{j-1}{k-1} S_j}$$

特别地 $k = 1$：

$$A_1 = \sum_{j=1}^{n} (-1)^{j-1} S_j$$

就是并集大小

如何识别

如果题目不是问：

> 一个条件都不能违反

而是问：

> 恰好有 k 个条件成立 / 违反

不要硬套普通容斥
应先计算每个阶数的交集总和 S_j，再做上述反演

对称容斥

如果任取 j 个条件同时成立时，方案数只与 j 有关，记为 F_j，那么

$$S_j = \binom{n}{j} F_j$$

此时指数级枚举子集可以直接压成：

$$E_0 = \sum_{j=0}^{n} (-1)^j \binom{n}{j} F_j$$

也就是从

$$O(2^n)$$

降到

$$O(n)$$

典型模型：满射

n 个不同元素映射到 m 个不同盒子，要求每个盒子非空
令坏事件 A_i 为“第 i 个盒子为空”
任取 j 个盒子强制为空后：

$$F_j = (m-j)^n$$

所以满射数为

$$\sum_{j=0}^{m} (-1)^j \binom{m}{j} (m-j)^n = m! \left\{ {n \atop m} \right\}$$

看到：

> 条件完全对称，交集计数只依赖选择了几个条件

时，应优先尝试把 2^n 容斥按集合大小合并

二项式反演

若

$$f_n = \sum_{k=0}^{n} \binom{n}{k} g_k$$

则

$$\boxed{g_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} f_k}$$

反之亦然
另一种常见写法：

$$f_n = \sum_{k=0}^{n} (-1)^k \binom{n}{k} g_k$$

则

$$g_n = \sum_{k=0}^{n} (-1)^k \binom{n}{k} f_k$$

这种带 $(-1)^k$ 的变换是自己的逆变换

抽象模型

二项式反演最常出现于：

> `f[n]` 统计“从 n 个可选条件中任选若干个”的总贡献，需要恢复“恰好使用全部 n 个条件”的贡献

典型关系：

```
容易求：至少包含某些东西 / 可以缺失若干东西
要求：  恰好包含全部 / 恰好命中 k 个
```

本质上就是把集合容斥按集合大小聚合后的结果
因此不要把“二项式反演”和“容斥”当成两个毫无关系的技巧

子集 Zeta 变换

若有一个集合函数 $f(S)$，定义

$$F(S) = \sum_{T \subseteq S} f(T)$$

称为子集 Zeta 变换
直接对每个 S 枚举子集需要约 $O(3^n)$，而 `zeta_sub()` 可以在

$$O(n2^n)$$

时间内求出所有 $F(S)$

如何理解

代码：

```
if (s >> i & 1)
    f[s] += f[s ^ (1 << i)];
```

表示依次决定：

> 第 i 位是否允许从当前集合中删掉

全部位处理完后，所有子集的贡献都汇总到了 `f[s]`

逆变换

若已知

$$F(S) = \sum_{T \subseteq S} f(T)$$

则

$$\boxed{f(S) = \sum_{T \subseteq S} (-1)^{|S|-|T|} F(T)}$$

这正是 Boolean 格上的 Möbius 反演
实现就是把 Zeta 中的 `+=` 改为 `-=`：

```
mobius_sub(f);
```

超集 Zeta / Möbius

另一种常见方向：

$$F(S) = \sum_{T \supseteq S} f(T)$$

使用：

```
zeta_sup(f);
```

恢复原函数：

```
mobius_sup(f);
```

怎么区分方向

如果要求的是：

```
所有 T ⊆ S 的贡献
```

用 `sub`
如果要求的是：

```
所有 T ⊇ S 的贡献
```

用 `sup`
赛场最容易犯的错误就是把方向写反

一般 Möbius 反演

Zeta / Möbius 并不只存在于子集
在偏序集合中，如果

$$F(x) = \sum_{y \le x} f(y)$$

则可以通过 Möbius 函数反演：

$$f(x) = \sum_{y \le x} \mu(y, x) F(y)$$

几个重要实例：

偏序	Zeta 求和	Möbius				
子集包含	$\sum_{T \subseteq S}$	$(-1)^{	S	-	T	}$
整除关系	$\sum_{d \mid n}$	数论 Möbius $\mu(n/d)$				
按集合大小聚合	$\sum \binom{n}{k}$	二项式反演				

因此：

```
普通容斥
   ↓
子集 Möbius
   ↓ 按集合大小聚合
二项式反演
```

本质上属于同一套反演思想
整除关系的 Möbius 反演放在数论模块，本节不重复代码

最大最小容斥

对数值 $x_1, \ldots, x_n$：

$$\max_i x_i = \sum_{\varnothing \neq S \subseteq [n]} (-1)^{|S|+1} \min_{i \in S} x_i$$

对偶地：

$$\min_i x_i = \sum_{\varnothing \neq S \subseteq [n]} (-1)^{|S|+1} \max_{i \in S} x_i$$

这个形式常用于：

- 期望最大值 / 最小值

- 所有子集的 `min/max` 容斥
- 把难处理的最大值转成容易统计的最小值

接口说明

- `ie<T>(n,f)`：计算没有任何坏条件成立的方案数；`f(s)` 返回 `s` 中所有坏条件同时成立的方案数
- `zeta_sub(f)`：`f[S] ← Σ_{T⊆S} f[T]`
- `mobius_sub(f)`：`zeta_sub()` 的逆变换
- `zeta_sup(f)`：`f[S] ← Σ_{T⊇S} f[T]`
- `mobius_sup(f)`：`zeta_sup()` 的逆变换

四个子集变换均要求：

```
f.size() = 2^n
```

并且 `T` 支持加减

注意事项

- 普通容斥的符号取决于“当前算的是合法方案还是并集”，不要只背 `+ - + -`
- `ie()` 中 `s = 0` 表示没有强制任何坏条件，其贡献就是全集大小
- 条件具有完全对称性时，先按 `|S|` 合并，不要直接枚举 2^n
- “恰好 k 个条件成立”应使用按命中数反演，不是普通容斥
- 二项式反演中的 $(-1)^{n-k}$ 与 $(-1)^k$ 两种写法不要混用
- `zeta_sub` / `mobius_sub` 与 `zeta_sup` / `mobius_sup` 方向相反
- 子集 Möbius 与 FWT 的 OR/AND 变换形式相近，但用途不同；OR/AND 卷积仍放在 FWT 模块
- 整除偏序上的 Möbius 反演属于数论，本节只说明统一抽象

常见用法

普通容斥

```
Z ans = ie<Z>(n, [&](int s) {
    // 返回 s 中所有坏条件同时成立时的方案数
    return calc(s);
});
```

所有子集和

已知 `f[S]`，求

$$g[S]=\sum_{T\subseteq S} f[T]$$

```
auto g = f;
zeta_sub(g);
```

从所有子集和恢复原数组

```
mobius_sub(g);
// 此时 g == f
```

所有超集和

```
auto g = f;
zeta_sup(g);
```

题型识别

题目特征	优先检查
禁止若干性质，交集容易统计	普通容斥
条件完全对称，只与选了几个有关	按大小合并容斥
恰好 / 至少满足 k 个条件	按命中数容斥
出现 $\sum \binom{n}{k} f_k$	二项式反演
对所有 `S` 求所有子集贡献和	`zeta_sub`
对所有 `S` 求所有超集贡献和	`zeta_sup`

题目特征	优先检查
已知上述和，恢复原数组	Möbius 逆变换
出现 $\sum_{d\mid n}$ 或倍数求和	数论 Möbius 反演

复杂度

接口 / 方法	时间复杂度	空间复杂度
直接子集容斥	$O(2^n)$ 次 `f` 计算	$O(1)$ 额外空间
对称容斥	$O(n)$ 次交集计数	$O(1)$
`zeta_sub` / `mobius_sub`	$O(n2^n)$	$O(1)$ 额外空间
`zeta_sup` / `mobius_sup`	$O(n2^n)$	$O(1)$ 额外空间
普通二项式反演直接计算	$O(n^2)$	依实现

9.4 置换与置换环

解决问题： 对置换求复合、逆、幂、循环分解、阶与奇偶性，并利用置换环分析周期结构
约定 `p[i]` 表示置换把 `i` 映射到 `p[i]`

```
struct Perm {
    vector<int> p;
    explicit Perm(int n = 0) : p(n) { iota(p.begin(), p.end(), 0); }
    explicit Perm(vector<int> p_) : p(move(p_)) {}
    int size() const { return p.size(); }
    int &operator[](int i) { return p[i]; }
    const int &operator[](int i) const { return p[i]; }
    // p * q = p ∘ q: 先作用 q，再作用 p
    Perm operator*(const Perm &q) const {
        int n{ size() };
        Perm r(n);
        for (int i = 0; i < n; ++ i) r[i] = p[q[i]];
        return r;
    }
    Perm inv() const {
        int n{ size() };
        Perm q(n);
        for (int i = 0; i < n; ++ i) q[p[i]] = i;
        return q;
    }
    vector<vector<int>> cycles() const {
        int n{ size() };
        vector<char> vis(n);
        vector<vector<int>> res;
        for (int i = 0; i < n; ++ i) if (!vis[i]) {
            vector<int> c;
            for (int u = i; !vis[u]; u = p[u]) {
                vis[u] = true;
                c.push_back(u);
            }
            res.push_back(move(c));
        }
        return res;
    }
    // O(n)，支持负指数
    Perm pow(i64 k) const {
        int n{ size() };
        Perm r(n);
        for (const auto &c : cycles()) {
            int m{ int(c.size()) };
            int d{ int(k % m) };
            if (d < 0) d += m;
            for (int i = 0; i < m; ++ i) r[c[i]] = c[(i + d) % m];
        }
        return r;
    }
    // ===== 以下函数可直接删除：只需要基本置换运算时 =====
```

```
    vector<int> lens() const {
        vector<int> a;
        for (const auto &c : cycles()) a.push_back(c.size());
        return a;
    }
    i64 ord() const {
        i64 res{ 1 };
        for (int x : lens()) res = lcm(res, i64(x));
        return res;
    }
    // 0: 偶置换，1: 奇置换
    int parity() const {
        return (size() - int(cycles().size())) & 1;
    }
};
```

抽象模型

一个 `0..n-1` 上的置换是一个双射

$$p : [0, n) \to [0, n)$$

把每个 `i` 向 `p[i]` 连一条有向边，由于每个点的入度、出度都恰为 (1)，整个图一定分解成若干个互不相交的有向环：

$$p = (a_1\, a_2\, \ldots\, a_k)(b_1\, b_2\, \ldots\, b_m)\cdots$$

因此处理置换时最重要的思想是：

> 不要把它当普通数组；先考虑能不能拆成置换环
> 很多看似需要快速幂、图遍历的问题，在环上都只是取模

接口说明

- `Perm(n)`：构造大小为 `n` 的恒等置换
- `Perm(p)`：由映射数组构造置换
- `p * q`：置换复合，定义为 (p\circ q)，即先 `q` 后 `p`
- `p.inv()`：返回逆置换
- `p.cycles()`：返回所有置换环，包括长度为 `1` 的不动点
- `p.pow(k)`：返回 (p^k)，支持负指数
- `p.lens()`：返回所有置换环长度
- `p.ord()`：返回置换的阶
- `p.parity()`：`0` 表示偶置换，`1` 表示奇置换
  所有接口要求 `p` 恰为 `0..n-1` 的一个排列；代码不额外检查合法性

置换复合与逆

若

$$r = p \circ q$$

则

$$r(i) = p(q(i))$$

代码中约定：

```
Perm r = p * q;
```

表示先作用 `q`，再作用 `p`
逆置换满足

$$p^{-1}(p(i)) = i$$

因此直接：

```
q[p[i]] = i;
```

即可在 $O(n)$ 求出
注意置换复合一般不满足交换律：

$$pq \neq qp$$

所以复合顺序必须统一

置换幂

若一个环为

$$(a_0\, a_1\, \dots\, a_{m-1})$$

则

$$p(a_i) = a_{i+1 \bmod m}$$

于是

$$p^k(a_i) = a_{i+k \bmod m}$$

所以 `pow(k)` 不需要普通二进制快速幂，可以直接在每个环上移动

$$k \bmod m$$

个位置，总复杂度只有

$$O(n)$$

这比把置换当普通映射做

$$O(n \log k)$$

快速幂更自然

负指数

因为

$$p^{-k} = (p^{-1})^k$$

环上只需把位移

$$k \bmod m$$

调整到 `0..m-1`，因此同一份代码可以直接处理负数 `k`

置换的阶

置换的阶是最小正整数 t，满足

$$p^t = \mathrm{id}$$

若置换环长度分别为

$$l_1, l_2, \dots, l_c$$

则

$$\boxed{\mathrm{ord}(p) = \mathrm{lcm}(l_1, l_2, \dots, l_c)}$$

原因是每个环都必须同时回到原位

> `ord()` 使用 `i64`，调用前应保证所有环长的最小公倍数不会溢出

p^k 的环结构

一个长度为 m 的环，在 p^k 中会分裂成

$$\gcd(m, k)$$

个环，每个环长度为

$$\frac{m}{\gcd(m, k)}$$

因此若 `p` 的环长为 $l_1, \dots, l_c$，则

$$\boxed{c(p^k) = \sum_{i=1}^{c} \gcd(l_i, k)}$$

其中 $c(q)$ 表示置换 `q` 的置换环数量

p^k 的不动点

原来一个长度为 m 的环中的所有元素在 p^k 下都是不动点，当且仅当

$$m \mid k$$

若长度为 m 的环有 a_m 个，则

$$\boxed{|\operatorname{Fix}(p^k)| = \sum_{m \mid k} m\, a_m}$$

这在循环群作用计数里尤其常见

奇偶性与交换次数

一个长度为 m 的环可以写成 $m-1$ 个交换：

$$(a_1\, a_2\, \dots\, a_m) = (a_1\, a_m) \cdots (a_1\, a_3)(a_1\, a_2)$$

若一个 n 元置换共有 c 个环，包括不动点，则总共需要

$$n - c$$

个交换
因此置换符号为

$$\operatorname{sgn}(p) = (-1)^{n-c}$$

所以：

- $n-c$ 为偶数：偶置换
- $n-c$ 为奇数：奇置换
  同时，

$$\boxed{n-c}$$

也是把该置换变成恒等置换所需的最少交换次数
这与逆序数奇偶性一致：

$$\operatorname{inv}(p) \equiv n - c \pmod 2$$

环型与共轭

只记录各个环的长度，而忽略环中具体元素，得到置换的环型
例如

$$(0\,3\,2)(1\,5)(4)$$

的环型为

$$(3, 2, 1)$$

所以一个 n 元置换的环型，本质上就是 n 的一个整数分拆
在对称群 S_n 中：

$$\boxed{p, q \text{ 共轭} \iff p, q \text{ 环型相同}}$$

因此以后处理 Burnside、共轭类或置换群时，很多时候真正重要的并不是具体置换，而是各种长度的环分别有多少个
若长度为 i 的环恰有 m_i 个，

$$\sum_{i \geq 1} i\, m_i = n$$

则具有该环型的置换数为

$$\boxed{\frac{n!}{\prod_{i \geq 1} i^{m_i} m_i!}}$$

与经典数列的关系

无符号第一类 Stirling 数

$$\begin{bmatrix} n \\ k \end{bmatrix}$$

正好计数：

> n 元置换恰有 k 个置换环的方案数

常见用法

循环分解

```
Perm p(a);
auto cs = p.cycles();
for (const auto &c : cs) {
    // c 是一个置换环
}
```

求逆置换

```
Perm q = p.inv();
```

复合

```
Perm r = p * q; // r = p ∘ q，先 q 后 p
```

求很大的置换幂

```
Perm q = p.pow(k);
```

即使 `k` 很大，也只需

$$O(n)$$

求周期

```
i64 t = p.ord();
```

满足

$$p^t = \mathrm{id}$$

判断奇偶置换

```
if (p.parity()) {
    // 奇置换
} else {
    // 偶置换
}
```

题型识别

题目特征	优先检查
每个元素恰好映射到另一个元素，且映射为双射	置换环
重复执行同一个重排很多次	环分解 + `k % len`
求多少次操作后整体恢复	环长 `lcm`
求重复作用后的环数	$\sum \gcd(l_i, k)$
求重复作用后的不动点	枚举满足 $l_i \mid k$ 的环
最少交换多少次恢复	$n - c$
判断奇偶置换	$(n - c) \bmod 2$
只关心置换的结构、不关心具体标号	环型
统计恰有 `k` 个环的排列	第一类 Stirling
排列字典序排名	康托展开
统计下降位置	Eulerian 数
旋转、反射等对称操作计数	下一节群作用 / Burnside / Pólya

注意事项

- 本节全部采用 `0base`
- `p[i]` 表示 `i → p[i]`
- `p * q` 定义为 (p\circ q)，即先 `q` 后 `p`
- `cycles()` 包含不动点
- `parity()` 和最少交换次数计算环数时也必须包含不动点

- 普通置换幂不要写成 (O(n\log k)) 快速幂；利用环可以做到 (O(n))
- `ord()` 的最小公倍数可能超过 `i64`
- “排列的逆序数”“下降数”等是排列作为序列的统计量；“环长、阶、共轭类”等是置换作为映射的结构量，两类概念不要混在一起

复杂度

接口	时间复杂度	空间复杂度
`p * q` / `inv()` / `cycles()` / `pow(k)` / `lens()` / `ord()` / `parity()`	$O(n)$	$O(n)$

10. 多项式、卷积与生成函数

10.1 卷积与变换基础

10.1.1 NTT（998244353）

```
template<class T>
vector<T> conv(const vector<T> &a, const vector<T> &b) {
    if (a.empty() || b.empty()) return {};
    vector<T> c(a.size() + b.size() - 1);
    for (int i = 0; i < int(a.size()); ++ i) {
        for (int j = 0; j < int(b.size()); ++ j) c[i + j] += a[i] * b[j];
    }
    return c;
}
namespace NTT {
    constexpr int MOD = 998244353, G = 3, MAX_N = 1 << 23;
    static_assert(Z::mod() == MOD);
    inline int base = 1;
    inline vector<int> rev{0, 1};
    inline vector<Z> roots{0, 1};

    int ntt_size(int need) { return int(bit_ceil(unsigned(need))); }
    void ensure(int n) {
        int k = countr_zero(unsigned(n));
        if (k <= base) return;
        rev.resize(n);
        for (int i = 0; i < n; ++ i) {
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
        }
        roots.resize(n);
        while (base < k) {
            Z e = Z(G).pow((MOD - 1) >> (base + 1));
            for (int i = 1 << (base - 1); i < (1 << base); ++ i) {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * e;
            }
            ++ base;
        }
    }
    void dft(vector<Z> &a) {
        int n = a.size();
        assert(n > 0 && has_single_bit(unsigned(n)) && n <= MAX_N);
        ensure(n);
        int shift = base - countr_zero(unsigned(n));
        for (int i = 0; i < n; ++ i) {
            int j = rev[i] >> shift;
            if (i < j) swap(a[i], a[j]);
        }
        for (int k = 1; k < n; k <<= 1) {
            for (int i = 0; i < n; i += 2 * k) {
                for (int j = 0; j < k; ++ j) {
                    Z x = a[i + j], y = a[i + j + k] * roots[k + j];
                    a[i + j] = x + y;
                    a[i + j + k] = x - y;
                }
```

```
            }
        }
    }
    void idft(vector<Z> &a) {
        assert(!a.empty());
        reverse(a.begin() + 1, a.end());
        dft(a);
        Z inv = (1 - MOD) / int(a.size());
        for (Z &x : a) x *= inv;
    }
    vector<Z> conv(const vector<Z> &a, const vector<Z> &b) {
        if (a.empty() || b.empty()) return {};
        if (min(a.size(), b.size()) <= 40) return ::conv<Z>(a, b);
        int need = a.size() + b.size() - 1, n = ntt_size(need);
        assert(n <= MAX_N);
        vector<Z> f(n);
        copy(a.begin(), a.end(), f.begin());
        dft(f);
        if (&a == &b) {
            for (Z &x : f) x *= x;
        } else {
            vector<Z> g(n);
            copy(b.begin(), b.end(), g.begin());
            dft(g);
            for (int i = 0; i < n; ++ i) f[i] *= g[i];
        }
        idft(f);
        f.resize(need);
        return f;
    }
}
vector<Z> conv(const vector<Z> &a, const vector<Z> &b) {
    return NTT::conv(a, b);
}
```

接口说明

- `conv(a, b)`：普通卷积，自动选朴素乘法或 NTT；相同容器引用传入 `conv(a,a)` 会复用变换。
- `NTT::dft(a)` / `NTT::idft(a)`：原地正变换 / 逆变换，`a.size()` 必须是合法的 2 的幂。
- `NTT::ntt_size(need)`：不小于 `need` 的最小 2 的幂（`need > 0`）。
- `NTT::MAX_N`：2^23；`ensure(n)` 为内部缓存，无需主动调用。
- 泛型 `conv<T>(a,b)`：朴素乘法，适用于非 `Z` 系数；不会使用 NTT。

注意事项

- 依赖已有静态模整数 `Z = MInt<998244353>`，并要求提供 `Z::pow()`、`val()` 与四则运算；需 C++20（`bit_ceil`、`countr_zero`、`has_single_bit`）。
- 当 `a.size()+b.size()-1 > 2^23` 时本版本不支持；不默认带大内存的超长卷积。若题目需要可分块。
- `dft` 会修改输入；做逆变换前频域必须满足预期语义。非 `Z` 的 `conv` 为 $O(nm)$。
- `conv(a,a)` 与 `conv(a, vector<Z>(a))` 结果相同，但只有前者触发平方优化。

常见用法

```
vector<Z> a{1, 2, 3}, b{4, 5};
auto c = conv(a, b);       // [4,13,22,15]
auto sq = conv(a, a);      // 平方只做一次正向 NTT
int k = 3;
c.resize(min<int>(k, c.size())); // 取前 k 项
```

频域逐项乘法（需自行补零到 `NTT::ntt_size(a.size()+b.size()-1)`）：

```
int n = NTT::ntt_size(a.size() + b.size() - 1);
a.resize(n); b.resize(n);
NTT::dft(a); NTT::dft(b);
for (int i = 0; i < n; ++ i) a[i] *= b[i];
NTT::idft(a);
```

复杂度

- 朴素分支：$O(|a||b|)$，其余 $O(N \log N)$ 时间、$O(N)$ 额外空间；`N=bit_ceil(|a|+|b|-1)`。

10.1.2 FFT

```
template<class Real = double>
struct FFT {
    static_assert(is_floating_point_v<Real>);
    using C = complex<Real>;
    inline static int base = 1;
    inline static vector<int> rev{0, 1};
    inline static vector<C> roots{C(0), C(1)};

    static void ensure(int n) {
        int k = countr_zero(unsigned(n));
        if (k <= base) return;
        rev.resize(n);
        for (int i = 0; i < n; ++ i) {
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
        }
        roots.resize(n);
        while (base < k) {
            int m = 1 << base;
            long double angle = acosl(-1.L) / m;
            complex<long double> w(cosl(angle), sinl(angle));
            for (int i = m / 2; i < m; ++ i) {
                roots[2 * i] = roots[i];
                auto z = complex<long double>(roots[i].real(), roots[i].imag()) * w;
                roots[2 * i + 1] = C(Real(z.real()), Real(z.imag()));
            }
            ++ base;
        }
    }
    static void fft(vector<C> &a, bool inverse = false) {
        int n = a.size();
        assert(n > 0 && has_single_bit(unsigned(n)));
        ensure(n);
        int shift = base - countr_zero(unsigned(n));
        for (int i = 0; i < n; ++ i) {
            int j = rev[i] >> shift;
            if (i < j) swap(a[i], a[j]);
        }
        for (int k = 1; k < n; k <<= 1) {
            for (int i = 0; i < n; i += 2 * k) {
                for (int j = 0; j < k; ++ j) {
                    C x = a[i + j], y = a[i + j + k] * roots[k + j];
                    a[i + j] = x + y;
                    a[i + j + k] = x - y;
                }
            }
        }
        if (inverse) {
            reverse(a.begin() + 1, a.end());
            Real inv = Real(1) / n;
            for (C &x : a) x *= inv;
        }
    }
    template<class A, class B>
    static vector<C> conv_complex(const vector<A> &a, const vector<B> &b) {
        if (a.empty() || b.empty()) return {};
        int need = a.size() + b.size() - 1;
        if (1LL * a.size() * b.size() <= 4096) {
            vector<C> c(need);
            for (int i = 0; i < int(a.size()); ++ i) {
                for (int j = 0; j < int(b.size()); ++ j) {
                    c[i + j] += C(a[i]) * C(b[j]);
                }
            }
            return c;
        }
        int n = int(bit_ceil(unsigned(need)));
        vector<C> f(n), g(n);
```

```
        for (int i = 0; i < int(a.size()); ++ i) f[i] = C(a[i]);
        for (int i = 0; i < int(b.size()); ++ i) g[i] = C(b[i]);
        fft(f);
        fft(g);
        for (int i = 0; i < n; ++ i) f[i] *= g[i];
        fft(f, true);
        f.resize(need);
        return f;
    }
    // 不打包：使用两次独立正向 FFT，降低额外消去误差。
    template<class A, class B>
    static vector<Real> conv_real(const vector<A> &a, const vector<B> &b) {
        if (a.empty() || b.empty()) return {};
        int need = a.size() + b.size() - 1;
        if (1LL * a.size() * b.size() <= 4096) {
            vector<Real> c(need);
            for (int i = 0; i < int(a.size()); ++ i) {
                for (int j = 0; j < int(b.size()); ++ j) {
                    c[i + j] += Real(a[i]) * Real(b[j]);
                }
            }
            return c;
        }
        auto f = conv_complex(a, b);
        vector<Real> c(f.size());
        for (int i = 0; i < int(f.size()); ++ i) c[i] = f[i].real();
        return c;
    }
    template<class A, class B>
    static vector<i64> conv_round(const vector<A> &a, const vector<B> &b) {
        auto c = conv_real(a, b);
        vector<i64> res(c.size());
        for (int i = 0; i < int(c.size()); ++ i) res[i] = llround(c[i]);
        return res;
    }
};
```

接口说明

- `FFT<Real>::fft(a, inverse = false)`：原地复数 FFT / IFFT，长度须为非零的 2 的幂；$O(n \log n)$。
- `FFT<Real>::conv_complex(a, b)`：复系数卷积，支持实数或复数输入；$O(N \log N)$。
- `FFT<Real>::conv_real(a, b)`：实系数卷积，采用两次独立正向 FFT，不使用复数打包；$O(N \log N)$。
- `FFT<Real>::conv_round(a, b)`：对实系数卷积结果执行 `llround`，返回 `vector<i64>`；$O(N \log N)$。

注意事项

- 所有卷积均为**浮点近似计算**。即使不打包，`conv_round()` 也不能保证对任意大整数输入都精确；若需模数卷积的严格正确性，请使用 NTT / 合法范围内的 CRT。
- `conv_round()` 的逐项正确性要求计算误差不越过半整数取整边界，真实答案和取整输入必须落在 `i64` 可表示范围内。
- 旋转因子预处理在 `long double` 精度下计算，再存为 `Real`；这只降低误差，**不是精度保证**。
- `fft()` 会修改输入；`conv_*()` 返回完整长度 $|a| + |b| - 1$ 的卷积，空数组返回空数组。
- 小规模 $|a||b| \le 4096$ 直接朴素卷积；复用缓存的 `rev`、`roots` 按本模板实例使用过的最大变换长度保留内存。
- 默认 `double` 偏快；大整数近似卷积可考虑 `long double`，但若题目要求严格整数结果，优先选择有数学精确性保证的方法。

常见用法

```
vector<int> a{1, 2, 3}, b{4, 5};
auto c = FFT<>::conv_round(a, b);          // {4, 13, 22, 15}
auto r = FFT<>::conv_real(a, b);           // vector<double>
auto hi = FFT<long double>::conv_round(a, b);

using C = complex<double>;
auto z = FFT<>::conv_complex(vector<C>{{1, 1}}, vector<C>{{2, 3}});
// z[0] = -1 + 5i

vector<C> f{1, 2, 3, 0};
FFT<>::fft(f);                             // 正向 FFT
FFT<>::fft(f, true);                       // 原地还原原数组
```

复杂度

设 $N = \text{bit_ceil}(|a| + |b| - 1)$。

操作	时间复杂度	额外空间复杂度
`fft()`	$O(n \log n)$	$O(n)$ 缓存（复用）
`conv_complex()`	$O(N \log N)$	$O(N)$
`conv_real()`	$O(N \log N)$	$O(N)$
`conv_round()`	$O(N \log N)$	$O(N)$

当 $|a||b| \le 4096$ 时，卷积使用 $O(|a||b|)$ 朴素算法。

10.1.3 任意模数卷积（三模 NTT + CRT）

```
namespace AnyModConv {
    constexpr int P1 = 167772161, P2 = 469762049, P3 = 998244353;
    constexpr int G = 3, MAX_N = 1 << 23;
    constexpr i64 P12 = 1LL * P1 * P2;

    template<int P>
    int power(int a, int b) {
        i64 r = 1;
        for (; b; b >>= 1, a = 1LL * a * a % P) {
            if (b & 1) r = r * a % P;
        }
        return r;
    }
    template<int P>
    void ntt(vector<int> &a, bool inverse = false) {
        int n = a.size();
        assert(n > 0 && has_single_bit(unsigned(n)) && (P - 1) % n == 0);
        for (int i = 1, j = 0; i < n; ++ i) {
            int b = n >> 1;
            while (j & b) j ^= b, b >>= 1;
            j ^= b;
            if (i < j) swap(a[i], a[j]);
        }
        for (int len = 2; len <= n; len <<= 1) {
            int w = power<P>(G, (P - 1) / len);
            if (inverse) w = power<P>(w, P - 2);
            for (int i = 0; i < n; i += len) {
                i64 t = 1;
                for (int j = 0; j < len / 2; ++ j) {
                    int x = a[i + j], y = t * a[i + j + len / 2] % P;
                    int u = x + y, v = x - y;
                    if (u >= P) u -= P;
                    if (v < 0) v += P;
                    a[i + j] = u;
                    a[i + j + len / 2] = v;
                    t = t * w % P;
                }
            }
        }
        if (inverse) {
            int inv = power<P>(n, P - 2);
            for (int &x : a) x = 1LL * x * inv % P;
        }
    }
    template<int P>
    vector<int> multiply(const vector<int> &a, const vector<int> &b) {
        int need = a.size() + b.size() - 1;
        int n = int(bit_ceil(unsigned(need)));
        assert(n <= MAX_N);
        vector<int> f(n);
        for (int i = 0; i < int(a.size()); ++ i) f[i] = a[i] % P;
        ntt<P>(f);
        if (&a == &b) {
            for (int &x : f) x = 1LL * x * x % P;
        } else {
```

```
            vector<int> g(n);
            for (int i = 0; i < int(b.size()); ++ i) g[i] = b[i] % P;
            ntt<P>(g);
            for (int i = 0; i < n; ++ i) f[i] = 1LL * f[i] * g[i] % P;
        }
        ntt<P>(f, true);
        f.resize(need);
        return f;
    }
    template<class T>
    vector<int> conv_mod(const vector<T> &a, const vector<T> &b, int mod) {
        static_assert(is_integral_v<T>);
        assert(mod > 0);
        if (a.empty() || b.empty()) return {};
        int need = a.size() + b.size() - 1;
        if (mod == 1) return vector<int>(need);
        vector<int> x(a.size()), y;
        auto norm = [&](T v) -> int {
            i64 r = i64(v % mod);
            if (r < 0) r += mod;
            return int(r);
        };
        for (int i = 0; i < int(a.size()); ++ i) x[i] = norm(a[i]);
        bool square = &a == &b;
        if (!square) {
            y.resize(b.size());
            for (int i = 0; i < int(b.size()); ++ i) y[i] = norm(b[i]);
        }
        const vector<int> &z = square ? x : y;
        if (min(x.size(), z.size()) <= 40) {
            vector<int> c(need);
            for (int i = 0; i < int(x.size()); ++ i) {
                for (int j = 0; j < int(z.size()); ++ j) {
                    c[i + j] = (c[i + j] + 1LL * x[i] * z[j]) % mod;
                }
            }
            return c;
        }
        assert(bit_ceil(unsigned(need)) <= MAX_N);
        auto c1 = multiply<P1>(x, z);
        auto c2 = multiply<P2>(x, z);
        auto c3 = multiply<P3>(x, z);
        static const i64 i12 = power<P2>(P1, P2 - 2);
        static const i64 i123 = power<P3>(P12 % P3, P3 - 2);
        vector<int> c(need);
        for (int i = 0; i < need; ++ i) {
            i64 t = (c2[i] - c1[i] + P2) * i12 % P2;
            i64 v = c1[i] + P1 * t;
            i64 u = ((c3[i] - v % P3 + P3) % P3) * i123 % P3;
            c[i] = (v + (__int128)P12 * u) % mod;
        }
        return c;
    }
}
template<class T>
vector<int> conv_mod(const vector<T> &a, const vector<T> &b, int mod) {
    return AnyModConv::conv_mod(a, b, mod);
}
```

接口说明

- `conv_mod(a,b,mod)`：返回 `vector<int>`，系数在 `[0,mod)`；支持负数输入与任意 `1 ≤ mod ≤ INT_MAX`。
- `AnyModConv::ntt<P>(a,inverse)`：单素数原地 NTT，仅供该实现内部使用。
- `AnyModConv::multiply<P>(a,b)`：内部单模卷积，可单独按需复制。

注意事项

- **严格精确的前提**：`bit_ceil(|a|+|b|−1) ≤ 2^23`、`mod ≤ INT_MAX`。此时卷积每个非负系数不超过 `min(|a|,|b|)(mod-1)^2`，在长度限制下小于 `P1*P2*P3`，CRT 重建唯一。
- 通过先按目标模数归一化，避免输入负值；`i64` 乘法安全，最终 CRT 汇总必须使用 `__int128`。
- 若 `mod == 998244353`，应优先直接用单模 `NTT::conv`，速度约为三模的 3 倍。

- 不支持系数绝对值超出整型 `T`、结果长度超限、目标模数超出 `int` 的情况。仅当有明确需要时再加大长度/任意大模数方案。

常见用法

```
vector<i64> a{1, -2, 3}, b{4, 5};
auto c = conv_mod(a, b, 1000000007); // {4, 1000000004, 2, 15}
auto d = conv_mod(a, a, 1000000000); // 平方优化
```

复杂度

- $O(N \log N)$ 时间（3 次独立卷积），$O(N)$ 额外空间；小数组回退 $O(|a||b|)$。

10.1.4 FWT（OR / AND / XOR 卷积）

```
namespace FWT {
    template<int type, class T>
    void transform(vector<T> &a, bool inverse = false) {
        int n = a.size();
        assert(n > 0 && has_single_bit(unsigned(n)));
        for (int k = 1; k < n; k <<= 1) {
            for (int i = 0; i < n; i += 2 * k) {
                for (int j = 0; j < k; ++ j) {
                    T &x = a[i + j], &y = a[i + j + k];
                    if constexpr (type == 0) { // OR: 子集 zeta / Möbius
                        if (inverse) y -= x;
                        else y += x;
                    } else if constexpr (type == 1) { // AND: 超集 zeta / Möbius
                        if (inverse) x -= y;
                        else x += y;
                    } else { // XOR: Walsh-Hadamard
                        T u = x, v = y;
                        x = u + v;
                        y = u - v;
                    }
                }
            }
        }
        if constexpr (type == 2) {
            if (inverse) for (T &x : a) x /= T(n);
        }
    }
    template<int type, class T>
    vector<T> conv(const vector<T> &a, const vector<T> &b) {
        if (a.empty() || b.empty()) return {};
        int n = int(bit_ceil(unsigned(max(a.size(), b.size()))));
        vector<T> f(n), g;
        copy(a.begin(), a.end(), f.begin());
        transform<type>(f);
        if (&a == &b) {
            for (T &x : f) x *= x;
        } else {
            g.resize(n);
            copy(b.begin(), b.end(), g.begin());
            transform<type>(g);
            for (int i = 0; i < n; ++ i) f[i] *= g[i];
        }
        transform<type>(f, true);
        return f;
    }
    template<class T>
    vector<T> conv(const vector<T> &a, const vector<T> &b, int type) {
        if (type == 0) return conv<0>(a, b);
        if (type == 1) return conv<1>(a, b);
        assert(type == 2);
        return conv<2>(a, b);
    }
}
template<class T> vector<T> conv_or(const vector<T> &a, const vector<T> &b) {
    return FWT::conv<0>(a, b);
```

```
}
template<class T> vector<T> conv_and(const vector<T> &a, const vector<T> &b) {
    return FWT::conv<1>(a, b);
}
template<class T> vector<T> conv_xor(const vector<T> &a, const vector<T> &b) {
    return FWT::conv<2>(a, b);
}
```

接口说明

- `conv_or / conv_and / conv_xor(a,b)`：按位或、按位与、按位异或卷积，结果长度为 `bit_ceil(max(|a|,|b|))`。
- `FWT::transform<type>(a,inverse)`：`type=0/1/2` 对应 OR/AND/XOR，编译期常量让蝶形内不出现动态分支；`inverse=true` 为逆变换。
- `FWT::conv<type>(a,b)`：快速编译期分发；也保留 `FWT::conv(a,b,type)` 运行时入口（三路只在函数入口分支）。

注意事项

- 输入表示 mask `0..2^k-1`；不足长度自动补零。多于 2^k 不会折回。
- XOR 逆变换需要 2^k 在系数环中可逆。使用 `Z` 没问题；**整数 `vector<i64>` 不适合逐步做 `/n` 后再用于非整数代数扩展**，且中间值可能溢出。
- OR 卷积 `a[A]b[B]` 汇总到 `A|B`，不是要求 `A∩B=∅` 的子集卷积。

常见用法

```
vector<Z> a(1 << k), b(1 << k);
auto c0 = conv_or(a, b);
auto c1 = conv_and(a, b);
auto c2 = conv_xor(a, b);
// 只做一次变换、多次频域乘法：
FWT::transform<2>(a);
FWT::transform<2>(a, true);
```

复杂度

- 三种变换及卷积均为 $O(k \cdot 2^k)$ 时间、$O(2^k)$ 额外空间。

10.1.5 循环与负循环卷积

> 依赖：全局 `conv()`；直接版额外依赖 `10.1.1 NTT` 的 `NTT::dft/idft`

```
namespace CyclicConv {
    template<class T>
    vector<T> reduce(const vector<T> &a, int n, bool neg = false) {
        assert(n > 0);
        vector<T> f(n);
        for (int i = 0; i < int(a.size()); ++ i) {
            if (neg && (i / n & 1)) f[i % n] -= a[i];
            else f[i % n] += a[i];
        }
        return f;
    }
    template<class T>
    vector<T> conv(const vector<T> &a, const vector<T> &b, int n, bool neg = false) {
        assert(n > 0);
        if (a.empty() || b.empty()) return vector<T>(n);
        auto f = reduce(a, n, neg);
        vector<T> g;
        if (&a != &b) g = reduce(b, n, neg);
        return reduce(::conv(f, &a == &b ? f : g), n, neg);
    }
    // 仅 Z；n 为 2 的幂。直接 NTT 的长度为 n 而非 2n。
    vector<Z> conv_ntt(const vector<Z> &a, const vector<Z> &b, int n, bool neg = false) {
        assert(n > 0 && has_single_bit(unsigned(n)));
        assert((neg ? 2LL * n : n) <= NTT::MAX_N);
        vector<Z> f = reduce(a, n, neg);
        bool square = &a == &b;
        vector<Z> g;
        if (!square) g = reduce(b, n, neg);
        Z w = 1, iw = 1;
        if (neg) {
            w = Z(NTT::G).pow((NTT::MOD - 1) / (2 * n));
```

```
            iw = w.inv();
        }
        Z t = 1;
        for (int i = 0; i < n; ++ i) {
            f[i] *= t;
            if (!square) g[i] *= t;
            t *= w;
        }
        NTT::dft(f);
        if (square) {
            for (Z &x : f) x *= x;
        } else {
            NTT::dft(g);
            for (int i = 0; i < n; ++ i) f[i] *= g[i];
        }
        NTT::idft(f);
        t = 1;
        for (Z &x : f) x *= t, t *= iw;
        return f;
    }
}
template<class T>
vector<T> conv_cyclic(const vector<T> &a, const vector<T> &b, int n) {
    return CyclicConv::conv(a, b, n);
}
template<class T>
vector<T> conv_negacyclic(const vector<T> &a, const vector<T> &b, int n) {
    return CyclicConv::conv(a, b, n, true);
}
```

接口说明

- `conv_cyclic(a,b,n)`：模 x^n-1 的乘法，返回恰好 `n` 项。
- `conv_negacyclic(a,b,n)`：模 x^n+1 的乘法，返回恰好 `n` 项。
- `CyclicConv::conv_ntt(a,b,n,neg)`：`Z` 的直接 NTT 版本，`neg=true` 代表负循环。
- `CyclicConv::reduce(a,n,neg)`：单个多项式按 $x^n\mp1$ 折回。

注意事项

- 通用版先折回两边，再调用全局 `conv`，可以使用非 2 的幂周期 `n`。
- 直接版要求 `n` 为 2 的幂，循环版 `n ≤ 2^23`，负循环版 `2n ≤ 2^23`；利用 `2n` 次单位根扭转。
- 输入可以长于 `n`；空输入按零多项式处理，结果仍返回长度 `n` 的全零数组。
- `conv_ntt` 在 `n=1` 时也成立；仅需要通用版时可以整个删除 `conv_ntt()`。

常见用法

```
vector<Z> a{1, 2, 3}, b{4, 5};
auto c = conv_cyclic(a, b, 3);     // {19,13,22}
auto d = conv_negacyclic(a, b, 3); // {-11,13,22}（模意义）
auto e = CyclicConv::conv_ntt(a, b, 4);       // 直接循环
// auto f = CyclicConv::conv_ntt(a, b, 4, true); // 直接负循环
```

复杂度

- 通用：$O(|a|+|b|+M(n))$（折回+一次长度至多 `2n-1` 的卷积）。
- 直接：$O(n\log n)$ 时间、$O(n)$ 空间；$M(n)$ 表示卷积复杂度。

10.1.6 CDQ 分治卷积与在线卷积

> 依赖：全局 `conv(a,b)`；若要快速卷积，系数用 `Z` 并引入 `10.1.1 NTT`。

```
// ===== CDQ 分治卷积：固定核 + 前向递推；不需要时可整段删除 =====
// solve(i, s) 根据 sum_{j<i} f[j] * g[i-j] 确定 f[i]；g[0] 不参与 s
template<class T, class F>
vector<T> cdq_conv(int n, const vector<T> &g, F solve) {
    vector<T> f(n), sum(n);
    auto dfs = [&](auto &&self, int l, int r) -> void {
        if (r - l == 1) {
            f[l] = solve(l, sum[l]);
```

```
            return;
        }
        int m{ (l + r) >> 1 };
        self(self, l, m);
        // 左半已确定，将 f[l..m) 对 sum[m..r) 的贡献批量加入
        int len{ min(r - l, int(g.size())) };
        if (len > 1) {
            vector<T> a(f.begin() + l, f.begin() + m);
            vector<T> b(g.begin() + 1, g.begin() + len);
            vector<T> c{ ::conv(a, b) };
            for (int i = m; i < r; ++ i) {
                int k{ i - l - 1 };
                if (k < c.size()) sum[i] += c[k];
            }
        }
        self(self, m, r);
    };
    if (n) dfs(dfs, 0, n);
    return f;
}
// ===== Semi-Relaxed: f 预先已知，g 逐项加入；不需要时可整段删除 =====
template<class T = Z>
struct SemiRelaxedConv {
    vector<T> f, g, h;
    explicit SemiRelaxedConv(vector<T> f_) : f(move(f_)) {}
private:
    void ensure(int n) {
        if (h.size() < n) h.resize(n);
    }
    // 已知前 q 项 g；d = lowbit(q)
    // 新完成块 g[q-d..q) × f[0..2d) 中，只把次数 [d,2d) 加到未来 h[q..q+d)
    void add_future(int q, int d) {
        int m{ min(2 * d, int(f.size())) };
        if (!m) return;
        vector<T> a(g.end() - d, g.end());
        vector<T> b(f.begin(), f.begin() + m);
        vector<T> c{ ::conv(a, b) };
        int r{ min(2 * d, int(c.size())) };
        ensure(q + r - d);
        for (int i = d; i < r; ++ i) h[q + i - d] += c[i];
    }
public:
    // 加入 g[i]，返回完整的 h[i]
    T append(const T &x) {
        int i{ int(g.size()) };
        g.push_back(x);
        ensure(i + 1);
        // pending() 已提前维护其它项；这里只补当前未知的 f[0] * g[i]
        if (!f.empty()) h[i] += f[0] * x;
        int q{ i + 1 }, d{ q & -q };
        add_future(q, d);
        return h[i];
    }
    // ===== 查询辅助：只需要 append() 时以下函数可直接删除 =====
    int size() const { return int(g.size()); }
    // 要求 0 ≤ i < size()；返回已经完整确定的 h[i]
    const T &operator[](int i) const { return h[i]; }
    // 加入 g[n] 前，h[n] 中已经由 g[0..n) 确定的部分
    T pending() const {
        int n{ size() };
        return n < h.size() ? h[n] : T{};
    }
};
// ===== Relaxed: f、g 均逐项加入；不需要时可整段删除 =====
template<class T = Z>
struct RelaxedConv {
    vector<T> f, g, h;
private:
    void ensure(int n) {
        if (int(h.size()) < n) h.resize(n);
    }
```

```
    // 将局部卷积次数 [d,2d) 加到全局 h[q..q+d)
    void add_slice(int q, int d, const vector<T> &c) {
        int r{ min(2 * d, int(c.size())) };
        ensure(q + r - d);
        for (int i = d; i < r; ++ i) h[q + i - d] += c[i];
    }
    // 已知前 q 项; d = lowbit(q), 提前维护下一段尚未完成的卷积系数
    void add_future(int q, int d) {
        if (q == d) {
            // q 为 2 的幂: 整个已知前缀第一次形成一个完整块
            vector<T> a(f.begin(), f.begin() + d);
            vector<T> b(g.begin(), g.begin() + d);
            add_slice(q, d, ::conv(a, b));
            return;
        }
        // 新块 [q-d,q) 分别与另一侧最前面的 2d 项交叉
        vector<T> a(f.begin() + q - d, f.begin() + q);
        vector<T> b(g.begin(), g.begin() + 2 * d);
        vector<T> c{ ::conv(a, b) };
        a.assign(g.begin() + q - d, g.begin() + q);
        b.assign(f.begin(), f.begin() + 2 * d);
        vector<T> t{ ::conv(a, b) };
        for (int i = 0; i < t.size(); ++ i) c[i] += t[i];
        add_slice(q, d, c);
    }
public:
    // 加入 f[i], g[i], 返回完整的 h[i]
    T append(const T &x, const T &y) {
        int i = f.size();
        f.push_back(x);
        g.push_back(y);
        ensure(i + 1);
        // h[i] 的内部项已经由之前的块提前加入, 只补两个新出现的端点
        if (i == 0) h[0] += x * y;
        else h[i] += x * g[0] + y * f[0];
        int q{ i + 1 }, d{ q & -q };
        add_future(q, d);
        return h[i];
    }
    // ===== 查询辅助: 只需要 append() 时以下函数可直接删除 =====
    int size() const { return int(f.size()); }
    // 要求 0 ≤ i < size(); 返回已经完整确定的 h[i]
    const T &operator[](int i) const { return h[i]; }
    // 加入 f[n], g[n] 前, h[n] 中由旧系数产生的内部贡献
    T pending() const {
        int n{ size() };
        return n < int(h.size()) ? h[n] : T{};
    }
};
```

接口说明

- `cdq_conv(n,g,solve)`：求 `f[i]=solve(i, Σ_{j<i}f[j]g[i-j])`，只计算旧项贡献，`g[0]` 不使用。
- `SemiRelaxedConv<T>(f)`：核 `f` 固定；`append(g_i)` 返回 `Σ_{j=0}^i f[j]g[i-j]`。
- `RelaxedConv<T>()`：两个数组同步在线加入；`append(f_i,g_i)` 返回卷积第 `i` 项。
- `pending()`：尚未加入下一项前已确定的 `h[n]` 的旧项部分。`size()`、`operator[]` 查询历史状态。

注意事项

- 两种在线结构只允许末尾追加，不允许修改已有项；若两边已全部给出，直接 `conv` 更快。
- `SemiRelaxedConv::pending()` 不含新项 `f[0]*g[n]`；`RelaxedConv::pending()` 不含两项新端点 `f[0]*g[n] + f[n]*g[0]`（`n>0`）。
- CDQ 中 `solve(i,sum)` 应只依赖已经计算的 `f[0..i)`，不能读取未来项。`g[0]` 不参与 `sum`，即使它非零也是如此。
- 所有数据结构复用全局 `conv`；对于非 `Z` 的数组，只会执行朴素乘法，不保证快速复杂度。

常见用法

```
// f[0]=1, f[i]=sum_{j<i} f[j]*g[i-j]
vector<Z> g(n);
auto f = cdq_conv<Z>(n, g, [&](int i, Z sum) -> Z {
    return i == 0 ? Z(1) : sum;
});
```

```
// 固定核的在线卷积：加入序列 x
SemiRelaxedConv<Z> s(kernel);
Z old = s.pending();
Z value = s.append(x);

// 双侧在线卷积
RelaxedConv<Z> r;
Z prev = r.pending();
Z cur = r.append(x, y);
```

复杂度

- CDQ：$O(N\log^2 N)$ 时间，$O(N)$ 存储加上临时卷积（使用 NTT）。
- 半在线 / 全在线：总计典型 $O(N\log^2 N)$ 时间，$O(N)$ 持久存储，临时卷积需 $O(N)$。

10.1.7 子集卷积（按 popcount 分层）

> 依赖：`10.1.4 FWT` 中的 `FWT::transform`（OR 变换）；不依赖 NTT。

```
namespace SubsetConv {
    // c[S] = sum_{A subseteq S} a[A] * b[S \ A]
    template<class T = Z>
    vector<T> conv_subset(const vector<T> &a, const vector<T> &b) {
        if (a.empty() || b.empty()) return {};
        int need{ int(max(a.size(), b.size())) };
        int n{ int(bit_ceil(unsigned(need))) };
        int k{ countr_zero(unsigned(n)) };
        // f[r][S] / g[r][S]：仅保留 popcount(S) = r 的系数，再做子集 Zeta
        vector<vector<T>> f(k + 1, vector<T>(n));
        bool square{ &a == &b };
        vector<vector<T>> g;
        if (!square) g.assign(k + 1, vector<T>(n));
        for (unsigned s = 0; s < a.size(); ++ s) f[popcount(s)][s] = a[s];
        if (!square) {
            for (unsigned s = 0; s < b.size(); ++ s) g[popcount(s)][s] = b[s];
        }
        for (auto &v : f) FWT::transform<0>(v);
        if (!square) for (auto &v : g) FWT::transform<0>(v);
        // 每个 mask 上按集合大小做普通卷积；从高 rank 向低 rank 可原地覆盖 f
        for (int s = 0; s < n; ++ s) {
            for (int r = k; r >= 0; -- r) {
                T x{};
                for (int i = 0; i <= r; ++ i) {
                    x += f[i][s] * (square ? f[r - i][s] : g[r - i][s]);
                }
                f[r][s] = x;
            }
        }
        for (auto &v : f) FWT::transform<0>(v, true);
        vector<T> res(n);
        for (unsigned s = 0; s < n; ++ s) res[s] = f[popcount(s)][s];
        return res;
    }
}
template<class T = Z>
vector<T> conv_subset(const vector<T> &a, const vector<T> &b) {
    return SubsetConv::conv_subset(a, b);
}
```

接口说明

- `conv_subset(a,b)`：返回 `c[S] = Σ_{A⊆S}a[A]b[S\A]`，其中 `S\A` 是集合差，要求两侧不相交。
- 输入至少有一项时输出长度为 `bit_ceil(max(|a|,|b|))`，小于长度的 mask 缺省系数按零处理。

注意事项

- 与 OR 卷积不同：OR 卷积允许 $A\cap B\neq\varnothing$，本卷积不允许。
- 要求状态数为 2^k，且所存数组索引等于集合的二进制 mask。

- 存储 $O(k2^k)$：当 `k` 比较大时内存压力很高；若只需要 OR/AND/XOR 卷积，不要引入此实现。
- 此实现复用 `FWT::transform<0>(v,...)`；只使用子集卷积时可将 OR 变换函数单独复制过来。

常见用法

```
vector<Z> a(1 << k), b(1 << k);
auto c = conv_subset(a, b);
// 集合划分：a[A] * b[S\\A]，枚举有序分组 (A, S\\A)
// 平方：直接传同一容器引用，避免第二套分层数组
c = conv_subset(a, a);
```

复杂度

- $O(k^2 \cdot 2^k)$ 时间、$O(k \cdot 2^k)$ 空间。

10.2 多项式基础

10.2.1 Poly（多项式与形式幂级数）

```
// ===== 模平方根：仅 Poly<Z>::sqrt(n) 自动开方依赖 =====
// 若不需要自动 FPS 平方根，可删除本函数与 sqrt(n)
optional<Z> mod_sqrt(Z a) {
    constexpr int P{ Z::mod() };
    if (a == Z(0)) return Z(0);
    if (a.pow((P - 1) / 2) != Z(1)) return nullopt;
    if (P % 4 == 3) return a.pow((P + 1) / 4);
    int s{ countr_zero(unsigned(P - 1)) };
    i64 q{ (P - 1) >> s };
    Z z{ 2 };
    while (z.pow((P - 1) / 2) == Z(1)) z += Z(1);
    Z c{ z.pow(q) }, x{ a.pow((q + 1) / 2) }, t{ a.pow(q) };
    int m{ s };
    while (t != Z(1)) {
        int i{ 1 };
        Z tt{ t * t };
        for (; i < m && tt != Z(1); ++ i) tt *= tt;
        assert(i < m);
        Z b{ c.pow(1LL << (m - i - 1)) };
        x *= b;
        c = b * b;
        t *= c;
        m = i;
    }
    if ((-x).val() < x.val()) x = -x;
    return x;
}
template<class T = Z>
struct Poly {
    vector<T> a;
    Poly() = default;
    explicit Poly(int n) : a(n) {}
    explicit Poly(vector<T> a_) : a(move(a_)) {}
    Poly(initializer_list<T> a_) : a(a_) {}
    // ===== 基础容器：几乎所有功能依赖，不建议删除 =====
    int size() const { return int(a.size()); }
    bool empty() const { return a.empty(); }
    T operator[](int i) const { return 0 <= i && i < size() ? a[i] : T{}; }
    T &operator[](int i) { return a[i]; }
    void resize(int n) { a.resize(n); }
    // 删除最高次连续零项；divmod() 依赖
    void norm() {
        while (!a.empty() && a.back() == T{}) a.pop_back();
    }
    // 截断为恰好 n 项；不足补零，只复制实际需要的前缀
    Poly trunc(int n) const {
        assert(n >= 0);
        Poly f(n);
        copy_n(a.begin(), min(n, size()), f.a.begin());
        return f;
```

```
    }
    // 乘 x^k; k < 0 时删除前 -k 项
    // sqrt() / pow() / mulT() 依赖
    Poly shift(int k) const {
        if (k >= 0) {
            Poly f(size() + k);
            copy(a.begin(), a.end(), f.a.begin() + k);
            return f;
        }
        int d{ -k };
        if (d >= size()) return Poly();
        return Poly(vector<T>(a.begin() + d, a.end()));
    }
    // 反转系数; 指定 n 时先截断 / 补零到 n 项再反转
    // divmod() / mulT() 依赖
    Poly rev(int n = -1) const {
        Poly f{ n < 0 ? *this : trunc(n) };
        ranges::reverse(f.a);
        return f;
    }
    // 单点求值; 完全独立
    T eval(T x) const {
        T res{};
        for (int i = size() - 1; i >= 0; --i) res = res * x + a[i];
        return res;
    }
    // ===== 加减: 多数 FPS 与多项式除法依赖 =====
    Poly &operator+=(const Poly &g) {
        if (g.size() > size()) a.resize(g.size());
        for (int i = 0; i < g.size(); ++i) a[i] += g.a[i];
        return *this;
    }
    Poly &operator-=(const Poly &g) {
        if (g.size() > size()) a.resize(g.size());
        for (int i = 0; i < g.size(); ++i) a[i] -= g.a[i];
        return *this;
    }
    friend Poly operator+(Poly f, const Poly &g) { return f += g; }
    friend Poly operator-(Poly f, const Poly &g) { return f -= g; }
    friend Poly operator-(Poly f) {
        for (T &x : f.a) x = -x;
        return f;
    }
    // ===== 标量乘除: sqrt() / pow() 等依赖 =====
    Poly &operator*=(const T &k) {
        for (T &x : a) x *= k;
        return *this;
    }
    Poly &operator/=(const T &k) {
        for (T &x : a) x /= k;
        return *this;
    }
    friend Poly operator*(Poly f, const T &k) { return f *= k; }
    friend Poly operator*(const T &k, Poly f) { return f *= k; }
    friend Poly operator/(Poly f, const T &k) { return f /= k; }
    // ===== 多项式乘法: 所有快速 FPS / divmod / mulT 依赖 =====
    friend Poly operator*(const Poly &f, const Poly &g) {
        return Poly(::conv(f.a, g.a));
    }
    Poly &operator*=(const Poly &g) { return *this = *this * g; }
    // ===== FPS 逆: log / sqrt / divmod 依赖 =====
    Poly inv(int n) const {
        assert(n >= 0);
        if (n == 0) return {};
        assert(!empty() && a[0] != T{});
        Poly x{ T(1) / a[0] };
        for (int m = 1; m < n; m <<= 1) {
            int k{ min(m << 1, n) };
            x = (x * (Poly{ T(2) } - trunc(k) * x)).trunc(k);
        }
        return x.trunc(n);
```

```
    }
    // ===== FPS 平方根：若不需要 sqrt，可删除这两个成员与上方 mod_sqrt =====
    // 已知常数项平方根 s0；要求 a[0] != 0
    Poly sqrt(int n, T s0) const {
        assert(n >= 0);
        if (n == 0) return {};
        assert(!empty());
        assert(a[0] != T{} && s0 != T{} && s0 * s0 == a[0]);
        Poly x{ s0 };
        T inv2{ T(1) / T(2) };
        for (int m = 1; m < n; m <<= 1) {
            int k{ min(m << 1, n) };
            x = ((x + (trunc(k) * x.inv(k)).trunc(k)) * inv2).trunc(k);
        }
        return x.trunc(n);
    }
    // Z 专用：支持前导零，自动求最低非零项的模平方根
    Poly sqrt(int n) const {
        static_assert(is_same_v<T, Z>);
        assert(n >= 0);
        if (n == 0) return {};
        int t{ 0 };
        while (t < min(size(), n) && a[t] == T{}) ++ t;
        if (t == min(size(), n)) return Poly(n);
        assert(t % 2 == 0);
        int s{ t >> 1 }, m{ n - 2 * s };
        Poly h{ shift(-2 * s).trunc(m) };
        auto s0{ mod_sqrt(h.a[0]) };
        assert(s0.has_value());
        return h.sqrt(m, *s0).shift(s).trunc(n);
    }
    // ===== 多项式除法：不需要 / % 时可整体删除 divmod 与两个运算符 =====
    // 依赖 norm / rev / inv / 多项式乘法 / 加减
    friend pair<Poly, Poly> divmod(Poly f, Poly g) {
        f.norm();
        g.norm();
        assert(!g.empty());
        if (f.size() < g.size()) return { Poly(), f };
        int n{ f.size() }, m{ g.size() }, k{ n - m + 1 };
        Poly rf{ f.rev() };
        rf.resize(k);
        Poly q{ (rf * g.rev().inv(k)).trunc(k).rev() };
        Poly r{ f - q * g };
        r.resize(m - 1);
        q.norm();
        r.norm();
        return { q, r };
    }
    friend Poly operator/(Poly f, const Poly &g) {
        return divmod(move(f), g).first;
    }
    friend Poly operator%(Poly f, const Poly &g) {
        return divmod(move(f), g).second;
    }
    // ===== 微积分与 log / exp / pow =====
    // deriv / integr: log 依赖
    // log: inv 依赖
    // exp: log 依赖
    // pow: log + exp + shift + 标量运算依赖
    Poly deriv() const {
        if (size() <= 1) return Poly();
        Poly f(size() - 1);
        for (int i = 1; i < size(); ++ i) f.a[i - 1] = a[i] * T(i);
        return f;
    }
    Poly integr() const {
        Poly f(size() + 1);
        for (int i = 0; i < size(); ++ i) f.a[i + 1] = a[i] / T(i + 1);
        return f;
    }
    Poly log(int n) const {
```

```
        assert(n >= 0);
        if (n == 0) return {};
        assert(!empty() && a[0] == T(1));
        return (deriv() * inv(n)).integr().trunc(n);
    }
    Poly exp(int n) const {
        assert(n >= 0);
        if (n == 0) return {};
        assert(empty() || a[0] == T{});
        Poly x{ T(1) };
        for (int m = 1; m < n; m <<= 1) {
            int k{ min(m << 1, n) };
            x = (x * (Poly{ T(1) } - x.log(k) + trunc(k))).trunc(k);
        }
        return x.trunc(n);
    }
    Poly pow(i64 k, int n) const {
        assert(k >= 0 && n >= 0);
        if (n == 0) return {};
        if (k == 0) {
            Poly f(n);
            f.a[0] = T(1);
            return f;
        }
        int t{ 0 };
        while (t < min(size(), n) && a[t] == T{}) ++ t;
        if (t == min(size(), n) || (t && k > (n - 1LL) / t)) return Poly(n);
        T c{ a[t] };
        int m{ int(n - 1LL * t * k) };
        Poly f{ shift(-t).trunc(m) / c };
        f = (f.log(m) * T(k)).exp(m);
        T ck{ 1 }, base{ c };
        for (i64 e{ k }; e; e >>= 1, base *= base) {
            if (e & 1) ck *= base;
        }
        return (f * ck).shift(int(1LL * t * k)).trunc(n);
    }
    // ===== 转置乘法：主要供多点求值 / 插值等高级模块使用 =====
    // 仅依赖 rev / shift / 多项式乘法
    Poly mulT(const Poly &g) const {
        if (g.empty()) return Poly();
        return ((*this) * g.rev()).shift(1 - g.size());
    }
};
```

接口说明

- `Poly<T>`：系数数组 `a`，低次到高次，`size()/empty()/resize()`；`f[i]` 常量版本越界补零，非 const 版本必须在范围内。
- `trunc(n)`：前 `n` 项，不足补零；`shift(k)`：乘 x^k（负数表示砍掉低次项）；`rev(n)`：截断/补零到 `n` 项再翻转（不传参数则整段翻转）。
- `norm()`：移除最高次零项；`eval(x)`：Horner 单点求值。
- `+ - *`：多项式运算；`* /` 标量；`divmod(f,g)` 得商余，`f/g`、`f%g` 为普通多项式除法。
- `inv(n)`：模 x^n 乘法逆；`sqrt(n,s0)`：常数项已知平方根；`sqrt(n)`：仅 `Z` 自动 Tonelli–Shanks 并处理前导零。
- `deriv()` / `integr()`：形式求导 / 积分；`log(n)`、`exp(n)`、`pow(k,n)`：均返回恰好 `n` 项。
- `mulT(g)`：转置乘法 `f * reverse(g)` 后删除前 `|g|-1` 项，主要用于多点求值。

注意事项

- `inv(n)` 要求 `f[0]≠0`；`log(n)` 要求 `f[0]=1`；`exp(n)` 要求 `f[0]=0`；`pow(k,n)` 只接受非负整数 `k`。
- `sqrt(n,s0)` 要求 `f[0]=s0²≠0` 且 `2` 可逆；自动 `sqrt(n)` 要求平方根存在，前导零个数为偶数，选择模意义下较小的常数项根。
- 形式积分除以 `i+1`，使用模数 `998244353` 时必须确保需要的所有分母可逆，常见条件 `n < 998244353`。
- `Poly<i64>` 的基础加减乘可用，但FPS 求逆、除法、积分、对数等必须在合适的域上使用；整数截断除法不满足要求。
- 普通多项式的 `/` 和 `%` 与 FPS `inv()` 是不同操作；普通除法要求 `g` 非零且最高项可逆。
- 多项式采用两种零的表示：空数组与固定长度全零。`norm()` 会去掉所有最高零；FPS 截断通常保留固定长度。
- 只需基础运算可以删除 `mod_sqrt`、两种 `sqrt`、`divmod`、`deriv/integr/log/exp/pow`、`mulT`；保留所需接口依赖链。

常见用法

```
Poly<Z> f{1, 2, 3}, g{1, 1};
auto sum = f + g;
auto mul = f * g;
```

```
auto val = f.eval(Z(2));
auto pre = f.trunc(5);              // {1,2,3,0,0}
auto inv = g.inv(5);               // (1+x)^(-1) mod x^5
auto lg = g.log(5);                // log(1+x)
auto ex = Poly<Z>{0, 1}.exp(5);  // exp(x)
auto power = g.pow(1000000007LL, 5);
auto [q, r] = divmod(f, g);
```

求平方根时有两个版本：

```
Poly<Z> sq{1, 2, 1};
auto a = sq.sqrt(3, Z(1)); // 手动给常数项根，少一次模 sqrt
// auto b = sq.sqrt(3);    // 自动选择一个根（仅 Z）
```

应用中最容易遗漏的前导零处理（求 f^k）：

```
Poly<Z> f{0, 0, 2, 3};
auto g = f.pow(3, 10); // 前 6 项为 0，常数系数移位由 pow 自动处理
```

复杂度

- `trunc/shift/rev/norm/deriv/integr/eval`：$O(n)$；`+/-`：$O(n)$；多项式乘法：$O(M(n))$。
- `inv/sqrt/log/exp/pow/divmod`：$O(M(n))$ 量级（常数不同）；`mulT`：$O(M(n))$；$M(n)=O(n\log n)$ 为 NTT 卷积。

10.3 形式幂级数

10.3.1 FPS 三角与反三角函数

> 依赖：`10.2.1 Poly`、静态 `Z=MInt<998244353>`。

```
// 仅用于 mod=998244353; i²=-1, f[0]=0
namespace FPSFunc {
    Z imag() { static const Z I = Z(3).pow((Z::mod() - 1) / 4); return I; }
    Poly<Z> sin(Poly<Z> f, int n) {
        Z I = imag();
        auto a = (f * I).exp(n), b = (f * -I).exp(n);
        return (a - b).trunc(n) / (Z(2) * I);
    }
    Poly<Z> cos(Poly<Z> f, int n) {
        Z I = imag();
        auto a = (f * I).exp(n), b = (f * -I).exp(n);
        return (a + b).trunc(n) / Z(2);
    }
    Poly<Z> atan(Poly<Z> f, int n) {
        return (f.deriv() * (Poly<Z>{1} + f * f).inv(n)).integr().trunc(n);
    }
    Poly<Z> asin(Poly<Z> f, int n) {
        return (f.deriv() * (Poly<Z>{1} - f * f).sqrt(n, Z(1)).inv(n)).integr().trunc(n);
    }
}
```

接口说明

- `FPSFunc::sin(f,n)` / `cos(f,n)`：分别计算形式三角函数。
- `FPSFunc::atan(f,n)` / `asin(f,n)`：计算常数项为零的形式反三角函数，归一化为常数项零。

注意事项

- 仅适用默认 `998244353`（存在 $i^2=-1$）、`n<mod` 且 `f[0]=0`；此时使用形式指数、积分定义。
- `atan`、`asin` 不含一般数值分支选择；不是实数角度计算器。
- `asin` 中取常数项为 `1` 的平方根分支。

常见用法

```
Poly<Z> x{0, 1};
auto s = FPSFunc::sin(x, 8);
auto c = FPSFunc::cos(x, 8);
auto t = FPSFunc::atan(x, 8);
auto arcs = FPSFunc::asin(x, 8);
```

复杂度

- 四种函数：$O(M(n))$ 量级时间、$O(n)$ 空间，三角函数含两次 `exp`。

10.4 多项式求值与插值

10.4.1 单点求值与连续点 Lagrange 插值

> 依赖：静态模整数或一般域上的 `T`，无需 NTT。

```
// y[i]=f(i), i=0..n-1; deg(f) < n
// 查询任意一个 x 的 f(x), O(n)
template<class T>
T lagrange(const vector<T> &y, T x) {
    int n = y.size();
    if (n == 0) return T{};
    for (int i = 0; i < n; ++ i) if (x == T(i)) return y[i];
    vector<T> fac(n), ifac(n), pre(n + 1), suf(n + 1);
    fac[0] = 1;
    for (int i = 1; i < n; ++ i) fac[i] = fac[i - 1] * T(i);
    ifac[n - 1] = T(1) / fac[n - 1];
    for (int i = n - 1; i >= 1; -- i) ifac[i - 1] = ifac[i] * T(i);
    pre[0] = suf[n] = 1;
    for (int i = 0; i < n; ++ i) pre[i + 1] = pre[i] * (x - T(i));
    for (int i = n - 1; i >= 0; -- i) suf[i] = suf[i + 1] * (x - T(i));
    T ans{};
    for (int i = 0; i < n; ++ i) {
        T cur = y[i] * pre[i] * suf[i + 1] * ifac[i] * ifac[n - 1 - i];
        ans += (n - 1 - i) & 1 ? -cur : cur;
    }
    return ans;
}
```

接口说明

- `f.eval(x)`：Poly 的 Horner 法，已知系数求一个点，$O(degf)$。
- `lagrange(y,x)`：已知 `f(0)..f(n-1)` 求 `f(x)`，不必先恢复系数，$O(n)$ 时间与空间。

注意事项

- 必须已知 `deg f < y.size()`，否则只有前 n 个采样值不足以确定多项式。
- 模数需使 `1..n-1` 可逆；`998244353` 下通常 `n≤998244353`（实际范围远小于此）。
- 若许多点重复查询，阶乘/逆阶乘可预处理在外部，避免反复 $O(n)$ 初始化。

常见用法

```
vector<Z> y{1, 4, 9, 16};   // f(x)=(x+1)^2
Z ans = lagrange(y, Z(100)); // 10201
Poly<Z> f{1, 2, 1};
Z val = f.eval(Z(100));
```

复杂度

- `eval` $O(n)$ 时间、$O(1)$ 额外空间；`lagrange` $O(n)$ 时间与空间。

10.4.2 Newton 差商插值

> 依赖：仅要求域上的 `+ - * /`，不依赖 `Poly`。

```
// 任意互异 x[i] 对应 y[i]，返回 Newton 基系数 c
// f(t)=c[0]+c[1](t-x[0])+...+c[n-1]prod_{i<n-1}(t-x[i])
template<class T>
vector<T> newton_coeff(const vector<T> &x, vector<T> y) {
    int n = x.size();
    assert(n == int(y.size()));
    for (int j = 1; j < n; ++ j) {
        for (int i = n - 1; i >= j; -- i) {
            y[i] = (y[i] - y[i - 1]) / (x[i] - x[i - j]);
```

```
        }
    }
    return y;
}
template<class T>
T newton_eval(const vector<T> &x, const vector<T> &c, T t) {
    int n = x.size();
    assert(n == int(c.size()));
    T r{};
    for (int i = n - 1; i >= 0; --i) r = r * (t - x[i]) + c[i];
    return r;
}
```

接口说明

- `newton_coeff(x,y)`：返回差商 `c`，即 Newton 基展开系数。
- `newton_eval(x,c,t)`：不转换成普通幂基，直接 $O(n)$ 求值。

注意事项

- 所有采样横坐标 `x[i]` 必须两两不同，且 `x[i]-x[j]` 可逆。
- 返回的系数不是标准幂基 $1, x, x^2, \dots$ 的系数！若需要常规 `Poly`，应显式做基底转换。

常见用法

```
vector<Z> x{2, 5, 7}, y{9, 36, 64}; // (x+1)^2
auto c = newton_coeff(x, y);
Z ans = newton_eval(x, c, Z(10)); // 121
```

复杂度

- 求差商 $O(n^2)$ 时间、$O(n)$ 额外空间；给定差商后单点求值 $O(n)$。

10.4.3 多点求值（子积树）

> 依赖：`10.2.1 Poly` 的 `divmod`、`*`、`eval`；底层 NTT。

```
struct ProductTree {
    int n;
    vector<Z> x;
    vector<Poly<Z>> p;
    explicit ProductTree(vector<Z> x_) : n(x_.size()), x(move(x_)), p(4 * max(1, n)) {
        if (n) build(1, 0, n);
    }
    void build(int u, int l, int r) {
        if (r - l == 1) {
            p[u] = Poly<Z>{-x[l], 1};
            return;
        }
        int m = (l + r) / 2;
        build(u * 2, l, m);
        build(u * 2 + 1, m, r);
        p[u] = p[u * 2] * p[u * 2 + 1];
    }
    void dfs_eval(int u, int l, int r, Poly<Z> f, vector<Z> &ans) const {
        if (r - l == 1) {
            ans[l] = f.eval(x[l]);
            return;
        }
        int m = (l + r) / 2;
        dfs_eval(u * 2, l, m, f % p[u * 2], ans);
        dfs_eval(u * 2 + 1, m, r, f % p[u * 2 + 1], ans);
    }
    vector<Z> eval(Poly<Z> f) const {
        vector<Z> ans(n);
        if (n) {
            if (f.size() >= p[1].size()) f = f % p[1];
            dfs_eval(1, 0, n, move(f), ans);
        }
```

```
        return ans;
    }
};
```

接口说明

- `ProductTree pts(x)`：按待查询点建立子积树，其中 `p[1]=Π(X-x[i])`。
- `pts.eval(f)`：计算每个 `f(x[i])`，按传入顺序返回；支持重复点。

注意事项

- 每个子树保存一个乘积多项式，复用 Poly 的普通除法 `%`。
- `x` 可以重复，用于多点求值没问题；但用于后续插值时必须互异。
- 对固定一组 `x` 多次求值，`ProductTree` 只建一次可复用，避免重复构造。

常见用法

```
vector<Z> xs{0, 1, 2, 3};
ProductTree pts(xs);
Poly<Z> f{1, 2, 3};
auto ys = pts.eval(f); // {1,6,17,34}
```

复杂度

- `n` 个点、$\deg f < n$：建树 $O(M(n)\log n)$；求值 $O(M(n)\log n)$，保存子积树 $O(n\log n)$ 空间。

10.4.4 快速多点插值（子积树）

> 依赖：必须先复制 `10.4.3 ProductTree`，以及 `10.2.1 Poly`。

```
// ===== 选配：只做多点求值时可删除整个函数 =====
// x_i 必须两两不同，pts 从 10.4.3 构造
Poly<Z> interpolate(const ProductTree &pts, const vector<Z> &y) {
    int n = pts.n;
    assert(int(y.size()) == n);
    if (!n) return {};
    auto d = pts.eval(pts.p[1].deriv());
    vector<Z> w(n);
    for (int i = 0; i < n; ++ i) {
        assert(d[i] != Z{});
        w[i] = y[i] / d[i];
    }
    auto dfs = [&](auto &&self, int u, int l, int r) -> Poly<Z> {
        if (r - l == 1) return Poly<Z>{w[l]};
        int m = (l + r) / 2;
        auto a = self(self, u * 2, l, m);
        auto b = self(self, u * 2 + 1, m, r);
        return a * pts.p[u * 2 + 1] + b * pts.p[u * 2];
    };
    auto f = dfs(dfs, 1, 0, n);
    f.norm();
    return f;
}
```

接口说明

- `interpolate(pts,y)`：根据 `pts.x[i], y[i]` 恢复唯一的次数 `< n` 的多项式 `Poly<Z>`，已复用相同子积树。

注意事项

- 点 `x[i]` 必须两两不同，否则 `P'(x_i)=0` 且插值不唯一。
- `ProductTree` 与插值应使用同一种系数域；本实现默认 `Z`。
- `n=0` 返回空多项式；恢复结果用 `norm()` 去最高零项。

常见用法

```
vector<Z> x{0, 1, 2}, y{1, 6, 17};
ProductTree pts(x);
```

```
auto f = interpolate(pts, y); // 1+2x+3x²
auto check = pts.eval(f);     // 等于 y
```

复杂度

- $O(M(n)\log n)$ 时间，已建树时额外 $O(n\log n)$ 空间（递归中间多项式）。

10.5 生成函数

10.5.1 普通 / 指数生成函数（OGF / EGF）

> 依赖：全局 `conv()`；若执行 FPS 则依赖 `Poly<Z>`。

```
// 用系数存储生成函数：a[i] 表示 x^i 项系数
// OGF: A(x) = Σ a[n]x^n
// EGF: E(x) = Σ a[n] x^n/n!，实际 Poly 系数为 a[n]/n!
vector<Z> to_egf(const vector<Z> &a) {
    int n = a.size();
    if (!n) return {};
    vector<Z> b(n), invfac(n);
    Z fac = 1;
    for (int i = 1; i < n; ++ i) fac *= i;
    invfac[n - 1] = fac.inv();
    for (int i = n - 1; i >= 1; -- i) invfac[i - 1] = invfac[i] * Z(i);
    for (int i = 0; i < n; ++ i) b[i] = a[i] * invfac[i];
    return b;
}
```

接口说明

- OGF：`A(x)=Σ a_n x^n`，适合无标号/顺序结构的和卷积。
- EGF：`A(x)=Σ a_n x^n/n!`，适合标号组合对象。
- 两个 EGF 相乘得到 `c_n=Σ binom(n,k)a_k b_{n-k}`。
- `to_egf(a)`：将原始计数序列转成 EGF 的系数数组。

注意事项

- EGF 必须使用阶乘归一化，直接对原始计数调用 `conv` 得到的是 OGF 卷积。
- 模 `998244353` 下要求所需阶乘可逆，即系数长度不超过模数。

常见用法

```
vector<Z> a{1, 1, 1, 1};
Poly<Z> A(to_egf(a)); // Σ x^i/i!
auto B = A * A;       // 两个标号结构组合
// 恢复原始计数：b[i] = B[i] * i!
```

复杂度

- 阶乘转换 $O(n)$；OGF/EGF 乘法复用 $O(M(n))$ 的多项式卷积。

10.5.2 二项式变换与经典生成函数

> 依赖：`10.1.1 NTT` 中 `conv`，`Z`。

```
// b[n]=Σ_{k=0}^n C(n,k) a[k]
// inverse=true: a[n]=Σ (-1)^(n-k) C(n,k) b[k]
vector<Z> binomial_transform(const vector<Z> &a, bool inverse = false) {
    int n = a.size();
    if (!n) return {};
    vector<Z> fac(n), ifac(n), f(n), g(n);
    fac[0] = 1;
    for (int i = 1; i < n; ++ i) fac[i] = fac[i - 1] * Z(i);
    ifac[n - 1] = fac[n - 1].inv();
    for (int i = n - 1; i >= 1; -- i) ifac[i - 1] = ifac[i] * Z(i);
    for (int i = 0; i < n; ++ i) {
        f[i] = a[i] * ifac[i];
        g[i] = (inverse && (i & 1)) ? -ifac[i] : ifac[i];
    }
    auto h = conv(f, g);
    h.resize(n);
```

```
    for (int i = 0; i < n; ++ i) h[i] *= fac[i];
    return h;
}
```

接口说明

- `binomial_transform(a,false)`：二项式正变换。
- `binomial_transform(a,true)`：二项式逆变换。
- 常见 OGF：`1/(1-x)` 全 1；$(1-x)^{-k}$ 系数 `C(n+k-1,k-1)`；$-\log(1-x)$ 系数 $1/n$ $(n \geq 1)$。
- 常见 EGF：`exp(x)` 对应全 1；`exp(exp(x)-1)` 对应 Bell 数。

注意事项

- 本变换使用阶乘逆元，仅在需要的 `i!` 可逆时有效。
- 公式里的 `a_n` 是原始系数，不是已经除以阶乘的 EGF 系数。

常见用法

```
vector<Z> a{1, 2, 3, 4};
auto b = binomial_transform(a);
auto recovered = binomial_transform(b, true); // recovered == a
```

复杂度

- 单次正/逆变换 $O(M(n))$ 时间、$O(n)$ 空间。

10.5.3 常系数递推与有理生成函数

> 依赖：`Poly<Z>`；可配合 Bostan–Mori。

```
// s[n] = Σ_{j=1}^k c[j-1] * s[n-j] (n≥k)
// Q(x)=1-c[0]x-...-c[k-1]x^k
// P(x)=(Σ_{i=0}^{k-1}s[i]x^i)*Q(x) mod x^k
pair<Poly<Z>, Poly<Z>> recurrence_gf(const vector<Z> &s,
                                     const vector<Z> &c) {
    int k = c.size();
    assert(int(s.size()) ≥ k && k > 0);
    Poly<Z> q(k + 1);
    q[0] = 1;
    for (int i = 0; i < k; ++ i) q[i + 1] = -c[i];
    Poly<Z> p = (Poly<Z>(vector<Z>(s.begin(), s.begin() + k)) * q).trunc(k);
    p.norm();
    return {p, q};
}
```

接口说明

- `recurrence_gf(initial,c)`：返回 `P(x),Q(x)`，满足 `Σ s_nx^n=P/Q`。
- 求前 `n` 项：`(P * Q.inv(n)).trunc(n)`。
- 求巨大下标的单项：传递给 `10.6.3 Bostan-Mori`，不必展开全部系数。

注意事项

- 递推只能从 `n≥k` 起使用，`s[0..k-1]` 是初始化，不允许遗漏 `P(x)` 中的修正项。
- 若 `c[k-1]=0` 可减小递推阶数，提高效率。

常见用法

```
vector<Z> s{0, 1}, c{1, 1};
auto [p, q] = recurrence_gf(s, c); // Fibonacci: x/(1-x-x²)
auto first = (p * q.inv(10)).trunc(10);
```

复杂度

- 建立分式 $O(M(k))$；输出前 n 项 $O(M(n))$。

10.5.4 Lagrange 反演（系数形式）

> 依赖：`10.2.1 Poly::pow`。

```
// F(x)=x * phi(F(x)), phi(0)≠0
// [x^n]F(x)=(1/n)[t^(n-1)]phi(t)^n, n≥1
Z lagrange_inverse_coeff(const Poly<Z> &phi, int n) {
    assert(n >= 1 && !phi.empty() && phi[0] != Z(0));
    return phi.pow(n, n)[n - 1] / Z(n);
}
```

接口说明

- `lagrange_inverse_coeff(phi,n)`：已知 `F=x*phi(F)`，求 `F` 的第 `n` 项，不必显式构造 F。
- 更一般地：`[x^n]H(F(x))=(1/n)[t^(n-1)]H'(t)phi(t)^n`（`n≥1`）。

注意事项

- 要求 `phi(0)≠0`（标准形式）、`n` 在模数下可逆；`n=0` 单独取 `H(0)`。
- 此接口使用 Poly 的 `pow`，只能在幂级数相关假设成立时调用；需要很多相邻项时，改用 FPS 复合逆/牛顿法更适合。

常见用法

```
// F=x(1+F)^2: Catalan 型生成函数 (F[1]=1,F[2]=2,F[3]=5)
Poly<Z> phi{1, 2, 1};
Z c3 = lagrange_inverse_coeff(phi, 3); // 5
```

复杂度

- 单个系数 $O(M(n))$ 量级；需要 `n` 次除法，约定 `1≤n<mod`。

10.6 线性递推

10.6.1 Berlekamp–Massey（最短线性递推）

> 依赖：系数域 `T` 支持 `+ - * /`。

```
// 给定序列 s，返回 c，使 s[n]=Σ_{i=1}^k c[i-1]s[n-i]
template<class T>
vector<T> berlekamp_massey(const vector<T> &s) {
    vector<T> c{1}, b{1};
    int l = 0, m = 1;
    T last = 1;
    for (int n = 0; n < int(s.size()); ++ n) {
        T d{};
        for (int i = 0; i <= l; ++ i) d += c[i] * s[n - i];
        if (d == T{}) {
            ++ m;
            continue;
        }
        auto old = c;
        T w = d / last;
        if (c.size() < b.size() + m) c.resize(b.size() + m);
        for (int i = 0; i < int(b.size()); ++ i) c[i + m] -= w * b[i];
        if (2 * l <= n) {
            l = n + 1 - l;
            b = move(old);
            last = d;
            m = 1;
        } else ++ m;
    }
    c.resize(l + 1);
    vector<T> rec(l);
    for (int i = 0; i < l; ++ i) rec[i] = -c[i + 1];
    return rec;
}
```

接口说明

- `berlekamp_massey(s)`：返回与已给序列一致的**最短阶数**线性递推系数 `c[0..k-1]`。
- `c.empty()` 表示在观测序列上得到零阶递推（如全零）。

注意事项

- 适用域上可逆非零差值：对质数模的 `Z` 合适，`i64` 不是通用的域。

- 给定有限项只能得到拟合递推；想恢复真实的最小递推，通常需足够多项（常见为真实阶数的两倍）并由题意保证递推存在。
- 输出系数按最近前项在前排列：`s[n]=c[0]*s[n-1]+...+c[k-1]*s[n-k]`。

常见用法

```
vector<Z> fib{0, 1, 1, 2, 3, 5, 8, 13};
auto c = berlekamp_massey(fib); // {1,1}
```

复杂度

- $O(n^2)$ 时间、$O(n)$ 空间。

10.6.2 Kitamasa（线性递推第 n 项）

> 依赖：无需其它多项式板子。

```
// initial[0..k-1], c[0..k-1]: s[n]=Σ c[j]s[n-1-j]
template<class T>
T kitamasa(const vector<T> &initial, const vector<T> &c, i64 n) {
    int k = c.size();
    assert(k > 0 && int(initial.size()) >= k && n >= 0);
    if (n < k) return initial[n];
    auto mul = [&](const vector<T> &a, const vector<T> &b) {
        vector<T> t(2 * k - 1);
        for (int i = 0; i < k; ++ i) {
            for (int j = 0; j < k; ++ j) t[i + j] += a[i] * b[j];
        }
        for (int i = 2 * k - 2; i >= k; -- i) {
            for (int j = 1; j <= k; ++ j) t[i - j] += t[i] * c[j - 1];
        }
        t.resize(k);
        return t;
    };
    vector<T> r(k), x(k);
    r[0] = 1;
    if (k == 1) x[0] = c[0];
    else x[1] = 1;
    for (; n; n >>= 1, x = mul(x, x)) {
        if (n & 1) r = mul(r, x);
    }
    T ans{};
    for (int i = 0; i < k; ++ i) ans += r[i] * initial[i];
    return ans;
}
```

接口说明

- `kitamasa(initial,c,n)`：已知 k 阶递推前 k 项，计算零基下标 `s[n]`。
- 本版不依赖 NTT、`Poly`，适合阶数较小、目标下标极大的递推。

注意事项

- 输入 `c` 与 BM 输出的顺序一致：`c[0]` 乘 `s[n-1]`。
- 允许 `k=1`；对一般环也成立（不需要求逆），但整型可能溢出。
- 当 k 很大时 $O(k^2 \log n)$ 可能慢，改用 `Bostan-Mori` 加 NTT。

常见用法

```
vector<Z> init{0, 1}, c{1, 1};
Z f100 = kitamasa(init, c, 100);
```

复杂度

- $O(k^2 \log n)$ 时间，$O(k)$ 额外空间。

10.6.3 Bostan–Mori（有理生成函数第 n 项）

> 依赖：`10.2.1 Poly`；`recurrence_gf` 可选。

```
// [x^n] P(x)/Q(x), 要求 Q[0]≠0
Z bostan_mori(Poly<Z> p, Poly<Z> q, i64 n) {
```

```
    assert(n >= 0 && !q.empty() && q[0] != Z(0));
    p.norm(); q.norm();
    Z whole{};
    if (p.size() >= q.size()) {
        auto [div, rem] = divmod(p, q);
        whole = n < div.size() ? div[int(n)] : Z(0);
        p = move(rem);
    }
    while (n > 0 && !p.empty()) {
        Poly<Z> neg = q;
        for (int i = 1; i < neg.size(); i += 2) neg[i] = -neg[i];
        auto a = p * neg, b = q * neg;
        Poly<Z> np, nq;
        for (int i = n & 1; i < a.size(); i += 2) np.a.push_back(a[i]);
        for (int i = 0; i < b.size(); i += 2) nq.a.push_back(b[i]);
        p = move(np); q = move(nq);
        n >>= 1;
    }
    return whole + (p.empty() ? Z(0) : p[0] / q[0]);
}
```

接口说明

- `bostan_mori(p,q,n)`：求 `[x^n]p(x)/q(x)`，`n` 可达 10^18。
- 若 `deg(p)≥deg(q)`，先做多项式长除，再对真分式做 Bostan–Mori。

注意事项

- `q[0]≠0`；系数域是 `Z`，所有除法有模逆。
- 需要递推求第 n 项时，先使用 `10.5.3 recurrence_gf` 将递推转成分式。
- `p=0` 返回零；`n=0` 返回常数系数 `p[0]/q[0]`（含可能存在的整式部分）。

常见用法

```
vector<Z> init{0, 1}, c{1, 1};
auto [p, q] = recurrence_gf(init, c);
Z fib = bostan_mori(p, q, 10000000000000LL);
```

复杂度

- 令 `d=max(deg P,deg Q)`，$O(M(d)\log n)$ 时间、$O(d)$ 额外空间，含初始长除。

10.6.4 最小多项式与递推选型

> 依赖：`10.6.1 BM`，`10.6.2 Kitamasa`，可选 `10.6.3 Bostan-Mori`。

```
// 已有足够初值、并已证明序列具有线性递推时使用
template<class T>
T kth_from_samples(const vector<T> &s, i64 n) {
    assert(n >= 0);
    if (n < int(s.size())) return s[n];
    auto rec = berlekamp_massey(s);
    if (rec.empty()) return T{};
    return kitamasa(vector<T>(s.begin(), s.begin() + rec.size()), rec, n);
}
```

接口说明

- BM 输出 `c` 的对应特征多项式 `Q(t)=t^k-c[0]t^(k-1)-...-c[k-1]`。
- 生成函数的分母 `D(x)=1-c[0]x-...-c[k-1]x^k`；注意两个变量方向相反。
- 矩阵或线性算子的最小多项式求值可化成最小递推；随机向量 Krylov 序列能在概率意义上恢复部分或全部最小多项式，但这里不提供未经验证的随机模板。

注意事项

- BM 短序列拟合不等于证明序列存在对应递推。
- 先确定实际递推阶数，再选择 Kitamasa（小 k）或 Bostan–Mori（较大 k）。

常见用法

```
vector<Z> s{0,1,1,2,3,5,8,13,21,34};
auto c = berlekamp_massey(s);
Z ans = kitamasa(vector<Z>(s.begin(), s.begin() + c.size()), c, 1000000);
```

复杂度

- BM $O(n^2)$；第 n 项 Kitamasa $O(k^2 \log n)$，Bostan–Mori $O(M(k) \log n)$。

10.7 多项式高级技巧

10.7.1 Taylor Shift（计算 f(x+c)）

> 依赖：全局 `conv`（NTT），`Poly<Z>`。

```
// 计算 f(x+c)，只返回 f.size() 项，模 998244353
Poly<Z> taylor_shift(const Poly<Z> &f, Z c) {
    int n = f.size();
    if (!n) return {};
    vector<Z> fac(n), ifac(n), a(n), b(n);
    fac[0] = 1;
    for (int i = 1; i < n; ++ i) fac[i] = fac[i - 1] * Z(i);
    ifac[n - 1] = fac[n - 1].inv();
    for (int i = n - 1; i >= 1; -- i) ifac[i - 1] = ifac[i] * Z(i);
    Z pw = 1;
    for (int i = 0; i < n; ++ i) {
        a[n - 1 - i] = f[i] * fac[i];
        b[i] = pw * ifac[i];
        pw *= c;
    }
    auto h = conv(a, b);
    Poly<Z> g(n);
    for (int i = 0; i < n; ++ i) g[i] = h[n - 1 - i] * ifac[i];
    return g;
}
```

接口说明

- `taylor_shift(f,c)`：返回普通幂基系数表示的 `f(x+c)`，不修改次数。

注意事项

- `n=f.size()` 必须使 `0!..(n-1)!` 可逆；模 `998244353` 下 `n≤mod`。
- 该版本自动保留尾部零系数，不调用 `norm()`；若需要最简多项式，可在返回后调用。

常见用法

```
Poly<Z> f{1, 2, 3}; // 1+2x+3x²
auto g = taylor_shift(f, Z(5)); // 86+32x+3x²
```

复杂度

- $O(M(n))$ 时间、$O(n)$ 空间。

10.7.2 连续点值平移（复用多点插值）

> 依赖：`10.4.3 ProductTree` + `10.4.4 interpolate`。

```
// 给定 y[i]=f(i) (deg f<n)；求 f(c),f(c+1),...,f(c+m-1)
vector<Z> shift_samples(const vector<Z> &y, Z c, int m) {
    assert(m >= 0);
    if (m == 0) return {};
    int n = y.size();
    if (n == 0) return vector<Z>(m);
    vector<Z> x(n), target(m);
    for (int i = 0; i < n; ++ i) x[i] = i;
    for (int j = 0; j < m; ++ j) target[j] = c + Z(j);
    ProductTree old(x), cur(target);
    auto f = interpolate(old, y);
    return cur.eval(f);
}
```

接口说明

- `shift_samples(y,c,m)`：由连续采样 `f(0)..f(n-1)` 求连续采样 `f(c)..f(c+m-1)`。
- 本版复用子积树，不另外维护容易在取到旧采样点时除零的专门平移公式。

注意事项

- 要求 $\deg f < n$，`0..n-1` 在模域中两两不同；通常 `n<mod`。
- 本实现并非理论最快的连续点值平移特化算法，但复用经过测试的插值与多点求值模块，适合少抄板子的赛场需求。
- 目标点允许和原采样点重复，输出依然正确。

常见用法

```
vector<Z> y{1, 4, 9, 16}; // (x+1)^2
auto shifted = shift_samples(y, Z(10), 3); // {121,144,169}
```

复杂度

- 约 $O(M(n+m)\log(n+m))$ 时间、$O((n+m)\log(n+m))$ 空间。

10.7.3 Chirp-Z（等比点求值）

> 依赖：全局 `conv` + `Poly<Z>`。

```
// 返回 f(q^j), j=0..m-1; q=0 时约定 q^0=1
vector<Z> chirp_z(const Poly<Z> &f, Z q, int m) {
    assert(m >= 0);
    int n = f.size();
    vector<Z> ans(m);
    if (m == 0 || n == 0) return ans;
    if (q == Z(0)) {
        ans[0] = f.eval(Z(1));
        for (int j = 1; j < m; ++ j) ans[j] = f[0];
        return ans;
    }
    vector<Z> a(n), b(n + m - 1);
    Z cur = 1, step = 1;
    for (int i = 0; i < n; ++ i) {
        a[i] = f[i] * cur;
        cur *= step;
        step *= q;
    }
    Z iq = q.inv();
    int start = 1 - n;
    cur = iq.pow(1LL * start * (start + 1) / 2);
    step = start + 1 < 0 ? q.pow(-start - 1) : iq.pow(start + 1);
    for (int k = start; k < m; ++ k) {
        b[k + n - 1] = cur;
        cur *= step;
        step *= iq;
    }
    auto c = conv(a, b);
    cur = 1; step = q;
    for (int j = 0; j < m; ++ j) {
        ans[j] = c[n - 1 + j] * cur;
        cur *= step;
        step *= q;
    }
    return ans;
}
```

接口说明

- `chirp_z(f,q,m)`：返回 `f(1),f(q),f(q²),...,f(q^(m-1))`。
- 非零 `q` 利用平方差分解，把等比点求值化成一次卷积。

注意事项

- 处理 `q=0`、`n=0`、`m=0`；`q=0` 的 `j=0` 位置按 `q^0=1`。
- `q` 为非零但阶很小时，查询点可能重复，求值仍正确；但如果要做插值需点互异。
- 用 `i64` 计算平方型指数，`n,m` 的规模不得使 `i64` 指数溢出（常见算法竞赛规模下无问题）。

常见用法

```
Poly<Z> f{1, 2, 3};
auto vals = chirp_z(f, Z(2), 4); // f(1),f(2),f(4),f(8)
```

复杂度

- $O(M(n+m))$ 主导时间（另有少量 $O(\log(n+m))$ 快速幂），$O(n+m)$ 空间。

10.7.4 下降阶乘 / 上升阶乘（多项式）

> 依赖：`Poly<Z>` 与 `conv`。

```
// falling: x(x-1)...(x-n+1)
// rising:  x(x+1)...(x+n-1)
Poly<Z> factorial_poly(int n, bool rising = false) {
    assert(n >= 0);
    auto dfs = [&](auto &&self, int l, int r) -> Poly<Z> {
        if (r - l == 1) return {rising ? Z(l) : Z(-l), Z(1)};
        int m = (l + r) / 2;
        return self(self, l, m) * self(self, m, r);
    };
    return n ? dfs(dfs, 0, n) : Poly<Z>{1};
}
```

接口说明

- `factorial_poly(n,false)`：下降阶乘 $x^{\underline{n}}$。
- `factorial_poly(n,true)`：上升阶乘 $x^{\overline{n}}$。
- 系数采用通常的 x^i 形式，便于直接卷积、Taylor Shift 等。

注意事项

- `n=0` 两者都等于 `1`；`n<mod` 不是本实现的数学必需条件（但规模必须可计算）。
- 下降阶乘在非负整数 `0..n-1` 处为零，可用于构造插值与组合恒等式。

常见用法

```
auto f = factorial_poly(3);       // x³-3x²+2x
auto g = factorial_poly(3, true); // x³+3x²+2x
```

复杂度

- $O(M(n)\log n)$ 时间、$O(n\log n)$ 递归临时系数总占用上界；可回收内存实际较低。

10.7.5 FPS 复合与复合逆（短版，适合中小 n）

> 依赖：`10.2.1 Poly`。

```
// ===== 选配：朴素复合，避免引入复杂的大规模复合算法 =====
// f(g(x)) mod x^n, 要求 g[0]=0
Poly<Z> compose(const Poly<Z> &f, const Poly<Z> &g, int n) {
    assert(n >= 0 && (g.empty() || g[0] == Z(0)));
    if (!n) return {};
    Poly<Z> ans(n);
    for (int i = min(f.size(), n) - 1; i >= 0; -- i) {
        ans = (ans * g).trunc(n);
        ans[0] += f[i];
    }
    return ans;
}
// F(G(x))=x mod x^n; F[0]=0, F[1]≠0
Poly<Z> compositional_inverse(const Poly<Z> &f, int n) {
    assert(n >= 0 && f[0] == Z(0) && f[1] != Z(0));
    if (!n) return {};
    Poly<Z> g(n);
    if (n == 1) return g;
    g[1] = Z(1) / f[1];
    for (int m = 2; m < n; m <<= 1) {
        int k = min(2 * m, n);
        auto err = compose(f, g.trunc(k), k);
```

```
        err[1] -= 1;
        auto d = compose(f.deriv(), g.trunc(k), k);
        g = (g.trunc(k) - (err * d.inv(k)).trunc(k)).trunc(n);
    }
    return g;
}
```

接口说明

- `compose(f,g,n)`：形式幂级数复合 `f(g(x)) mod x^n`，Horner 实现。
- `compositional_inverse(f,n)`：复合逆 `g`，使 `f(g(x))=x mod x^n`，使用 Newton 纠错。

注意事项

- 这份实现专门保留较短、容易核对的版本：适合 `n` 不大；大规模 $n \approx 10^5$ 不应使用这里的 `compose`，需要 Brent–Kung 或更高级的专用复合算法。
- 复合默认 `g[0]=0`，复合逆要求 `f[0]=0`、`f[1]` 可逆。`f` 的前两项必须存在，否则不满足调用前提。
- 复合逆过程中只关心模 x^n 的前缀，勿把普通乘法逆 `inv()` 当作复合逆。

常见用法

```
Poly<Z> f{0, 1, 1}; // x+x²
auto g = compositional_inverse(f, 10);
auto id = compose(f, g, 10); // {0,1,0,...}
```

复杂度

- `compose`：$O(nM(n))$ 时间；复合逆约 $O(nM(n))$ 量级、$O(n)$ 临时空间。

11. 线性代数、数值计算与数值类型

11.1 线性方程组

求解 $Ax = b$

实数 Gauss–Jordan（特解、秩、零空间）

```
struct RealLinearSolution {
    bool consistent;
    int rank;
    vector<ld> particular;
    vector<vector<ld>> nullspace;
};
RealLinearSolution gauss_real(vector<vector<ld>> a, int m, ld eps = 1E-12L) {
    int n = a.size();
    for (const auto &v : a) assert(int(v.size()) == m + 1);
    vector<int> where(m, -1);
    int r = 0;
    for (int col = 0; col < m && r < n; ++ col) {
        int p = r;
        for (int i = r + 1; i < n; ++ i) {
            if (abs(a[i][col]) > abs(a[p][col])) p = i;
        }
        if (abs(a[p][col]) <= eps) continue;
        swap(a[p], a[r]);
        where[col] = r;
        ld inv = 1 / a[r][col];
        for (int j = col; j <= m; ++ j) a[r][j] *= inv;
        for (int i = 0; i < n; ++ i) if (i != r) {
            ld t = a[i][col];
            if (abs(t) <= eps) continue;
            for (int j = col; j <= m; ++ j) a[i][j] -= t * a[r][j];
        }
        ++ r;
    }
    for (int i = 0; i < n; ++ i) {
```

```
        bool zero = true;
        for (int j = 0; j < m; ++ j) zero &= abs(a[i][j]) <= eps;
        if (zero && abs(a[i][m]) > eps) return {false, r, {}, {}};
    }
    vector<ld> x(m);
    for (int j = 0; j < m; ++ j) {
        if (where[j] != -1) x[j] = a[where[j]][m];
    }
    vector<vector<ld>> basis;
    for (int f = 0; f < m; ++ f) if (where[f] == -1) {
        vector<ld> v(m);
        v[f] = 1;
        for (int j = 0; j < m; ++ j) {
            if (where[j] != -1) v[j] = -a[where[j]][f];
        }
        basis.push_back(move(v));
    }
    return {true, r, move(x), move(basis)};
}
// 非空矩阵的便利接口；0 个方程时请显式传 m
RealLinearSolution gauss_real(vector<vector<ld>> a, ld eps = 1E-12L) {
    assert(!a.empty() && !a[0].empty());
    int m = int(a[0].size()) - 1;
    return gauss_real(move(a), m, eps);
}
```

素数模 Gauss–Jordan

```
i64 linear_pow_mod(i64 a, i64 b, int mod) {
    i64 res = 1;
    for (; b; b >>= 1, a = a * a % mod) {
        if (b & 1) res = res * a % mod;
    }
    return res;
}
struct ModLinearSolution {
    bool consistent;
    int rank;
    vector<int> particular;
    vector<vector<int>> nullspace;
};
ModLinearSolution gauss_mod_prime(vector<vector<int>> a, int m, int mod) {
    assert(mod >= 2); // 必须为质数
    int n = a.size();
    for (auto &v : a) {
        assert(int(v.size()) == m + 1);
        for (int &x : v) x = (x % mod + (i64)mod) % mod;
    }
    vector<int> where(m, -1);
    int r = 0;
    for (int col = 0; col < m && r < n; ++ col) {
        int p = r;
        while (p < n && a[p][col] == 0) ++ p;
        if (p == n) continue;
        swap(a[p], a[r]);
        where[col] = r;
        i64 inv = linear_pow_mod(a[r][col], mod - 2, mod);
        for (int j = col; j <= m; ++ j) a[r][j] = a[r][j] * inv % mod;
        for (int i = 0; i < n; ++ i) if (i != r && a[i][col]) {
            i64 t = a[i][col];
            for (int j = col; j <= m; ++ j) {
                a[i][j] = (a[i][j] - t * a[r][j]) % mod;
                if (a[i][j] < 0) a[i][j] += mod;
            }
        }
        ++ r;
    }
    for (int i = r; i < n; ++ i) {
```

```
        if (a[i][m]) return {false, r, {}, {}};
    }
    vector<int> x(m);
    for (int j = 0; j < m; ++ j) {
        if (where[j] != -1) x[j] = a[where[j]][m];
    }
    vector<vector<int>> basis;
    for (int f = 0; f < m; ++ f) if (where[f] == -1) {
        vector<int> v(m);
        v[f] = 1;
        for (int j = 0; j < m; ++ j) if (where[j] != -1) {
            int t = a[where[j]][f];
            v[j] = t ? mod - t : 0;
        }
        basis.push_back(move(v));
    }
    return {true, r, move(x), move(basis)};
}
// 与原接口兼容；非空矩阵时从增广矩阵推断变量数。
ModLinearSolution gauss_mod_prime(vector<vector<int>> a, int mod) {
    assert(!a.empty() && !a[0].empty());
    int m = int(a[0].size()) - 1;
    return gauss_mod_prime(move(a), m, mod);
}
```

GF(2) / 异或方程组（`bitset`）

每行 `bit [0,m)` 为系数，`bit m` 为常数项；额外位必须为零。

```
template<size_t M>
struct XorLinearSolution {
    bool consistent;
    int rank;
    bitset<M> particular;
    vector<bitset<M>> nullspace;
};
template<size_t M>
XorLinearSolution<M> gauss_xor(vector<bitset<M>> a, int m) {
    assert(0 <= m && m < int(M));
    int n = a.size(), r = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && r < n; ++ col) {
        int p = r;
        while (p < n && !a[p][col]) ++ p;
        if (p == n) continue;
        swap(a[p], a[r]);
        where[col] = r;
        for (int i = 0; i < n; ++ i) {
            if (i != r && a[i][col]) a[i] ^= a[r];
        }
        ++ r;
    }
    for (int i = r; i < n; ++ i) {
        if (a[i][m]) return {false, r, {}, {}};
    }
    bitset<M> x;
    for (int j = 0; j < m; ++ j) {
        if (where[j] != -1) x[j] = a[where[j]][m];
    }
    vector<bitset<M>> basis;
    for (int f = 0; f < m; ++ f) if (where[f] == -1) {
        bitset<M> v;
        v[f] = 1;
        for (int j = 0; j < m; ++ j) {
            if (where[j] != -1) v[j] = a[where[j]][f];
        }
        basis.push_back(v);
    }
```

```
    return {true, r, x, move(basis)};
}
```

三对角线性方程组（Thomas / 追赶法）

```
optional<vector<ld>> tridiagonal(vector<ld> lower,
        vector<ld> diagonal, vector<ld> upper, vector<ld> rhs,
        ld eps = 1E-18L) {
    int n = diagonal.size();
    assert(int(rhs.size()) == n);
    assert(int(lower.size()) == max(0, n - 1));
    assert(int(upper.size()) == max(0, n - 1));
    if (!n) return vector<ld>{};
    for (int i = 1; i < n; ++ i) {
        if (abs(diagonal[i - 1]) <= eps) return nullopt;
        ld t = lower[i - 1] / diagonal[i - 1];
        diagonal[i] -= t * upper[i - 1];
        rhs[i] -= t * rhs[i - 1];
    }
    if (abs(diagonal.back()) <= eps) return nullopt;
    vector<ld> x(n);
    x.back() = rhs.back() / diagonal.back();
    for (int i = n - 2; i >= 0; -- i) {
        x[i] = (rhs[i] - upper[i] * x[i + 1]) / diagonal[i];
    }
    return x;
}
```

接口说明

四种算法均为 `0base`

接口	输入	返回
`gauss_real(a,m,eps)`	`n × (m+1)` 实数增广矩阵	`consistent, rank, particular, nullspace`
`gauss_mod_prime(a,m,mod)`	`n × (m+1)` 模质数矩阵	同上，数值在 `[0,mod)`
`gauss_xor<M>(a,m)`	`bitset<M>` 行，`M≥m+1`	同上，特解和零空间基为 `bitset<M>`
`tridiagonal(lower,diagonal,upper,rhs,eps)`	下/主/上对角与右端	`optional<vector<ld>>`

- 若方程组有解，`particular` 是**自由变量设为零**时的一组特解，`nullspace` 有 `m-rank` 个向量，全部解为 $x = x_0 + \sum_i c_i v_i$。
- 无解时 `consistent=false`，只保证 `rank` 有效；其他结果为空。
- 素数模域 $\mathbf{F}_p$ 中，若有解则解数为 p^{m-r}；GF(2) 中为 2^{m-r}。
- Thomas 数组：`lower[i]=A[i+1][i]`，`diagonal[i]=A[i][i]`，`upper[i]=A[i][i+1]`，`rhs[i]=b[i]`。
- `gauss_real(a,eps)` / `gauss_mod_prime(a,mod)` 是方便旧代码接入的非空矩阵重载；零方程、`m>0` 时一定使用带 `m` 的主接口。

注意事项

- **浮点**：实数版采用列部分选主元，但使用绝对 `eps` 判断；系数数量级差异很大时需重新调整阈值，最好用原始 A, b 检查残差。近奇异的秩与零空间基不保证高精度。
- **素数模**：必须验证题目的 `mod` 是质数；合数模的非零元素不保证有逆元。整数模的中间乘法在 `mod ≤ INT_MAX` 下由 `i64` 承担。
- **GF(2)**：`M` 是编译期容量（包含常数项），`m` 是实际变量数；`m` 及以上的额外未使用 bit 不能含有数据。
- **Thomas**：没有行交换；`nullopt` 只表示本追赶法遇到过小主元，**不等价于原系统无解**。若矩阵非严格对角占优，可能需要带选主元的带状消元。循环三对角不能直接使用。
- 只要 `rank(A)=rank([A|b])` 就有解；若又有 `rank(A)=m`，解唯一。零空间维数为 `m-r`。
- 若仅需秩或存在性，可删除三个 Gauss–Jordan 算法末尾构造 `particular/nullspace` 的代码，减少返回结果空间。
- 最小二乘和最小范数不由本节的"自由变量设为零"保证，使用 11.5 的 QR。

常见用法

实数解空间和唯一性

```
vector<vector<ld>> a{{1, 1, 1}}; // x+y=1
auto sol = gauss_real(a, 2);
if (!sol.consistent) cout << "No solution\n";
else if (sol.rank == 2) cout << "Unique\n";
```

```
else cout << "Multiple\n";
// particular={1,0}, nullspace[0]={-1,1}
```

素数模方程组

```
vector<vector<int>> a{{1, 1, 3}, {1, 2, 5}};
auto sol = gauss_mod_prime(a, 998244353);
// consistent=true, particular={1,2}
```

GF(2) 开关类问题

```
vector<bitset<101>> a(1);
a[0][0] = a[0][2] = a[0][4] = 1;
a[0][5] = 1; // x0 ^ x2 ^ x4 = 1, m=5
auto sol = gauss_xor(a, 5);
```

O(n) 解三对角系统

```
auto x = tridiagonal({-1, -1}, {2, 2, 2},
                     {-1, -1}, {1, 0, 1});
// 有解时 x={1,1,1}
```

复杂度

算法	时间	额外空间（含副本与输出）
实数 / 素数模消元	$O(nm\min(n,m)+m(m-r))$，模数版另有 $O(r\log p)$ 逆元	$O(nm+m(m-r))$
GF(2) 消元	约 $O(nm\lceil M/w\rceil+m(m-r)\lceil M/w\rceil)$ 字操作	$O((n+m-r)\lceil M/w\rceil)$ 字
Thomas	$O(n)$	$O(n)$

11.2 异或线性基

`T` 为无符号整数（默认 `u64`），`B` 是有效二进制位数。

普通异或线性基

```
template<class T = u64, int B = numeric_limits<T>::digits>
struct LinearBasis {
    static_assert(is_unsigned_v<T>);
    static_assert(1 <= B && B <= 64 && B <= numeric_limits<T>::digits);
    using value_type = T;
    array<T, B> p{};
    int siz{ 0 };
    // ===== 基础接口：插入、判定、最值与秩 =====
    bool insert(T x) {
        if (siz == B) return false;
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (!p[i]) {
                p[i] = x;
                ++siz;
                return true;
            }
            x ^= p[i];
        }
        return false;
    }
    bool contains(T x) const {
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (!p[i]) return false;
            x ^= p[i];
        }
        return true;
    }
    T max_xor(T x = 0) const {
```

```
        for (int i = B - 1; i >= 0; --i) if ((x ^ p[i]) > x) x ^= p[i];
        return x;
    }
    T min_xor(T x) const {
        for (int i = B - 1; i >= 0; --i) if ((x ^ p[i]) < x) x ^= p[i];
        return x;
    }
    T min_positive() const {
        for (int i = 0; i < B; ++i) if (p[i]) return p[i];
        return 0;
    }
    int rank() const { return siz; }
    void clear() {
        p.fill(0);
        siz = 0;
    }
    // ===== 规范化与第 k 小 =====
    vector<T> basis() const {
        auto q{ p };
        for (int i = 0; i < B; ++i) if (q[i]) {
            for (int j = i - 1; j >= 0; --j) {
                if (q[i] >> j & 1) q[i] ^= q[j];
            }
        }
        vector<T> res;
        res.reserve(siz);
        for (int i = 0; i < B; ++i) if (q[i]) res.push_back(q[i]);
        return res;
    }
    optional<T> kth(u64 k) const {
        if (siz < 64 && k >= (1ULL << siz)) return nullopt;
        auto a{ basis() };
        T res{};
        for (int i = 0; i < siz; ++i) if (k >> i & 1) res ^= a[i];
        return res;
    }
    // ===== 张成空间运算：合并 / 求交 =====
    void merge(const LinearBasis &o) {
        for (int i = B - 1; i >= 0; --i) if (o.p[i]) insert(o.p[i]);
    }
    LinearBasis intersection(const LinearBasis &o) const {
        // q 表示两个空间的并空间基，tag 表示 q 中对应向量的当前空间分量
        auto q{ p };
        auto tag{ p };
        LinearBasis res;
        for (int i = B - 1; i >= 0; --i) if (o.p[i]) {
            T x{ o.p[i] }, t{};
            bool independent{ false };
            for (int j = B - 1; j >= 0; --j) if (x >> j & 1) {
                if (!q[j]) {
                    q[j] = x;
                    tag[j] = t;
                    independent = true;
                    break;
                }
                x ^= q[j];
                t ^= tag[j];
            }
            // 出现并空间中的线性关系时，t 同时属于两个原空间
            if (!independent) res.insert(t);
        }
        return res;
    }
};
```

前缀区间线性基（静态区间 / 末尾追加）

查询区间均为 `[l,r)`。维护每个前缀的最大右端位置主元。

```cpp
template<class T = u64, int B = numeric_limits<T>::digits>
struct PrefixLinearBasis {
    static_assert(is_unsigned_v<T>);
    static_assert(1 <= B && B <= 64 && B <= numeric_limits<T>::digits);
    struct Version {
        array<T, B> value{};
        array<int, B> pos;
        Version() { pos.fill(-1); }
    };
    vector<Version> pre{ 1 };
    PrefixLinearBasis() = default;
    explicit PrefixLinearBasis(const vector<T> &a) {
        pre.reserve(a.size() + 1);
        for (T x : a) push(x);
    }
    int size() const { return int(pre.size()) - 1; }
    void push(T x) {
        int position{ size() };
        Version cur{ pre.back() };
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (!cur.value[i]) {
                cur.value[i] = x;
                cur.pos[i] = position;
                break;
            }
            // 同一主元优先保留位置更靠后的向量
            if (cur.pos[i] < position) {
                swap(cur.value[i], x);
                swap(cur.pos[i], position);
            }
            x ^= cur.value[i];
        }
        pre.push_back(cur);
    }
    int range_rank(int l, int r) const {
        const Version &v{ pre[r] };
        int res{};
        for (int i = 0; i < B; ++i) res += v.pos[i] >= l;
        return res;
    }
    bool range_contains(int l, int r, T x) const {
        const Version &v{ pre[r] };
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (v.pos[i] < l) return false;
            x ^= v.value[i];
        }
        return true;
    }
    T range_max(int l, int r, T x = 0) const {
        const Version &v{ pre[r] };
        for (int i = B - 1; i >= 0; --i) {
            if (v.pos[i] >= l && (x ^ v.value[i]) > x) x ^= v.value[i];
        }
        return x;
    }
    T range_min(int l, int r, T x) const {
        const Version &v{ pre[r] };
        for (int i = B - 1; i >= 0; --i) {
            if (v.pos[i] >= l && (x ^ v.value[i]) < x) x ^= v.value[i];
        }
        return x;
    }
};
```

线段树结点信息（单点修改、区间查询）

直接接入已有 `SegTree<Info>`

```
template<class Basis = LinearBasis<>>
struct BasisInfo {
    using T = typename Basis::value_type;
    Basis basis;
    BasisInfo() = default;
    explicit BasisInfo(T x) { basis.insert(x); }
    friend BasisInfo operator+(BasisInfo a, const BasisInfo &b) {
        a.basis.merge(b.basis);
        return a;
    }
};
```

可撤销异或线性基（线段树分治 / DFS 回溯）

一次插入只会新增一个主元，历史中记录主元位置，`-1` 表示本次秩没有变化。

```
template<class T = u64, int B = numeric_limits<T>::digits>
struct RollbackLinearBasis {
    static_assert(is_unsigned_v<T>);
    static_assert(1 <= B && B <= 64 && B <= numeric_limits<T>::digits);
    array<T, B> p{};
    int siz{ 0 };
    vector<int> hist;
    // ===== 修改与回滚 =====
    int snapshot() const { return int(hist.size()); }
    bool insert(T x) {
        if (siz == B) {
            hist.push_back(-1);
            return false;
        }
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (!p[i]) {
                p[i] = x;
                ++siz;
                hist.push_back(i);
                return true;
            }
            x ^= p[i];
        }
        hist.push_back(-1);
        return false;
    }
    void rollback(int snap) {
        while (int(hist.size()) > snap) {
            int i{ hist.back() };
            hist.pop_back();
            if (i != -1) {
                p[i] = 0;
                --siz;
            }
        }
    }
    void clear() {
        p.fill(0);
        siz = 0;
        hist.clear();
    }
    // ===== 基础查询 =====
    int rank() const { return siz; }
    bool contains(T x) const {
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (!p[i]) return false;
            x ^= p[i];
        }
        return true;
    }
    T max_xor(T x = 0) const {
        for (int i = B - 1; i >= 0; --i) if ((x ^ p[i]) > x) x ^= p[i];
        return x;
    }
```

```
    T min_xor(T x) const {
        for (int i = B - 1; i >= 0; --i) if ((x ^ p[i]) < x) x ^= p[i];
        return x;
    }
};
```

带来源的线性基（最多 64 个输入元素）

除了“是否可表示”，还可恢复一组实现指定 XOR 的下标集合，以及一组非空 XOR 为 0 的线性关系。这一块与普通 `LinearBasis` 独立，按需复制。

```
template<class T = u64, int B = numeric_limits<T>::digits>
struct WitnessLinearBasis {
    static_assert(is_unsigned_v<T>);
    static_assert(1 <= B && B <= 64 && B <= numeric_limits<T>::digits);
    array<T, B> p{};
    array<u64, B> from{};
    int n{0}, siz{0};
    // 成功插入返回 nullopt；相关时返回非空零异或下标掩码
    optional<u64> insert(T x) {
        assert(n < 64);
        u64 mask = 1ULL << n++;
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (!p[i]) {
                p[i] = x;
                from[i] = mask;
                ++siz;
                return nullopt;
            }
            x ^= p[i];
            mask ^= from[i];
        }
        return mask;
    }
    // nullopt = 不可表示；其他情况返回一组下标掩码（0 可由空集表示）
    optional<u64> represent(T x) const {
        u64 mask = 0;
        for (int i = B - 1; i >= 0; --i) if (x >> i & 1) {
            if (!p[i]) return nullopt;
            x ^= p[i];
            mask ^= from[i];
        }
        return mask;
    }
    int rank() const { return siz; }
};
```

接口说明

普通 `LinearBasis<T,B>`

- `insert(x)`：插入，若提升秩返回 `true`；$O(B)$。
- `contains(x)`：是否可由某个子集异或得到；$O(B)$。
- `max_xor(x=0)` / `min_xor(x)`：$x \oplus y$ 的最大 / 最小值，y 为任意可表示值；$O(B)$。
- `min_positive()`：最小正可表示值，秩为零时返回 `0`；$O(B)$。
- `rank()` / `clear()`：秩 $O(1)$、清空 $O(B)$。
- `basis()`：按最低到最高主元排序的规范基，不修改原基；$O(B^2)$。
- `kth(k)`：第 `k` 小不同可表示 XOR，`0base` 且包括 `0`；越界 `nullopt`；$O(B^2)$。
- `merge(other)`：张成空间的和 $V+W$，而非集合并去重；$O(B^2)$。
- `intersection(other)`：两个张成空间的交 $V \cap W$；$O(B^2)$。

区间、线段树与回滚

- `PrefixLinearBasis(a)` / `push(x)`：建全部前缀 $O(nB)$ / 追加一个 $O(B)$。
- `range_rank(l,r)`、`range_contains(l,r,x)`、`range_max(l,r,x=0)`、`range_min(l,r,x)`：$O(B)$，全部是半开区间。
- `BasisInfo<Basis>()` / `BasisInfo<Basis>(x)`：空单位元 / 单元素；`operator+` 用 `merge()` 合并，$O(B^2)$。
- `RollbackLinearBasis::insert(x)`：每次都记录历史（包括无效插入）；$O(B)$。
- `snapshot()` / `rollback(snap)`：快照 $O(1)$，恢复 O(撤销的插入次数)。

- `WitnessLinearBasis::insert(x)`：独立则 `nullopt`，依赖时返回一个非空零异或子集掩码；`represent(x)` 返回一组解的下标掩码，无法表示则 `nullopt`；均为 $O(B)$。

注意事项

- `B` 表示参与运算的位数，输入必须 `x < 2^B`（`B=64` 时自然覆盖 `u64`）；代码不会自动检测高位截断。达到满秩时普通 `insert()` 可跳过后续插入，但前缀版 `push()` 不可跳过，因为主元时间戳还会变化。
- 秩为 r 的异或空间有 2^r 个不同值。原始有 n 个带下标元素时，每个可达值恰好对应 2^{n-r} 个下标子集（包括空集，包含重复元素也成立）。
- `kth(0)=0`，秩为 64 时 `u64` 的全部取值都可作为合法 `k`；若问非空子集第 `k` 小，需考虑存在非空 XOR 为 0 的子集时 `0` 的情况，不能直接套不同值排序。
- `min_xor(0)=0` 恒成立；求最小正可表示值调用 `min_positive()`。若原始元素非空，则最小非空子集异或值在 `n>rank()` 时为 `0`，否则是 `min_positive()`。
- `PrefixLinearBasis` 空区间只能表示 `0`，`range_max(l,l,x)=range_min(l,l,x)=x`，`range_rank(l,l)=0`。
- `merge` 与 `intersection` 只处理张成空间，不会记录原始元素个数或重复次数；合并后的第 `k` 小无法直接还原按子集计数的排列。
- `RollbackLinearBasis` 不能在回滚历史之间手动规范化主元或直接修改 `p`，否则仅记录一个主元位置不足以复原。
- `WitnessLinearBasis` 的掩码第 `i` 位对应第 `i` 次 `insert()`，只支持至多 64 个输入元素；`represent(0)` 合法返回 `0`（空集），要获得非空零异或集合应查看 `insert()` 的依赖返回。
- 如果需要解有 RHS 的 GF(2) 方程组，使用 11.1 的 `gauss_xor()`；如果只关心 XOR 张成空间，则本节更短。

常见用法

最大子集异或与第 k 小

```
LinearBasis basis;
for (u64 x : a) basis.insert(x);
u64 mx = basis.max_xor();
auto kth = basis.kth(k); // 0base, 包含 0
```

区间查询和在线单点修改

```
PrefixLinearBasis pre(a);
u64 ans = pre.range_max(l, r); // [l,r)
// 有在线修改时可用 BasisInfo<> 作为 SegTree 的 Info 类型。
```

撤销 / 线段树分治

```
RollbackLinearBasis lb;
int snap = lb.snapshot();
lb.insert(x);
lb.insert(y);
auto ans = lb.max_xor();
lb.rollback(snap);
```

恢复表达方案和异或依赖

```
WitnessLinearBasis wb;
auto dep0 = wb.insert(5); // nullopt
wb.insert(3);
auto dep = wb.insert(6);  // 0b111: 5 ^ 3 ^ 6 = 0
auto mask = wb.represent(6); // 例如 0b011
```

图上的最大路径异或（无向图）

从根 DFS 得到树路径异或 `dis[v]`，每条非树边 `(u,v,w)` 插入 `dis[u] ^ dis[v] ^ w`，则从 `s` 到 `t` 的最大路径异或（允许重复经过边/点的游走）为：

```
u64 ans = basis.max_xor(dis[s] ^ dis[t]);
```

若题目限定简单路径，不能直接套这条结论。

复杂度

结构 / 操作	时间	空间
普通 / 可撤销插入和查询	$O(B)$	$O(B)$；可撤销另有 $O(q)$ 历史
规范化 / 第 k 小 / 合并 / 求交	$O(B^2)$	$O(B)$
前缀线性基建表	$O(nB)$	$O(nB)$
前缀区间查询	$O(B)$	$O(1)$

结构 / 操作	时间	空间
`BasisInfo` 线段树一次区间查询 / 单点修改	$O(B^2 \log n)$	整棵树 $O(nB)$
来源追踪的插入 / 表示	$O(B)$	$O(B)$

11.3 数值积分、方程与单纯形

方程求根（实根、二分、Newton）

```cpp
constexpr ld EPS = 1E-12L;
int sgn(ld x, ld eps = EPS) { return (x > eps) - (x < -eps); }
// 返回所有不同的实根（重根只返回一次）；恒等式 0=0 的情形无穷多根，返回空数组。
vector<ld> quadratic(ld a, ld b, ld c) {
    if (sgn(a) == 0) {
        if (sgn(b) == 0) return {};
        return {-c / b};
    }
    ld d = b * b - 4 * a * c;
    if (sgn(d) < 0) return {};
    if (sgn(d) == 0) return {-b / (2 * a)};
    ld root = sqrt(max((ld)0, d));
    ld q = -0.5L * (b + copysign(root, b));
    ld x = q / a, y = c / q;
    if (x > y) swap(x, y);
    return {x, y};
}
// 要求 [l,r] 两端异号或至少一端接近 0；只能保证找到一个根。
template<class F>
optional<ld> bisect_root(F f, ld l, ld r, ld eps = EPS,
        int iters = 120) {
    assert(l <= r);
    ld fl = f(l), fr = f(r);
    if (abs(fl) <= eps) return l;
    if (abs(fr) <= eps) return r;
    if ((fl < 0) == (fr < 0)) return nullopt;
    for (int it = 0; it < iters; ++ it) {
        ld m = midpoint(l, r), fm = f(m);
        if (abs(fm) <= eps || m == l || m == r) return m;
        if ((fl < 0) == (fm < 0)) l = m, fl = fm;
        else r = m;
    }
    return midpoint(l, r);
}
// 系数按最高次到常数项；给出 [l,r] 内全部数值可分辨的不同实根。
// 仅适用于低次数、系数尺度适中且不会在区间上发生严重数值消去的多项式。
vector<ld> polynomial_real_roots(vector<ld> a, ld l, ld r,
        ld eps = EPS) {
    assert(l <= r);
    while (a.size() > 1 && abs(a.front()) <= eps) a.erase(a.begin());
    int d = int(a.size()) - 1;
    if (d <= 0) return {};
    auto f = [&](ld x) {
        ld ans = 0;
        for (ld c : a) ans = ans * x + c;
        return ans;
    };
    if (d == 1) {
        ld x = -a[1] / a[0];
        return l - eps <= x && x <= r + eps ? vector<ld>{clamp(x, l, r)} : vector<ld>{};
    }
    vector<ld> der(d);
    for (int i = 0; i < d; ++ i) der[i] = a[i] * (d - i);
    vector<ld> critical = polynomial_real_roots(der, l, r, eps);
    vector<ld> points{l};
    for (ld x : critical) {
        if (x > l && x < r) points.push_back(x);
```

```
    }
    points.push_back(r);
    vector<ld> roots;
    auto add = [&](ld x) {
        if (roots.empty() || abs(roots.back() - x) > 16 * eps) {
            roots.push_back(clamp(x, l, r));
        }
    };
    for (int i = 0; i < int(points.size()); ++ i) {
        ld x = points[i], fx = f(x);
        if (abs(fx) <= eps) add(x); // 同时处理偶重根（导数根）
        if (i + 1 == int(points.size())) break;
        ld y = points[i + 1], fy = f(y);
        if ((fx < -eps && fy > eps) || (fx > eps && fy < -eps)) {
            auto root = bisect_root(f, x, y, eps);
            if (root) add(*root);
        }
    }
    return roots;
}
template<class F, class DF>
optional<ld> newton(F f, DF df, ld x, int iters = 100,
        ld eps = EPS) {
    for (int it = 0; it < iters; ++ it) {
        ld y = f(x), d = df(x);
        if (!isfinite(y) || !isfinite(d)) return nullopt;
        if (abs(y) <= eps) return x;
        if (abs(d) <= eps) return nullopt;
        ld nx = x - y / d;
        if (!isfinite(nx)) return nullopt;
        if (abs(nx - x) <= eps * max((ld)1, abs(nx))) {
            if (abs(f(nx)) <= eps) return nx;
        }
        x = nx;
    }
    return abs(f(x)) <= eps ? optional<ld>(x) : nullopt;
}
```

自适应 Simpson 积分（缓存端点 / 中点值）

```
template<class F>
ld adaptive_simpson_impl(F &f, ld l, ld r, ld fl, ld fm, ld fr, ld whole, ld eps, int depth) {
    ld m = midpoint(l, r);
    ld lm = midpoint(l, m), rm = midpoint(m, r);
    ld flm = f(lm), frm = f(rm);
    ld left = (fl + 4 * flm + fm) * (m - l) / 6;
    ld right = (fm + 4 * frm + fr) * (r - m) / 6;
    ld delta = left + right - whole;
    if (depth == 0 || m == l || m == r || abs(delta) <= 15 * eps) {
        return left + right + delta / 15;
    }
    return adaptive_simpson_impl(f, l, m, fl, flm, fm, left, eps / 2, depth - 1)
         + adaptive_simpson_impl(f, m, r, fm, frm, fr, right, eps / 2, depth - 1);
}
template<class F>
ld adaptive_simpson(F f, ld l, ld r, ld eps = 1E-10L, int max_depth = 25) {
    if (l > r) return -adaptive_simpson(f, r, l, eps, max_depth);
    if (l == r) return 0;
    ld fl = f(l), fr = f(r), fm = f(midpoint(l, r));
    ld whole = (fl + 4 * fm + fr) * (r - l) / 6;
    return adaptive_simpson_impl(f, l, r, fl, fm, fr, whole, eps, max_depth);
}
```

Romberg 积分（适合足够光滑的函数）

```cpp
template<class F>
ld romberg(F f, ld l, ld r, ld eps = 1E-10L, int max_level = 20) {
    assert(0 <= max_level && max_level <= 25);
    if (l > r) return -romberg(f, r, l, eps, max_level);
    if (l == r) return 0;
    vector<ld> prev{(f(l) + f(r)) * (r - l) / 2};
    ld trapezoid = prev[0];
    for (int level = 1; level <= max_level; ++ level) {
        int cnt = 1 << (level - 1);
        ld h = (r - l) / (1 << level), sum = 0;
        for (int i = 0; i < cnt; ++ i) sum += f(l + (2 * i + 1) * h);
        trapezoid = trapezoid / 2 + h * sum;
        vector<ld> cur(level + 1);
        cur[0] = trapezoid;
        ld p4 = 1;
        for (int j = 1; j <= level; ++ j) {
            p4 *= 4;
            cur[j] = cur[j - 1] + (cur[j - 1] - prev[j - 1]) / (p4 - 1);
        }
        if (abs(cur.back() - prev.back()) <= eps) return cur.back();
        prev = move(cur);
    }
    return prev.back();
}
```

两阶段单纯形（最大化标准型）

求 $\max c^T x$，满足 $Ax \le b,\ x \ge 0$。返回 `-INF` = 不可行，`INF` = 无界，有限数 = 最优值

```cpp
struct LinearProgram {
    static constexpr ld EPS = 1E-10L;
    static constexpr ld INF = numeric_limits<ld>::infinity();
    int n, m;
    vector<int> basic, nonbasic;
    vector<vector<ld>> t;

    LinearProgram(const vector<vector<ld>> &a, const vector<ld> &b,
            const vector<ld> &c)
        : n(b.size()), m(c.size()), basic(n), nonbasic(m + 1),
          t(n + 2, vector<ld>(m + 2)) {
        for (const auto &row : a) assert(int(row.size()) == m);
        assert(int(a.size()) == n);
        for (int i = 0; i < n; ++ i) {
            basic[i] = m + i;
            t[i][m] = -1;
            t[i][m + 1] = b[i];
            for (int j = 0; j < m; ++ j) t[i][j] = a[i][j];
        }
        for (int j = 0; j < m; ++ j) {
            nonbasic[j] = j;
            t[n][j] = -c[j];
        }
        nonbasic[m] = -1;
        t[n + 1][m] = 1;
    }
    // ===== 内部转轴与两阶段迭代 =====
    void pivot(int r, int s) {
        ld inv = 1 / t[r][s];
        for (int i = 0; i < n + 2; ++ i) if (i != r) {
            for (int j = 0; j < m + 2; ++ j) if (j != s) {
                t[i][j] -= t[r][j] * t[i][s] * inv;
            }
        }
        for (int j = 0; j < m + 2; ++ j) if (j != s) t[r][j] *= inv;
        for (int i = 0; i < n + 2; ++ i) if (i != r) t[i][s] *= -inv;
        t[r][s] = inv;
        swap(basic[r], nonbasic[s]);
    }
```

```
    bool simplex(int phase) {
        int obj = phase == 1 ? n + 1 : n;
        while (true) {
            int s = -1;
            for (int j = 0; j <= m; ++ j) {
                if (phase == 2 && nonbasic[j] == -1) continue;
                if (s == -1 || t[obj][j] < t[obj][s] - EPS
                    || (abs(t[obj][j] - t[obj][s]) <= EPS
                        && nonbasic[j] < nonbasic[s])) s = j;
            }
            if (s == -1 || t[obj][s] >= -EPS) return true;
            int r = -1;
            for (int i = 0; i < n; ++ i) {
                if (t[i][s] <= EPS) continue;
                if (r == -1) r = i;
                else {
                    ld left = t[i][m + 1] / t[i][s];
                    ld right = t[r][m + 1] / t[r][s];
                    if (left < right - EPS || (abs(left - right) <= EPS
                            && basic[i] < basic[r])) r = i;
                }
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }
    // 只调用一次 solve；需要新题 / 新约束应重新构造对象
    ld solve(vector<ld> &x) {
        x.clear();
        if (m == 0) {
            for (int i = 0; i < n; ++ i) if (t[i][1] < -EPS) return -INF;
            return 0;
        }
        int r = 0;
        for (int i = 1; i < n; ++ i) {
            if (t[i][m + 1] < t[r][m + 1]) r = i;
        }
        if (n && t[r][m + 1] < -EPS) {
            pivot(r, m);
            if (!simplex(1) || t[n + 1][m + 1] < -EPS || abs(t[n + 1][m + 1]) > EPS) return -INF;
            auto it = find(basic.begin(), basic.end(), -1);
            if (it != basic.end()) {
                r = it - basic.begin();
                int s = 0;
                for (int j = 1; j <= m; ++ j) {
                    if (abs(t[r][j]) > abs(t[r][s])) s = j;
                }
                if (abs(t[r][s]) > EPS) pivot(r, s);
            }
        }
        if (!simplex(2)) return INF;
        x.assign(m, 0);
        for (int i = 0; i < n; ++ i) {
            if (basic[i] < m && basic[i] >= 0) x[basic[i]] = t[i][m + 1];
        }
        return t[n][m + 1];
    }
};
```

Kahan 补偿求和

```
template<class Range>
ld kahan_sum(const Range &values) {
    ld sum = 0, c = 0;
    for (auto raw : values) {
        ld y = (ld)raw - c;
        ld t = sum + y;
        c = (t - sum) - y;
```

```
        sum = t;
    }
    return sum;
}
```

接口说明

接口	返回 / 约定	复杂度
`quadratic(a,b,c)`	二次 / 退化一次方程的不同实根（升序）	$O(1)$
`bisect_root(f,l,r,eps,iters)`	存在异号端点时返回一个根的近似值；否则 `nullopt`	$O(I)$ 次函数求值
`polynomial_real_roots(a,l,r,eps)`	系数降幂排列，返回区间内不同实根	最坏约 $O(d^3 I)$
`newton(f,df,x0,iters,eps)`	单点 Newton 迭代；失败 `nullopt`	$O(I)$
`adaptive_simpson(f,l,r,eps,max_depth)`	定积分近似	自适应，最多 $O(2^D)$ 求值
`romberg(f,l,r,eps,max_level)`	Romberg 积分近似	$O(2^K + K^2)$
`LinearProgram(a,b,c).solve(x)`	标准型 LP 的最优值及一组 `x`；±∞ 两类失败	每转轴 $O(nm)$
`kahan_sum(range)`	按容器迭代顺序的补偿求和	$O(N)$

`EPS` 是绝对误差阈值。多项式实根算法默认有限区间，且目标函数低次数、尺度适中；`l==r` 时只检查该端点

注意事项

- `quadratic()` 采用防消去误差公式；判别式 `b²-4ac` 仍可能发生舍入或溢出。对于 `a=b=0` 返回空数组，无法用该返回区分“无根”和“恒等式”。
- 多项式实根通过导数根分段寻找单调区间，并检查驻点来覆盖偶重根；接近的根可能因 `eps` 合并，接近零的函数值可能产生伪根，无法保证高次数和病态系数下全部正确。
- `bisect_root()` 不要求函数单调，但要求区间连续并有变号信息；切线重根通常没有变号。`newton()` 只保证局部迭代，没有全局收敛保证。
- 自适应 Simpson 的 `eps` 是整体绝对误差预算，已知的断点、奇点应先分段；达到深度上限不代表误差已满足要求。对窄峰与非光滑函数，采样可能漏掉贡献。
- Romberg 对足够光滑函数有效；`max_level=25` 已可能需要约 2^{25} 次函数求值，不要无约束调高。
- 单纯形只接受 $Ax \le b, x \ge 0$ 的最大化形式。自由变量拆为 $x^+ - x^-$；`=` 拆成两条 `≤`；`≥` 两侧同乘 `-1`；最小化用负目标函数转换。
- 单纯形采用浮点比较与防退化的编号平局规则，不提供精确整数最优值的保证；最坏转轴次数为指数级。有限返回值才可读取 `x`，调用对象应每组数据重建。
- `m=0` 的 LP 特判为空变量问题；`n=0` 且有变量时，目标存在正系数则无界，否则最优值为 `0`。
- Kahan 不能修复输入转换已丢失的精度，也不能消除 `NaN/Inf`。
- 单峰函数极值搜索复用 2.11 二分、三分与黄金分割 的 `golden_search`；0/1 分数规划可二分 λ 并判断 $\max \sum_i (a_i - \lambda b_i) \ge 0$（必须按原题选取约束下的方案，且分母为正）。

常见用法

求积分与全部低次数实根

```
auto integral = adaptive_simpson([](ld x) { return exp(-x * x); }, 0, 1);
auto roots = polynomial_real_roots({1, 0, -1, 0}, -2, 2);
// x^3-x, 在 [-2,2] 上的实根为 -1、0、1
```

线性规划

```
vector<vector<ld>> a{{1, 1}, {1, 0}, {0, 1}};
vector<ld> b{4, 2, 3}, c{3, 2}, x;
LinearProgram lp(a, b, c);
ld optimum = lp.solve(x);
if (optimum == -LinearProgram::INF) cout << "Infeasible\n";
else if (optimum == LinearProgram::INF) cout << "Unbounded\n";
else cout << optimum << '\n'; // 最优值 10, x={2,2}
```

复杂度

算法	时间 / 求值次数	空间
求根、Newton	见接口表，递归多项式求根约 $O(d^3 I)$	$O(d^2)$
自适应 Simpson	取决于采样与误差，最多指数于 `max_depth`	$O(D)$ 调用栈
Romberg	$O(2^K + K^2)$	$O(K)$
单纯形	每转轴 $O(nm)$，转轴次数最坏指数级	$O(nm)$
Kahan	$O(N)$	$O(1)$

11.4 Kirchhoff 矩阵树定理

依赖 11.5 的 `determinant_mod_prime(a, mod)`，它返回模质数行列式。这样全章只维护一套行列式实现；如果要求合数模，替换为该节的 `determinant_mod_any(a, mod)`。

```
struct MatrixTree {
    int n, mod;
    vector<vector<int>> lap;
    MatrixTree(int n_, int mod_) : n(n_), mod(mod_), lap(n_, vector<int>(n_)) {
        assert(n >= 0 && mod >= 2); // mod 必须为质数
    }
private:
    void add(int i, int j, i64 w) {
        lap[i][j] = (lap[i][j] + w % mod + (i64)mod) % mod;
    }
public:
    // ===== 三种边模型：按计数目标选择其中一种调用，不混用 =====
    void add_undirected(int u, int v, i64 w = 1) {
        if (u == v) return;
        add(u, u, w);
        add(v, v, w);
        add(u, v, -(w % mod));
        add(v, u, -(w % mod));
    }
    // 有向入树：每个非根点沿边走到 root（out-degree Laplacian）
    void add_toward_root(int u, int v, i64 w = 1) {
        if (u == v) return;
        add(u, u, w);
        add(u, v, -(w % mod));
    }
    // 有向出树：从 root 沿边可达所有点（in-degree Laplacian）
    void add_away_from_root(int u, int v, i64 w = 1) {
        if (u == v) return;
        add(v, v, w);
        add(v, u, -(w % mod));
    }
    // ===== 余子式行列式：计数接口 =====
    int count(int root = 0) const {
        if (n == 0) return 0;
        assert(0 <= root && root < n);
        vector a(n - 1, vector<int>(n - 1));
        for (int i = 0, x = 0; i < n; ++ i) if (i != root) {
            for (int j = 0, y = 0; j < n; ++ j) if (j != root) {
                a[x][y ++] = lap[i][j];
            }
            ++ x;
        }
        return determinant_mod_prime(move(a), mod);
    }
};
```

接口说明

- `MatrixTree(n, mod)`：创建 n 个点（`0base`）的拉普拉斯矩阵，`mod` 为质数，$O(n^2)$ 空间。
- `add_undirected(u,v,w=1)`：无向边，两端度数加权，非对角元减权，$O(1)$。
- `add_toward_root(u,v,w=1)`：插入有向边 `u → v`，用于计数**朝根有向生成树**，$O(1)$。
- `add_away_from_root(u,v,w=1)`：插入有向边 `u → v`，用于计数**从根出发的有向生成树**，$O(1)$。
- `count(root=0)`：删去根的行与列，计算主余子式，$O(n^3 + n \log mod)$；无向图中任意根结果相同。
  三类 `add` 都可以添加权重 w，结果为所有合法树的**边权乘积之和**。无权图使用默认权重 `1`。

注意事项

- `mod` **必须是质数**，除非自行将依赖改为 11.5 的任意模数行列式实现。不能对合数模数直接用费马小定理求逆。
- 无向重边可重复添加，权重可为负数；自环对生成树没有贡献，直接忽略。
- 有向生成树的方向非常容易写反：`toward_root` 是所有非根节点**最终指向根**，`away_from_root` 是从根**能到达所有节点**。一棵合法树恰有 `n-1` 条边。
- 未连通无向图和无法形成对应方向树的有向图返回 `0`。但模数下返回 `0` 也可能仅表示真实计数是模数的倍数。
- `n=0` 约定返回 `0`；`n=1` 返回 `1`，因为空边集是一棵单点树（自环被忽略）。
- 按题目使用一种有向/无向建图方式；不要把同一条有向边同时用 `add_toward_root` 与 `add_away_from_root` 建入一个计数器。

- 这里使用稠密矩阵，n 很大、图很稀疏时 $O(n^3)$ 通常不可承受。计数精确整数值可用多模 CRT 或大整数行列式，需证明模数乘积覆盖答案上界。

数学速查

- 无向图：$L = D - A$，$\tau(G) = \det L^{(r)}$，即删去任一行、同编号一列后的主子式。
- 带权图：$\tau_w(G) = \sum_{T\text{ spanning tree}} \prod_{e\in T} w_e$。
- 完全图：$\tau(K_n) = n^{n-2}$ $(n \geq 2)$；无向环：$\tau(C_n) = n$ $(n \geq 3)$。
- 有向图：以出度减邻接矩阵得到**朝根入树**计数；以内度构成的列拉普拉斯矩阵得到**离根出树**计数。均删除指定根的行和列。

常见用法

无向三角形：3 棵生成树

```
MatrixTree mt(3, 998244353);
mt.add_undirected(0, 1);
mt.add_undirected(1, 2);
mt.add_undirected(0, 2);
int ans = mt.count(); // 3
```

边权 / 重边

```
MatrixTree mt(2, 998244353);
mt.add_undirected(0, 1, 3);
mt.add_undirected(0, 1, 5);
int ans = mt.count(); // 8: 可选其中任意一条边
```

有向生成树

```
MatrixTree in(3, 998244353);
in.add_toward_root(0, 1);
in.add_toward_root(1, 2);
int a = in.count(2); // 1: 0 → 1 → 2

MatrixTree out(3, 998244353);
out.add_away_from_root(0, 1);
out.add_away_from_root(1, 2);
int b = out.count(0); // 1: 0 → 1 → 2
```

复杂度

操作	时间	额外空间
构造	$O(n^2)$	$O(n^2)$
插入一条边	$O(1)$	$O(1)$
一次 `count(root)`	$O(n^3 + n \log mod)$	$O(n^2)$

11.5 矩阵求逆、行列式、最小二乘与最小范数

实数矩阵求逆与行列式

```
optional<vector<vector<ld>>> inverse_real(vector<vector<ld>> a, ld eps = 1E-12L) {
    int n = a.size();
    for (const auto &row : a) assert(int(row.size()) == n);
    vector aug(n, vector<ld>(2 * n));
    for (int i = 0; i < n; ++ i) {
        for (int j = 0; j < n; ++ j) aug[i][j] = a[i][j];
        aug[i][n + i] = 1;
    }
    for (int k = 0; k < n; ++ k) {
        int p = k;
        for (int i = k + 1; i < n; ++ i) {
            if (abs(aug[i][k]) > abs(aug[p][k])) p = i;
        }
        if (abs(aug[p][k]) <= eps) return nullopt;
        swap(aug[p], aug[k]);
```

```
        ld inv = 1 / aug[k][k];
        for (int j = 0; j < 2 * n; ++ j) aug[k][j] *= inv;
        for (int i = 0; i < n; ++ i) if (i != k) {
            ld t = aug[i][k];
            if (t == 0) continue;
            for (int j = 0; j < 2 * n; ++ j) aug[i][j] -= t * aug[k][j];
        }
    }
    vector res(n, vector<ld>(n));
    for (int i = 0; i < n; ++ i) {
        for (int j = 0; j < n; ++ j) res[i][j] = aug[i][n + j];
    }
    return res;
}
ld determinant_real(vector<vector<ld>> a, ld eps = 1E-12L) {
    int n = a.size();
    for (const auto &row : a) assert(int(row.size()) == n);
    ld det = 1;
    for (int k = 0; k < n; ++ k) {
        int p = k;
        for (int i = k + 1; i < n; ++ i) {
            if (abs(a[i][k]) > abs(a[p][k])) p = i;
        }
        if (abs(a[p][k]) <= eps) return 0;
        if (p != k) swap(a[p], a[k]), det = -det;
        ld pivot = a[k][k];
        det *= pivot;
        for (int i = k + 1; i < n; ++ i) {
            ld t = a[i][k] / pivot;
            for (int j = k + 1; j < n; ++ j) a[i][j] -= t * a[k][j];
            a[i][k] = 0;
        }
    }
    return det;
}
```

素数模矩阵求逆与行列式

```
i64 matrix_pow_mod(i64 a, i64 b, int mod) {
    i64 res = 1;
    for (; b; b >>= 1, a = a * a % mod) {
        if (b & 1) res = res * a % mod;
    }
    return res;
}
optional<vector<vector<int>>> inverse_mod_prime(vector<vector<int>> a, int mod) {
    assert(mod >= 2); // 必须是质数
    int n = a.size();
    for (const auto &row : a) assert(int(row.size()) == n);
    vector aug(n, vector<int>(2 * n));
    for (int i = 0; i < n; ++ i) {
        for (int j = 0; j < n; ++ j) {
            aug[i][j] = (a[i][j] % mod + (i64)mod) % mod;
        }
        aug[i][n + i] = 1;
    }
    for (int k = 0; k < n; ++ k) {
        int p = k;
        while (p < n && !aug[p][k]) ++ p;
        if (p == n) return nullopt;
        swap(aug[p], aug[k]);
        i64 inv = matrix_pow_mod(aug[k][k], mod - 2, mod);
        for (int j = 0; j < 2 * n; ++ j) aug[k][j] = aug[k][j] * inv % mod;
        for (int i = 0; i < n; ++ i) if (i != k && aug[i][k]) {
            i64 t = aug[i][k];
            for (int j = 0; j < 2 * n; ++ j) {
                aug[i][j] = (aug[i][j] - t * aug[k][j]) % mod;
                if (aug[i][j] < 0) aug[i][j] += mod;
```

```
            }
        }
    }
    vector res(n, vector<int>(n));
    for (int i = 0; i < n; ++ i) {
        for (int j = 0; j < n; ++ j) res[i][j] = aug[i][n + j];
    }
    return res;
}
int determinant_mod_prime(vector<vector<int>> a, int mod) {
    assert(mod >= 2); // 必须是质数
    int n = a.size();
    for (auto &row : a) {
        assert(int(row.size()) == n);
        for (int &x : row) x = (x % mod + (i64)mod) % mod;
    }
    i64 det = 1;
    for (int k = 0; k < n; ++ k) {
        int p = k;
        while (p < n && !a[p][k]) ++ p;
        if (p == n) return 0;
        if (p != k) swap(a[p], a[k]), det = (mod - det) % mod;
        i64 pivot = a[k][k];
        det = det * pivot % mod;
        i64 inv = matrix_pow_mod(pivot, mod - 2, mod);
        for (int i = k + 1; i < n; ++ i) if (a[i][k]) {
            i64 t = a[i][k] * inv % mod;
            for (int j = k + 1; j < n; ++ j) {
                a[i][j] = (a[i][j] - t * a[k][j]) % mod;
                if (a[i][j] < 0) a[i][j] += mod;
            }
            a[i][k] = 0;
        }
    }
    return det;
}
```

任意模数行列式（无除逆元）

当模数可能为合数时，使用整数欧几里得式消元，不直接求非零主元的逆元。

```
int determinant_mod_any(vector<vector<int>> a, int mod) {
    assert(mod >= 2);
    int n = a.size();
    for (auto &row : a) {
        assert(int(row.size()) == n);
        for (int &x : row) x = (x % mod + (i64)mod) % mod;
    }
    i64 det = 1;
    for (int k = 0; k < n; ++ k) {
        for (int i = k + 1; i < n; ++ i) {
            while (a[i][k]) {
                i64 t = a[k][k] / a[i][k];
                for (int j = k; j < n; ++ j) {
                    a[k][j] = (a[k][j] - t * a[i][j]) % mod;
                    if (a[k][j] < 0) a[k][j] += mod;
                }
                swap(a[k], a[i]);
                det = (mod - det) % mod;
            }
        }
        det = det * a[k][k] % mod;
    }
    return det;
}
```

Householder QR（满列秩，高矩阵）

`HouseholderQR` 和 `householder_qr()` 是下面两个求解器的公共依赖。如果只需要求逆或行列式，整段 QR 都可删除。

```
struct HouseholderQR {
    int n, m;
    vector<vector<ld>> r, v;
    // Q^T x: 反射从前往后应用
    vector<ld> apply_qt(vector<ld> x) const {
        assert(int(x.size()) == n);
        for (int k = 0; k < m; ++ k) {
            ld dot = 0;
            for (int i = k; i < n; ++ i) dot += v[k][i - k] * x[i];
            for (int i = k; i < n; ++ i) x[i] -= 2 * dot * v[k][i - k];
        }
        return x;
    }
    // Q x: 反射从后往前应用
    vector<ld> apply_q(vector<ld> x) const {
        assert(int(x.size()) == n);
        for (int k = m - 1; k >= 0; -- k) {
            ld dot = 0;
            for (int i = k; i < n; ++ i) dot += v[k][i - k] * x[i];
            for (int i = k; i < n; ++ i) x[i] -= 2 * dot * v[k][i - k];
        }
        return x;
    }
};
optional<HouseholderQR> householder_qr(vector<vector<ld>> a,
        ld eps = 1E-12L) {
    int n = a.size(), m = n ? int(a[0].size()) : 0;
    assert(n >= m);
    for (const auto &row : a) assert(int(row.size()) == m);
    ld scale = 0;
    for (const auto &row : a) {
        for (ld x : row) scale = max(scale, abs(x));
    }
    HouseholderQR qr{n, m, {}, vector<vector<ld>>(m)};
    for (int k = 0; k < m; ++ k) {
        ld norm = 0;
        for (int i = k; i < n; ++ i) norm = hypot(norm, a[i][k]);
        if (norm <= eps * scale) return nullopt;
        ld alpha = a[k][k] >= 0 ? -norm : norm;
        vector<ld> v(n - k);
        for (int i = k; i < n; ++ i) v[i - k] = a[i][k];
        v[0] -= alpha;
        ld vn = 0;
        for (ld x : v) vn = hypot(vn, x);
        for (ld &x : v) x /= vn;
        for (int j = k; j < m; ++ j) {
            ld dot = 0;
            for (int i = k; i < n; ++ i) dot += v[i - k] * a[i][j];
            for (int i = k; i < n; ++ i) a[i][j] -= 2 * dot * v[i - k];
        }
        a[k][k] = alpha;
        for (int i = k + 1; i < n; ++ i) a[i][k] = 0;
        qr.v[k] = move(v);
    }
    qr.r = move(a);
    return qr;
}
struct LeastSquaresSolution {
    bool full_column_rank;
    vector<ld> x;
    ld residual_norm;
};
LeastSquaresSolution least_squares_qr(const vector<vector<ld>> &a,
        const vector<ld> &b, ld eps = 1E-12L) {
    int n = a.size(), m = n ? int(a[0].size()) : 0;
    assert(int(b.size()) == n && n >= m);
    auto qr = householder_qr(a, eps);
    if (!qr) return {false, {}, 0};
    vector<ld> y = qr->apply_qt(b), x(m);
```

```
    for (int i = m - 1; i >= 0; -- i) {
        ld s = y[i];
        for (int j = i + 1; j < m; ++ j) s -= qr->r[i][j] * x[j];
        x[i] = s / qr->r[i][i];
    }
    ld residual = 0;
    for (int i = m; i < n; ++ i) residual = hypot(residual, y[i]);
    return {true, move(x), residual};
}
struct MinimumNormSolution {
    bool full_row_rank;
    vector<ld> x;
};
MinimumNormSolution minimum_norm_qr(const vector<vector<ld>> &a,
        const vector<ld> &b, ld eps = 1E-12L) {
    int n = a.size(), m = n ? int(a[0].size()) : 0;
    assert(int(b.size()) == n && n <= m);
    vector at(m, vector<ld>(n));
    for (int i = 0; i < n; ++ i) {
        for (int j = 0; j < m; ++ j) at[j][i] = a[i][j];
    }
    auto qr = householder_qr(move(at), eps);
    if (!qr) return {false, {}};
    vector<ld> z(m);
    // R^T y = b, R^T 是下三角
    for (int i = 0; i < n; ++ i) {
        ld s = b[i];
        for (int j = 0; j < i; ++ j) s -= qr->r[j][i] * z[j];
        z[i] = s / qr->r[i][i];
    }
    return {true, qr->apply_q(move(z))};
}
```

接口说明

接口	功能	前提 / 失败含义
`inverse_real(a,eps)`	求实数逆矩阵	方阵； `nullopt` = 当前阈值下无法判定可逆
`determinant_real(a,eps)`	实数行列式	方阵，若主元过小返回 `0`
`matrix_pow_mod(a,b,p)`	模快速幂	`0≤a<mod`， `b≥0`， `mod≥2`
`inverse_mod_prime(a,p)`	素数模逆矩阵	奇异时 `nullopt`
`determinant_mod_prime(a,p)`	素数模行列式	`p` 必须是质数
`determinant_mod_any(a,m)`	任意模数行列式	`m≥2`，不依赖逆元
`householder_qr(a,eps)`	高矩阵 Householder QR	`n≥m` 且数值上满列秩；否则 `nullopt`
`least_squares_qr(a,b,eps)`	最小化 $\|Ax-b\|_2$	`n≥m`，必须满列秩
`minimum_norm_qr(a,b,eps)`	在 $Ax=b$ 中最小化 $\|x\|_2$	`n≤m`，必须满行秩

`LeastSquaresSolution::residual_norm` 是残差的二范数；求解失败时 `x` 为空，残差无效。所有输入矩阵按值或 `const&` 传递，不修改调用者原数据。

注意事项

- **求解 `Ax=b` 不应先求逆矩阵。** 若只是解方程，直接用 11.1 的高斯消元。
- 浮点逆矩阵、行列式的 `eps` 是绝对判零阈值；QR 的 `eps` 按输入最大元素的尺度进行近似判秩。所有浮点结果都应结合输入尺度、残差检验；病态矩阵仍可能误差较大。
- 素数模实现要求 `2 ≤ mod ≤ INT_MAX` 且模数为**质数**。模合数时求行列式可使用 `determinant_mod_any()`；模合数的逆矩阵不能直接按素数模算法求。
- 空方阵的行列式定义为 `1`，逆矩阵返回空矩阵； `determinant_mod_any()` 对模 `m` 返回 `1 % m`。
- 不要把最小二乘、最小范数混淆：高斯消元中"自由变量置零"未必是最小范数解。例如 $x+y=1$ 的最小范数解是 $(1/2,1/2)$。
- QR 本版只处理数值上**满秩**的对应矩阵：一般秩亏最小二乘、伪逆及最小范数最小二乘解需要带列主元 QR 或 SVD。
- `n=0` 时仅接受零列数的空矩阵；无法从空 `vector` 推断正的变量数。QR 满列秩还需确保输入中没有 `NaN/Inf`。
- **仅看行列式是否靠近零**不适合作为一般数值判秩方法；数量级过大或过小时，行列式本身可能上下溢。
- 若对整数矩阵要求**精确整数值**的行列式，不能由浮点结果四舍五入保证正确；应考虑 Bareiss（中间结果需防溢）或多模数 CRT。

可直接复用的恒等式

- 满列秩最小二乘：$A^TAx=A^Tb$；满行秩最小范数：$x=A^T(AA^T)^{-1}b$。这两式仅用于推导，数值计算建议采用 QR，因为 $\kappa(A^TA)=\kappa(A)^2$（二范数条件数）。
- 矩阵行列式引理（域上 A 可逆）：$\det(A+uv^T)=\det(A)(1+v^TA^{-1}u)$。

- Sherman–Morrison：若 $1+v^TA^{-1}u\neq 0$，则 $(A+uv^T)^{-1}=A^{-1}-\dfrac{A^{-1}uv^TA^{-1}}{1+v^TA^{-1}u}$。更新逆矩阵时应先保存原矩阵对应的两侧向量。

常见用法

模行列式、矩阵树依赖

```
int d = determinant_mod_prime({{1, 2}, {3, 4}}, 998244353); // p-2
int d2 = determinant_mod_any({{1, 2}, {3, 4}}, 1000);      // 998
```

实数最小二乘：拟合直线 `y = ax+b`

```
vector<vector<ld>> a(n, vector<ld>(2));
vector<ld> b(n);
for (int i = 0; i < n; ++ i) {
    a[i] = {xs[i], 1};
    b[i] = ys[i];
}
auto sol = least_squares_qr(a, b);
if (sol.full_column_rank) {
    ld slope = sol.x[0], intercept = sol.x[1];
    ld error = sol.residual_norm;
}
```

最小范数解

```
vector<vector<ld>> a{{1, 1}};
vector<ld> b{1};
auto sol = minimum_norm_qr(a, b);
// sol.x 近似为 {0.5, 0.5}
```

复杂度

设 A 为 $n\times m$，其中行列式与求逆时 $n=m$

接口	时间	额外空间
实数 / 素数模求逆	$O(n^3+n\log p)$（实数无幂项）	$O(n^2)$
实数 / 素数模行列式	$O(n^3+n\log p)$	$O(n^2)$
任意模数行列式	$O(n^3\log mod)$（上界）	$O(n^2)$
Householder QR、最小二乘（$n\geq m$）	$O(nm^2)$	$O(nm)$
最小范数（$n\leq m$，对 A^T 分解）	$O(mn^2)$	$O(mn)$